ZYNQMP 开发平台 Linux 驱动教程

2022/10/17 09:26:59

ALINX

版权声明

Copyright ©2012-2019 芯驿电子科技（上海）有限公司

公司网址:
Http://www.alinx.com.cn

技术论坛:
http://www.heijin.org

官方旗舰店:
http://alinx.jd.com

邮箱:
avic@alinx.com.cn

电话:
021-67676997

传真:
021-37737073

ALINX 微信公众号:



我们承诺本教程并非一劳永逸，固守不变的文档。我们会根据论坛上大家的反馈意见，以及实际的开发实践经验积累不断的修正和优化教程

文档修订记录:

版本	时间	描述
1.0	2022/10/13	初始版本

前言

首先感谢大家购买芯驿电子科技（上海）有限公司出品的 ZYNQMP 的开发板！您对我们和我们产品的支持和信任,给我们增添了永往直前的信心和勇气。

“播下一粒种子，收获一片森林”，更是芯驿电子科技（上海）有限公司的美好愿望，同时我们会在黑金动力社区 *http://www.heijin.org* 和大家一起讨论，一起学习，一起进步，一起成长。

通过前面的教程我们熟悉了 ZYNQMP 开发的基本流程，学习了 SDK 下 ZYNQMP 程序的开发，但是很多应用可能需要使用嵌入式 Linux，由于 Linux 系统庞大，芯驿电子科技（上海）有限公司只能抛砖引玉，把最常用的知识点收集分享给大家。

目录

版权声明 2
前言 4
目录 5
第一章 字符设备 10
1.1 Linux 驱动程序概述 10
1.1.1 Linux 应用程序与驱动程序的关系 10
1.1.2 Linux 驱动程序的分类 10
1.2 字符设备的开发步骤 11
1.2.1 实现系统调用 11
1.2.2 驱动模块的加载与卸载 12
1.2.3 字符设备的注册与注销 12
1.2.4 Linux 设备号 13
1.2.5 实现设备操作函数 13
1.2.6 添加驱动描述信息 13
1.3 字符设备驱动开发实验 14
1.3.1 查看硬件原理图以及数据手册 14
1.3.2 编写字符设备驱动程序 15
1.3.3 编译驱动程序 17
1.3.4 使用 Ubuntu 虚拟机中的 QT 编写测试 APP 21
1.3.5 运行测试 26
第二章 字符设备的新写法 28
2.1 设备号管理 28
2.2 新的注册方法 29
2.2.1 字符设备数据结构 29
2.2.2 cdev 结构体初始化 29
2.2.3 新的注册和注销函数 29
2.3 自动创建设备文件 30
2.3.1 mdev 30
2.3.2 类的创建和删除 30
2.2.3 设备节点的创建和删除 31
2.4 字符设备新驱动实验 32
2.4.1 编写字符设备驱动程序 32
2.4.2 编译驱动程序 34
2.4.3 编写测试 APP 34
2.4.4 运行测试 34
第三章 设备树和 of 函数 35

3.1 设备树 35
3.1.1 DTS、 DTB 和 DTSI 35
3.1.2 节点 36
3.1.3 属性 36
3.1.4 在文件系统中查看设备树 38
3.1.5 petalinux 中的设备树 38
3.2 of 函数 39
3.2.1 查找节点的 of 函数 39
3.2.2 提取属性的 of 函数 40
3.3 设备树下的 led 驱动实验 41
3.3.1 修改设备树 42
3.3.2 驱动程序 43
3.3.4 测试程序 45
3.3.5 运行测试 45
第四章 pinctrl 和 gpio 子系统 46
4.1 pinctrl 子系统 46
4.1.1 zynqmp 平台 pinctrl 子系统的设备树节点和属性 46
4.1.2 pinctrl 子系统的使用 48
4.2 gpio 子系统 48
4.2.1 设备树中的 gpio 子系统 49
4.2.2 gpio 接口函数 49
4.3 实验 50
4.3.1 设备树 50
4.3.2 驱动代码 51
4.3.3 测试代码 53
4.3.4 运行测试 53
第五章 并发的处理 54
5.3Linux 对并发的处理 54
5.3.1 原子操作 54
5.3.2 锁机制 55
5.3.3 信号量 56
5.4 实验 57
5.4.1 驱动代码 57
5.4.2 测试代码 61
5.4.3 运行测试 61
第六章 定时器 63
6.1 Linux 内核中的定时器 63
6.1.1 系统频率 63

6.1.2 节拍数 64
6.1.3 内核定时器及相关函数 65
6.2 实验 66
6.2.1 原理图 ..... 错误！未定义书签。
6.2.2 设备树 ..... 错误！未定义书签。
6.2.3 驱动程序 66
6.2.4 测试程序 69
6.3.5 运行测试 70
第七章 platform 平台设备 71
7.1 驱动分离 71
7.2 platform 模型 72
7.2.1 platform_bus 73
7.2.2 platform_driver 73
7.2.3 platform_device 74
7.3 实验 75
7.3.1 原理图 ..... 错误！未定义书签。
7.3.2 设备树 ..... 错误！未定义书签。
7.3.3 驱动程序 75
7.3.4 测试程序 79
7.3.5 运行测试 79
第八章 platform 与设备树 80
8.1 设备树下的 platform 80
8.2 实验 80
8.2.1 原理图 81
8.2.2 设备树 81
8.2.3 驱动程序 81
8.2.4 测试程序 81
8.2.5 运行测试 83
第九章 MISC 设备驱动 85
9.1 MISC 设备简介 85
9.2 MISC 框架的使用 85
9.3 实验 86
9.3.1 原理图 86
9.3.2 设备树 ..... 错误！未定义书签。
9.3.3 驱动代码 86
9.3.4 测试代码 88
9.3.5 运行测试 88
第十章 pwm 驱动 89
10.1 zynqmp 上的 pwm 实现 89

10.1.1 修改 vivado 工程 89
10.1.2 控制 pwm 输出的方法 91
10.2 Linux 中的 pwm 框架 92
10.2.1 pwm_device 92
10.2.1 pwm_chip 93
10.2.1 示例 94
10.3 实验 96
10.3.1 原理图 96
10.3.2 设备树 96
10.3.3 驱动程序 97
10.3.4 测试程序 ..... **错误！未定义书签。**
10.3.5 运行测试 99
第十一章 I2C 驱动 101
11.1 I2C 驱动框架 101
11.1.1 I2C 控制器驱动 101
11.1.2 I2C 设备驱动 102
11.1.3 I2C 设备驱动的实现流程 104
11.3 实验 106
11.3.1 原理图 106
11.3.2 设备树 106
11.3.3 驱动程序 107
11.3.4 测试程序 110
11.3.5 运行测试 110
第十二章 SPI 驱动 112
12.1 SPI 控制器驱动 112
12.2 SPI 设备驱动 112
12.3 SPI 设备驱动中数据收发处理 115
12.4 实验 116
12.4.1 原理图 116
12.4.2 设备树 117
12.4.3 驱动程序 117
12.4.4 测试程序 121
12.4.5 运行测试 121
第十三章 串口驱动 122
13.1 uart 驱动框架 122
13.2 xilinx 的 uart 驱动 124
第十四章 块设备驱动 129
14.1 块设备简介 129
14.2 内核中的块设备访问 129

14.3 块设备框架 131
14.3.1 gendisk 结构体 132
14.3.2 request_queue 133
14.4 实验 133
14.1.1 驱动程序 133
14.1.2 运行测试 ..... **错误！未定义书签。**
第十五章 网卡驱动 137
15.1 Linux 网卡驱动 137
15.1.1 初始化 137
15.1.2 发包 139
15.1.3 收包 141
15.2 实验 141
15.2.1 驱动程序 141
15.2.2 运行测试 143
第十六章 DMA 驱动 144
16.1 DMA 简介 144
16.2 DMAC 144
16.2.1DMA channels 144
16.2.2 DMA request line 144
16.2.3 transfer size、 transfer wide、 burst size 144
16.2.4 scatter-gather 144
16.4 linux 中的 DMA 框架 145
16.4.1 DMA controller 驱动框架 145
16.4.2 DMA client 驱动框架 149

第一章 字符设备

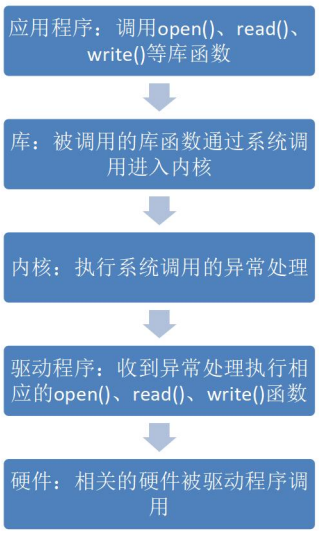
这一章我们使用的 petalinux 工程是“Linux 基础教程”中“第一章 使用 Petalinux 定制 Linux 系统”制作的。

1.1 Linux 驱动程序概述

在动手写程序之前，先了解一下基本的 Linux 设备及驱动程序相关的知识。

1.1.1 Linux 应用程序与驱动程序的关系

Linux 应用程序调用驱动程序的步骤大致如下：



①应用程序调用库函数提供的 open()函数打开某个设备文件；

②库根据 open()函数的输入参数引起 CPU 异常，进入内核；

③内核的异常处理函数根据输入参数找到相应的驱动程序，返回文件句柄给库，库函数再返回给应用程序；

④应用程序再使用得到的文件句柄调用 write()、read()等函数发出控制指令；

⑤库根据 write()、read()等函数的输入参数引起 CPU 异常，进入内核；

⑥内核的异常处理函数根据输入参数调用相应的驱动程序执行相应的操作。

步骤一中 open()函数打开的设备文件，是在驱动加载成功后再目录”/dev”中生成的相关文件，是应用程序调用对应硬件的入口。

在这个过程中应用程序涉及到的 open()、read()、write()等由库提供的接口函数成为系统调用，他们通过执行某条指令引发异常进入内核，是应用程序操作硬件沟通的途径。应用程序执行系统调用后，最终会使用驱动函数中对应的函数，而驱动函数中的 open()、read()、write()等函数则需要驱动开发人员去实现。

应用程序运行于用户空间，驱动程序运行于内核空间。Linux 系统可以通过 MMU 限制应用程序运行在某个内存块中，以避免这个应用程序出错导致整个系统崩溃。而运行于内核空间的驱动程序是系统的一部分，驱动程序出错有可能牵连整个系统，因此在实现驱动时要格外小心。

1.1.2 Linux 驱动程序的分类

驱动可分为三类：字符设备驱动、块设备驱动、网络设备驱动。

字符设备即是按字节访问的设备。比如以一个字节的收发数据的串口，就是典型的字符设备。还有较为简单简单的我们接下来会讲的 LED 灯驱动，相对复杂的如音频都属于字符设备，字符设备在 Linux 外设中占比最大。

块设备的特点是他们是按一定格式存取数据，具体的格式有文件系统决定。他们通常以存储设备为主如 EMMC、FLASH、SD 卡、EEPROM 等。存储设备的特点是以存储块为基础，因此得名为块设备。

对于应用程序来说，块设备和字符设备访问方式基本无差，即驱动程序实现系统调用，应用程序通过设备文件（比如/dev/mtd0、/dev/i2c-0 等）来调用对应的库函数。当然这需要驱动程序开发者来保证，即再实现块设备驱动时将数据按一定格式组织成块后再进行读写。另外，块设备还需要向内核其他部件提供一些接口，使得块设备上可以存放文件系统挂接块设备。因此块设备驱动的实现相比字符设备驱动要复杂得多。

网络设备比较特殊，应用程序和网络设备驱动间的通信与上面两种设备完全不同，由库和内核提供的一套数据包传输函数替代了 open()、read()、write()等函数。但是网络设备也很好理解和辨认，WIFI 或者有线接口都属于网络设备。

设备驱动中还会出现一个驱动包含多种设备驱动的情况，如 SPI（字符设备）连接的 FLASH（块设备）、USB 接口（字符设备）的 WIFI（网络设备）。接下来我们会循序渐进的学习各种设备的驱动，由浅入深，先从最简单的字符设备开始。

1.2 字符设备的开发步骤

Linux 驱动程序是有框架的，现有框架方便的我们增加新的驱动程序，熟悉现有框架之后，在现有驱动程序上稍微修改就能得到新设备的驱动，而学习现有框架就是我们的目标之一。

熟悉框架之后，实现设备驱动的步骤大致如下：

①查看原理图以及数据手册了解设备操作方法；

②修改设备树文件；

③套用与设备相近的框架，或找到内核中相似设备的驱动代码直接修改，实现驱动程序的初始化以及操作函数；

④将驱动编译进内核或单独编译加载驱动；

⑤编写应用程序测试驱动程序。

不难看出上述步骤中重点以及难点是步骤③，下面我们细说字符设备驱动中步骤③中需要实现的内容。

1.2.1 实现系统调用

系统调用即设备操作函数，是字符设备驱动的核心。在内核文件 include/linux/fs.h 中定义了数据结构 file_operations，涵括了所有内核驱动操作函数，内容如下：

```
1.  struct file_operations {
2.      struct module *owner;
3.      loff_t (*llseek) (struct file *, loff_t, int);
4.      ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
5.      ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
6.      ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
```

```
7.      ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
8.      int (*iterate) (struct file *, struct dir_context *);
9.      int (*iterate_shared) (struct file *, struct dir_context *);
10.     unsigned int (*poll) (struct file *, struct poll_table_struct *);
11.     long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
12.     long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
13.     int (*mmap) (struct file *, struct vm_area_struct *);
14.     int (*open) (struct inode *, struct file *);
15.     int (*flush) (struct file *, fl_owner_t id);
16.     int (*release) (struct inode *, struct file *);
17.     int (*fsync) (struct file *, loff_t, loff_t, int datasync);
18.     int (*fasync) (int, struct file *, int);
19.     int (*lock) (struct file *, int, struct file_lock *);
20.     ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t *, int);
21.     unsigned long (*get_unmapped_area)(struct file *, unsigned long, unsigned long, unsigned lon
    g, unsigned long);
22.     int (*check_flags)(int);
23.     int (*flock) (struct file *, int, struct file_lock *);
24.     ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t *, size_t, unsigned
    int);
25.     ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info *, size_t, unsigned i
    nt);
26.     int (*setlease)(struct file *, long, struct file_lock **, void **);
27.     long (*fallocate)(struct file *file, int mode, loff_t offset,
28.               loff_t len);
29.     void (*show_fdinfo)(struct seq_file *m, struct file *f);
30. #ifndef CONFIG_MMU
31.     unsigned (*mmap_capabilities)(struct file *);
32. #endif
33.     ssize_t (*copy_file_range)(struct file *, loff_t, struct file *,
34.             loff_t, size_t, unsigned int);
35.     int (*clone_file_range)(struct file *, loff_t, struct file *, loff_t,
36.             u64);
37.     ssize_t (*dedupe_file_range)(struct file *, u64, u64, struct file *,
38.             u64);
39. };
```

简单介绍一下接下来的实验中会用到的成员和函数。

第 2 行 owner 一般设置为 THIS_MODULE。

第 4 行 read()函数用于从设备读取数据。

第 5 行 write()函数用于向设备写入或发送数据。

第 14 行 open()函数已经提到多次，用于打开设备文件。

第 16 行 release()函数用于关闭设备文件，与应用程序中的 close()函数相对应。

1.2.2 驱动模块的加载与卸载

驱动程序完成后，可以选择编译进内核，或单独编译后使用命令 insmod 加载到系统中。后者对于驱动开发而言方便很多。驱动程序单独编译后，会得到后缀为.ko 的驱动模块文件，比如加载名为 led.ko 的驱动模块，就使用命令 insmod led.ko 即可。当然有加载就有卸载，卸载命令为 rmmod，卸载驱动与加载驱动稍有区别，不是 rmmod led.ko 了。可以先使用 lsmod 命令来查看现已加载的驱动，再使用 rmmod 命令加空格再加上 lsmod 命令罗列出的你想卸载的驱动名称即可。

在实现驱动时，需要注册这两个操作函数，注册函数为：

module_init(xxx_init);　　//注册模块加载函数

module_exit(xxx_exit);　　//注册模块卸载函数

xxx_init 是驱动入口函数，xxx_exit 是驱动出口函数。当执行 insmod 命令时，xxx_init 函数就会被调用。xxx_exit 函数同理。

1.2.3 字符设备的注册与注销

驱动入口函数是驱动加载时执行的第一个函数，我们需要抓住这个机会在入口函数中实现

驱动的初始化即注册字符设备。注册和注销字符设备的函数分别为：

```
int register_chrdev(unsigned int major, const char *name, const structfile_operations *fops);
void unregister_chrdev(unsigned int major, const char *name);
```

两个函数总共涉及到三个输入参数：

major：主设备号，linux 系统下每个设备都有一个设备号，主设备下还有子设备号 minor；

name：设备名；

fops：file_operations 型指针，设备操作函数合集变量。

1.2.4 Linux 设备号

每个设备文件都有主次设备号 major，主设备号是唯一的，每个主设备下有次设备号 minor，次设备号在这个主设备下也是唯一的。在 Linux 系统下输入 cat /proc/devices 命令可以查看已被注册的主设备号。

1.2.5 实现设备操作函数

在接下来的点亮 led 实验中，会用到那些设备操作函数呢，先看一下点亮 led 的步骤。

首先我们要初始化 led 所连接的 IO，使能时钟、设置 IO 为输出等，open()函数是应用程序关联设备文件的开始，设备的初始化可以放在 open()函数中。实现了 open()函数必然就要实现对应的 release()函数。IO 初始化完成后，我们需要点亮 led，实际上就是往 IO 对应的寄存器内写入对应的值，写入自然就是实现 write()函数了。相应的，也可以实现 read()函数来读取 IO 当前的状态。

1.2.6 添加驱动描述信息

在驱动代码中我们需要添加 LICENSE 信息和作者信息等驱动描述信息，其中 LICENSE 是必须添加的，方法如下：

```
MODULE_LICENSE("GPL");
```

现在我们已经知道了注册驱动入口和出口函数、注册和注销字符设备、需要实现的设备操作函数，大致可以写出如下的框架：

```
1.  /* 驱动名称 */
2.  #define DEVICE_NAME       "gpio_leds"
3.  /* 驱动主设备号 */
4.  #define GPIO_LED_MAJOR    200
5.
6.  /* open 函数实现，对应到 Linux 系统调用函数的 open 函数 */
7.  static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
8.  {
9.      return 0;
10. }
11.
12. /* write 函数实现，对应到 Linux 系统调用函数的 write 函数 */
13. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size_t len, loff_t *
    loff_t_p)
14. {
15.     return 0;
16. }
17.
18. /* release 函数实现，对应到 Linux 系统调用函数的 close 函数 */
19. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
20. {
21.     return 0;
22. }
23.
24. /* file_operations 结构体声明，是上面 open、write 实现函数与系统调用函数对应的关键 */
25. static struct file_operations gpio_leds_fops = {
26.     .owner    =   THIS_MODULE,
27.     .open     =   gpio_leds_open,
```

```
    .write   =   gpio_leds_write,
    .release =   gpio_leds_release,
};

/* 模块加载时会调用的函数 */
static int __init gpio_led_init(void)
{
    int ret;

    /* 通过模块主设备号、名称、模块带有的功能函数(及 file_operations 结构体)来注册模块 */
    ret = register_chrdev(GPIO_LED_MAJOR, DEVICE_NAME, &gpio_leds_fops);
    if (ret < 0)
    {
        return ret;
    }
    else
    {

    }
    return 0;
}

/* 卸载模块 */
static void __exit gpio_led_exit(void)
{
    /* 注销模块，释放模块对这个设备号和名称的占用 */
    unregister_chrdev(GPIO_LED_MAJOR, DEVICE_NAME);
}

/* 注册模块入口和出口函数 */
module_init(gpio_led_init);
module_exit(gpio_led_exit);

/* 添加 LICENSE 信息 */
MODULE_LICENSE("GPL");
```

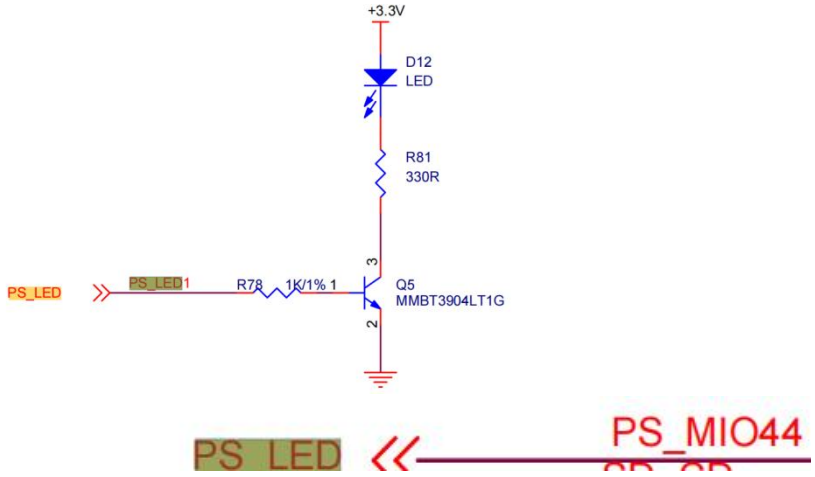
这里我把驱动命名为"gpio_leds"。设备号设置为 200，我这里 200 号设备号没有被占用，要根据自己的板载系统实际情况更改。

1.3 字符设备驱动开发实验

现在我们已经大致了解了字符设备的实现方法，接下来就一步一步的去实现。我们的目标是编写开发板上”PS LED”这个 led 的设备驱动，通过驱动程序能点亮、熄灭这个 led。

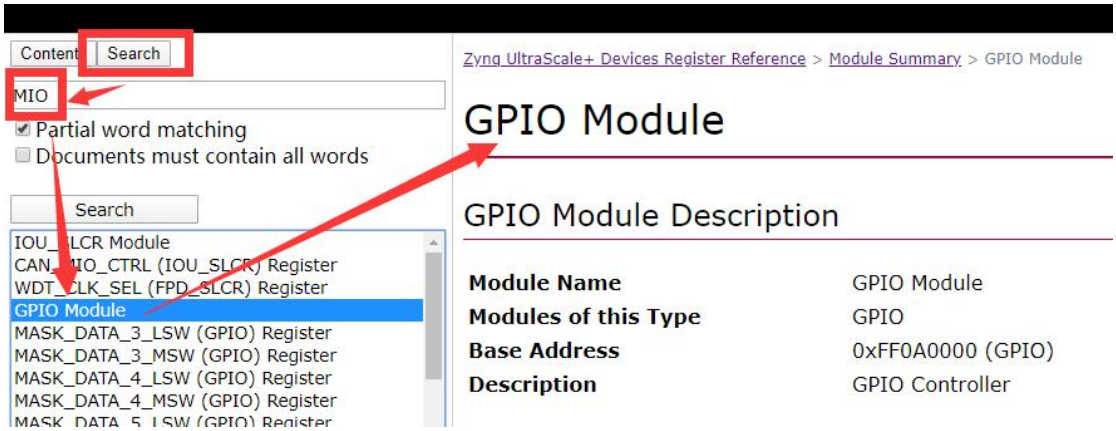
1.3.1 查看硬件原理图以及数据手册

打开 AXUZ7-P 开发板原理图，查看 PS LED 的连接方式。



可知目标 led 连接在开发板 PS 端的 MIO44 引脚上。

再打开 Ultra scale+的寄存器手册《ug1087-zynq-ultrascale-registers》(是一个 HTML 文件包，在 xilinx 官网搜索 ug1087 就可以找到)。搜索关键词 MIO，找到 GPIO Module。



GPIO 寄存器的基地址为 0xFF0A0000，控制 GPIO 我们需要三步：使能、设置方向、控制输出。MIO44 在 GPIO BANK1 中。对应找到使能寄存器 OEN_1：0xFF0A0248、方向寄存器 DIRM_1：0xFF0A0244、控制寄存器 DATA_1：0xFF0A0044，驱动程序第 85 行，因为 MIO 是 44，所以手册里从 MIO26 开始，MIO44 是 01000000000000000000，转换十进制就是 40000，同理，第 52，86 行都是一样的；第 56 行同理。

1.3.2 编写字符设备驱动程序

上面我们已经得到了 led 驱动的大体框架，接下来要做的实际就是完善设备操作函数中的内容，也是再对应的函数中设置相关的寄存器。在 open()函数中实现 led 初始化，在 write()函数中实现 led 的控制，在 release()函数中完成 led 的使能。最终的代码如下：

```
1.  #include <linux/module.h>
2.  #include <linux/kernel.h>
3.  #include <linux/fs.h>
4.  #include <linux/init.h>
5.  #include <linux/ide.h>
6.  #include <linux/types.h>
7.
8.  #define GPIO_BASE     0x00FF0A0000
9.  #define GPIO_BASE_LEN 0x300
10. static unsigned int *gpio_addr_vir = 0;
11.
12. #define GPIO_OEN1     (unsigned int *)(0x248 + (unsigned long)gpio_addr_vir)
13. #define GPIO_DIRM1    (unsigned int *)(0x244 + (unsigned long)gpio_addr_vir)
14. #define GPIO_DATA1    (unsigned int *)(0x044 + (unsigned long)gpio_addr_vir)
15.
16.
17. #define CHAR_DEV_MAJOR 200
18. #define CHAR_DEV_NAME  "led_dev"
19.
20. static int char_dev_open(struct inode *inode_p, struct file *file_p)
21. {
22.     printk(KERN_ERR"char dev open\n");
23.     return 0;
24. }
25.
26. static int char_dev_release(struct inode *inode_p, struct file *file_p)
27. {
28.     printk("char dev release\n");
29.     return 0;
30. }
31.
32. static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t *l
    off_t_p)
33. {
34.     int ret = 0;
35.     char lc_buf = 0;
```

```
36.
37.     ret = copy_from_user(&lc_buf, buf, len);
38.     if(ret != 0)
39.     {
40.         printk("copy_from_user failed\n");
41.         return -1;
42.     }
43.
44.     if(len != 1)
45.     {
46.         printk("len err\n");
47.         return -2;
48.     }
49.
50.     if(1 == lc_buf)
51.     {
52.         *GPIO_DATA1 |= 0x00000020;
53.     }
54.     else if(0 == lc_buf)
55.     {
56.         *GPIO_DATA1 &= 0xFFFFFFDF;
57.     }
58.     else
59.     {
60.         printk("data err\n");
61.     }
62.
63.     return 0;
64. }
65.
66. static struct file_operations char_dev_opt = {
67.     .owner = THIS_MODULE,
68.     .open = char_dev_open,
69.     .write = char_dev_write,
70.     .release = char_dev_release,
71. };
72.
73. static int __init char_drv_init(void)
74. {
75.     int ret = 0;
76.
77.     ret = register_chrdev(CHAR_DEV_MAJOR, CHAR_DEV_NAME, &char_dev_opt);
78.     if(ret < 0)
79.     {
80.
81.     }
82.     else
83.     {
84.         gpio_addr_vir = ioremap_wc(GPIO_BASE, GPIO_BASE_LEN);
85.         *GPIO_DIRM1 |= 0x00000020;
86.         *GPIO_OEN1  |= 0x00000020;
87.     }
88.
89.     return ret;
90. }
91.
92. static void __exit char_drv_exit(void)
93. {
94.     iounmap(gpio_addr_vir);
95.     unregister_chrdev(CHAR_DEV_MAJOR, CHAR_DEV_NAME);
96. }
97.
98.
99.
100. module_init(char_drv_init);
101. module_exit(char_drv_exit);
102.
103. MODULE_AUTHOR("Alinx");
104. MODULE_ALIAS("gpio_led");
105. MODULE_DESCRIPTION("GPIO LED driver");
106. MODULE_VERSION("v1.0");
107. MODULE_LICENSE("GPL");
```

22 行出现的 printk()函数，是内核态输出字符串到控制台的函数，这里用于调试，相当于应用程序中的 printf()。printk()函数存在消息级别，定义在头文件 include/linux/kern_levels.h 中，如

下：

```
1.  #define KERN_EMERG  KERN_SOH "" /* system is unusable */
2.  #define KERN_ALERT  KERN_SOH "" /* action must be taken immediately */
3.  #define KERN_CRIT   KERN_SOH "" /* critical conditions */
4.  #define KERN_ERR    KERN_SOH "" /* error conditions */
5.  #define KERN_WARNING    KERN_SOH "" /* warning conditions */
6.  #define KERN_NOTICE KERN_SOH "" /* normal but significant condition */
7.  #define KERN_INFO   KERN_SOH "" /* informational */
8.  #define KERN_DEBUG  KERN_SOH "" /* debug-level messages */
```

其中 0 的优先级最高， 7 的优先级最低。如果要设置消息级别，可设置如下:

printk(KERN_INFO"gpio_test module open\n");

如果不设置消息级别，那么 printk()会采用默认级别 MESSAGE_LOGLEVEL_DEFAULT，默认级别为 4。只有消息级别大于头文件 include/linux/printk.h 中定义的宏 CONSOLE_LOGLEVEL_DEFAULT，消息才会被打印，CONSOLE_LOGLEVEL_DEFAULT 值为 7.

84 行出现的函数 ioremap_wc()，用于把物理地址映射到虚拟地址。在 Linux 中由于 MMU 内存映射的关系，我们无法直接操作物理地址，而需要吧物理地址映射到虚拟地址上再操作对应的虚拟地址。注意一点，AXUZ7-P 是 64 位 SoC，我们这里运行的 linux 系统也是 64 位的，64 位系统中使用 ioremap_wc()，32 位系统中使用 oremap()。指针大小也是 64 位，需要使用 unsignned long 类型来获取。

ioremap()定义在头文件 arch/arm/include/asm/io.h 中，如下:

#define ioremap(cookie,size) __arm_ioremap((cookie), (size),MT_DEVICE)

cookie 指代物理地址，size 为需要映射的地址长度。115 行的代码即是把 GPIO 寄存器基地址，范围是整个 GPIO 寄存器映射到虚拟地址并赋值给全局变量 gpio_add_minor，之后可以直接对变量 gpio_add_minor 进行读写操作。

94 行的 iounmap()函数是与 ioremap()相对的释放虚拟地址函数。输入参数为 ioremap()函数返回得到的虚拟地址首地址，比如这里的 gpio_add_minor。

剩下的就是寄存器操作了。对于虚拟地址的操作，我们这里都是直接通过指针访问的，但是 Linux 有推荐的读写方法，而不是使用指针，如下:

读函数:

u8 readb(const volatile void __iomem *addr);
u16 readw(const volatile void __iomem *addr);
u32 readl(const volatile void __iomem *addr);

写函数

void writeb(u8 value, volatile void __iomem *addr);
void writew(u16 value, volatile void __iomem *addr);
void writel(u32 value, volatile void __iomem *addr);

value 为需要写入的值，addr 为需要操作的地址。

32 行的 write()函数中通过判断用户输入的 buf 值为 0 还是 1 相应的执行点灯和灭灯的操作。

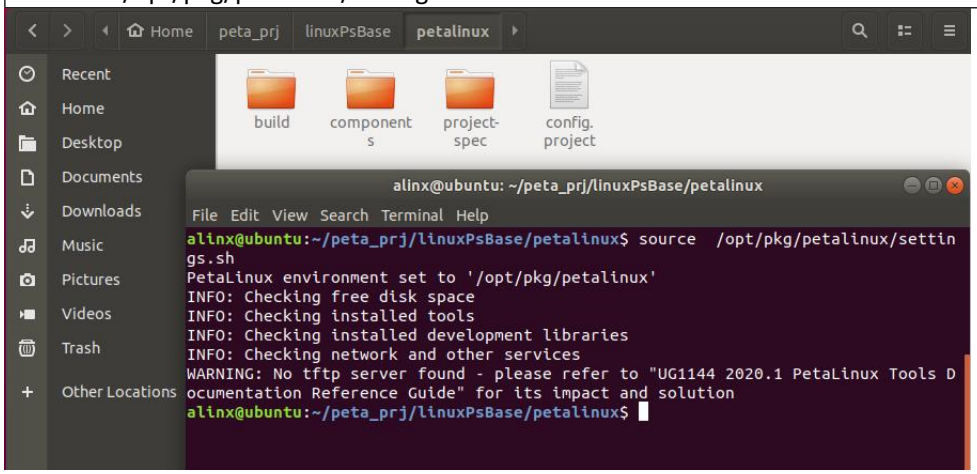
1.3.3 编译驱动程序

编译驱动程序这里介绍两种方法。第一种是使用 petalinux 工具创建驱动模块，第二种是使用 makefile 单独编译驱动。

1.3.3.1 在 Petalinux 定制系统中添加新的驱动

在 petalinux 工程路径中打开终端。在终端中输入下面的命令设置 petalinux 环境变量：

```
source    /opt/pkg/petalinux/settings.sh
```

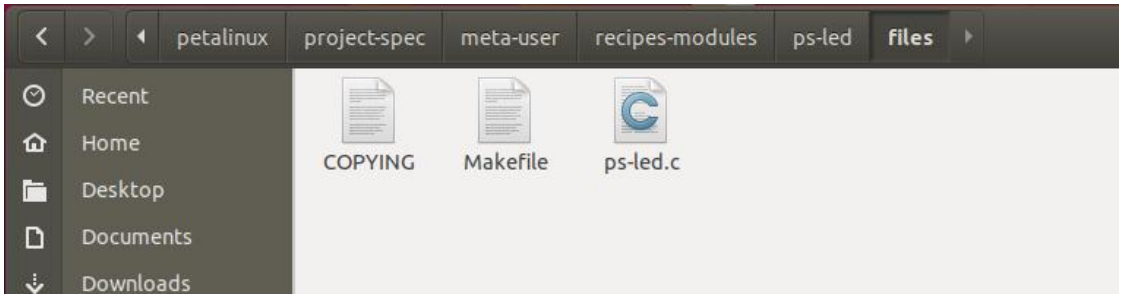


输入下面的命令添加新的驱动：

```
petalinux-create   -t   modules   --name   ps-led   --enable
```

```
alinx@ubuntu:~/peta_prj/linuxPsBase/petalinux$ petalinux-create  -t  modules  --name  ps-led  --enable
INFO: Create modules: ps-led
INFO: New modules successfully created in /home/alinx/peta_prj/linuxPsBase/petalinux/project-spec/meta-user/recipes-modules/ps-led
INFO: Enabling created component...
INFO: sourcing build environment
INFO: silentconfig rootfs
INFO: ps-led has been enabled
alinx@ubuntu:~/peta_prj/linuxPsBase/petalinux$
```

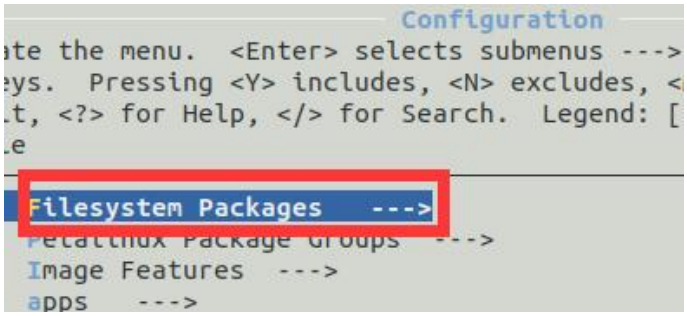
查看 petalinux 工程中目录“project-spec/meta-user/recipes-modules”，下面有了名为 ps-led 的文件夹，再进入 petalinux 工程中目录“project-spec/meta-user/recipes-modules/ps-led/files”，目录下名为 ps-led.c 的文件就是 petalinux 帮我们新建的驱动文件。



打开这个 c 文件，把 1.3.2 中编写的 led 驱动代码粘贴进去保存退出。

```
petalinux-config -c rootfs
```

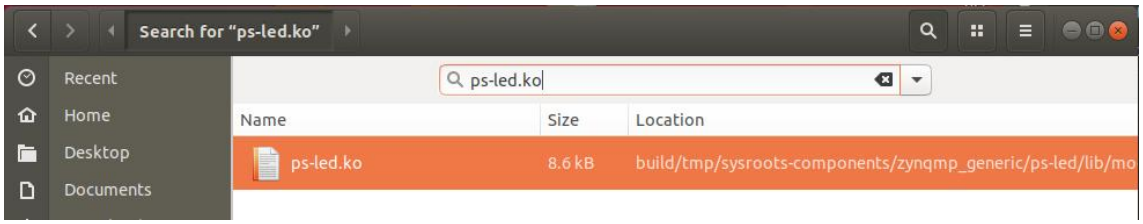
打开 rootfs，在里面添加 stdc++库：





保存退出。

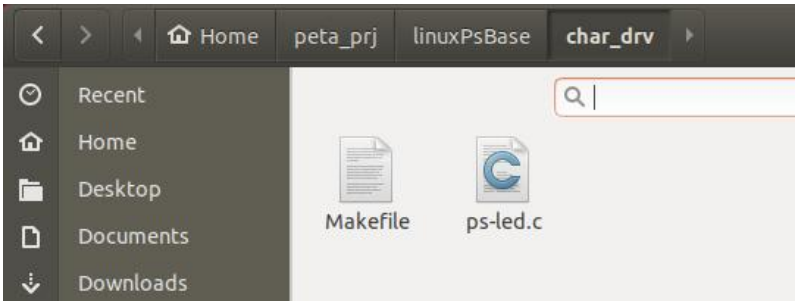
在终端输入命令 petalinux-build 编译 petalinux 工程，编译完成后在 petalinux 工程目录下搜索“ps-led.ko”，就是对应的驱动模块文件。



编译完打包 BOOT.bin 等三个文件到 SD 卡。

1.3.3.2 使用 makefile 单独编译驱动

在虚拟机中新建一个文件夹，在这个文件夹中新建一个 Makefile 文件和一个 c 文件。



打开这个 c 文件，把 1.3.2 中编写的 led 驱动代码粘贴进去保存退出。

打开 Makefile，输入下面的内容保存。

```
modname:=ps-led
obj-m:=$(modname).o
```

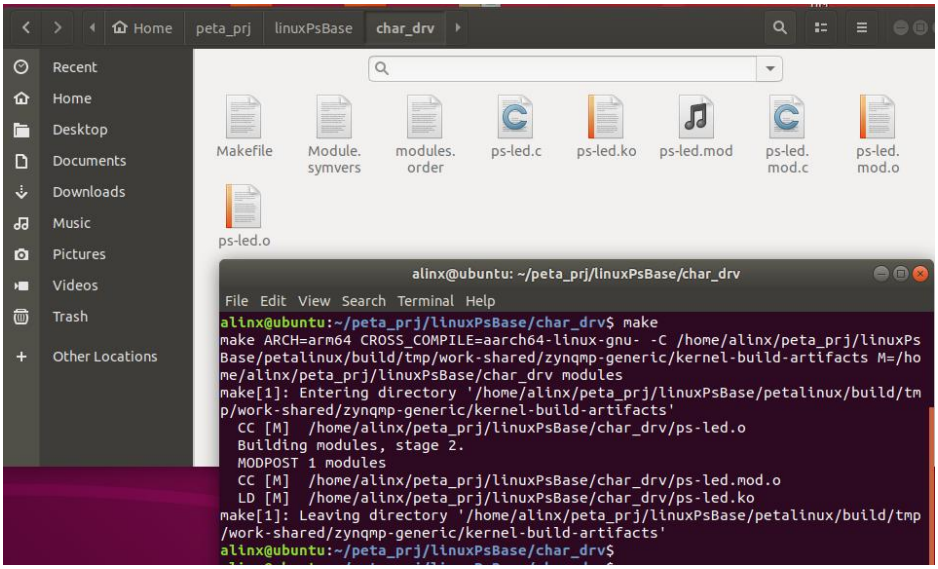
```
PWD :=$(shell pwd)
MAKE :=make
KERNELDIR =
/home/alinx/peta_prj/linuxPsBase/petalinux/build/tmp/work-shared/zynqmp-generic/kernel-build-artifacts
CROSS_COMPILE=aarch64-linux-gnu-
ARCH=arm64


all:
	$(MAKE) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE) -C $(KERNELDIR)
M=$(PWD) modules

clean:
	rm -rf $(modname).ko *.o *mod* \.*cmd *odule* .tmp_versions

.PHONY: all clean
```

然后在这个文件夹中打开一个终端，输入 make 即可编译驱动程序。“ps-led.ko”就是驱动模块。



这个 Makefile 对于不同的驱动程序和不同的 petalinux 工程，只要对应修改“modname:=”和“KERNELDIR =”两个字段后面的内容即可使用。

“modname:=”字段后面的内容是 c 文件的名字，不带后缀。

如果 make 后出现这个错误：

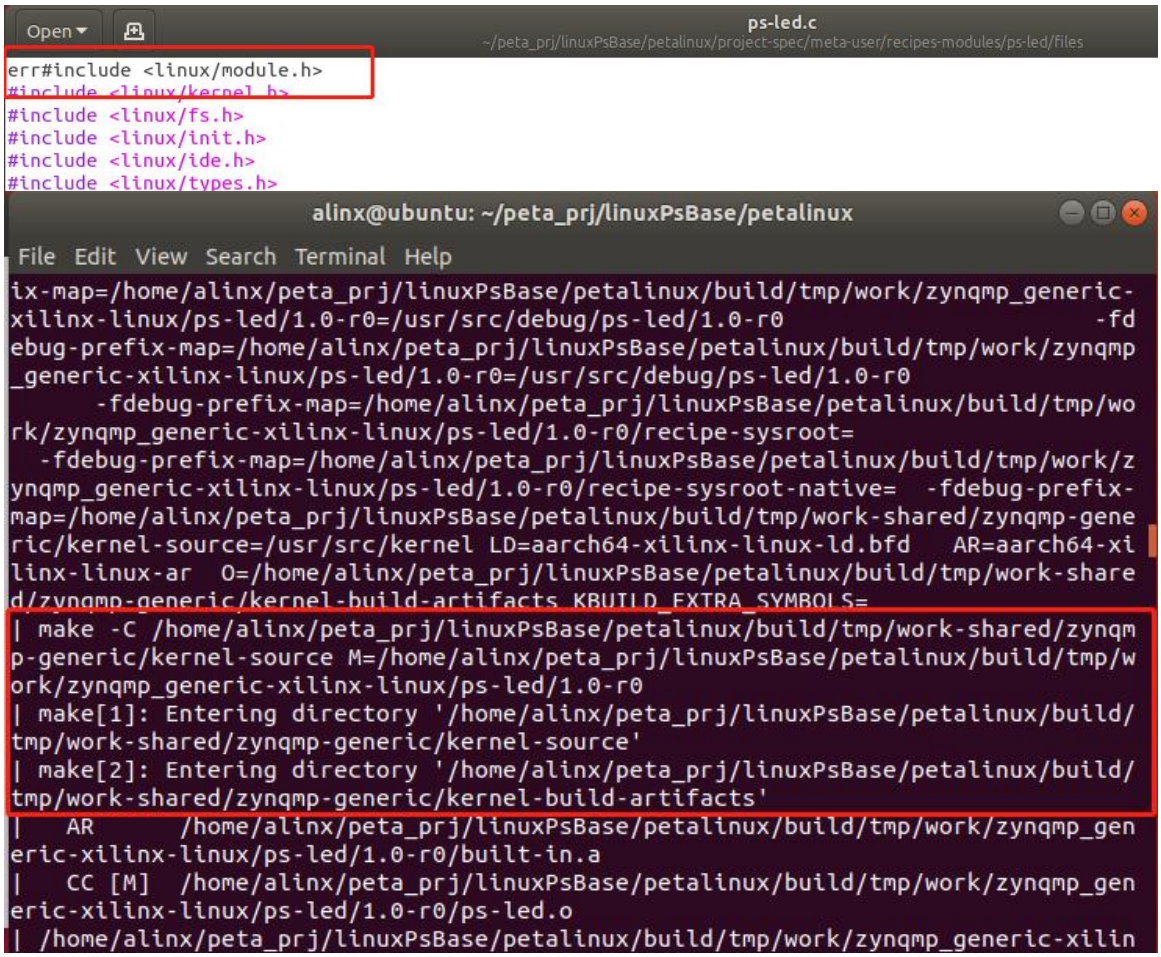
```
alinx@ubuntu:~/Downloads/linux_drvs/ax_block$ make
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -C /home/alinx/peta_prj/linuxPsBase/petalinux/build/tmp/work-shared/zynqmp-generic/kernel-build-artifacts M=/home/alinx/Downloads/linux_drvs/ax_block modules
make[1]: Entering directory '/home/alinx/peta_prj/linuxPsBase/petalinux/build/tmp/work-shared/zynqmp-generic/kernel-build-artifacts'
make[1]: *** No rule to make target 'modules'.  Stop.
make[1]: Leaving directory '/home/alinx/peta_prj/linuxPsBase/petalinux/build/tmp/work-shared/zynqmp-generic/kernel-build-artifacts'
Makefile:12: recipe for target 'all' failed
make: *** [all] Error 2
```

就按照下面操作更新“KERNELDIR =”字段后面编译驱动的路径，这个需要通过一些手段来找到。对于一个 petalinux 一般只要找到一次就可以了。

简单介绍一下查找“KERNELDIR =”后路径的方法。首先用下面的命令在 petalinux 工程中创建一个驱动模块。

```
petalinux-create -t modules --name ps-led --enable
```

然后找到这个驱动模块对应的 c 文件，在这个 c 文件中添加一个语法错误并保存，然后 petalinux-build 编译工程，查看编译信息。

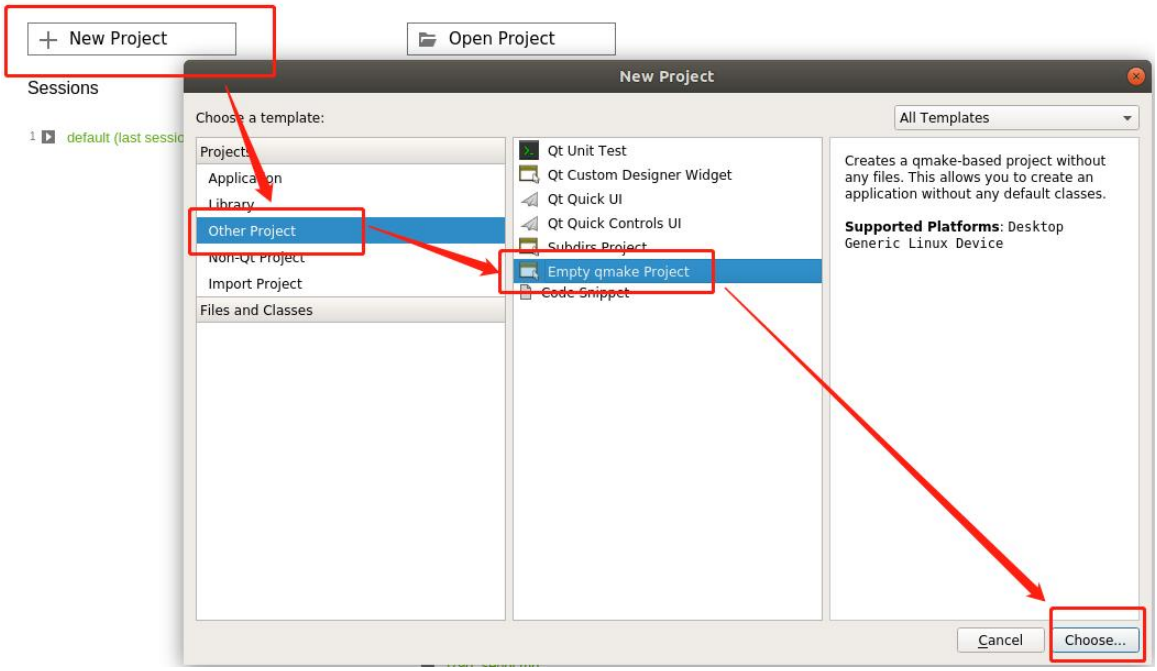


在编译信息中，先找到“make –C /.....”这句话，然后往下找到最后一次“Entering directory”的路径，就是我们需要的编译驱动的路径。

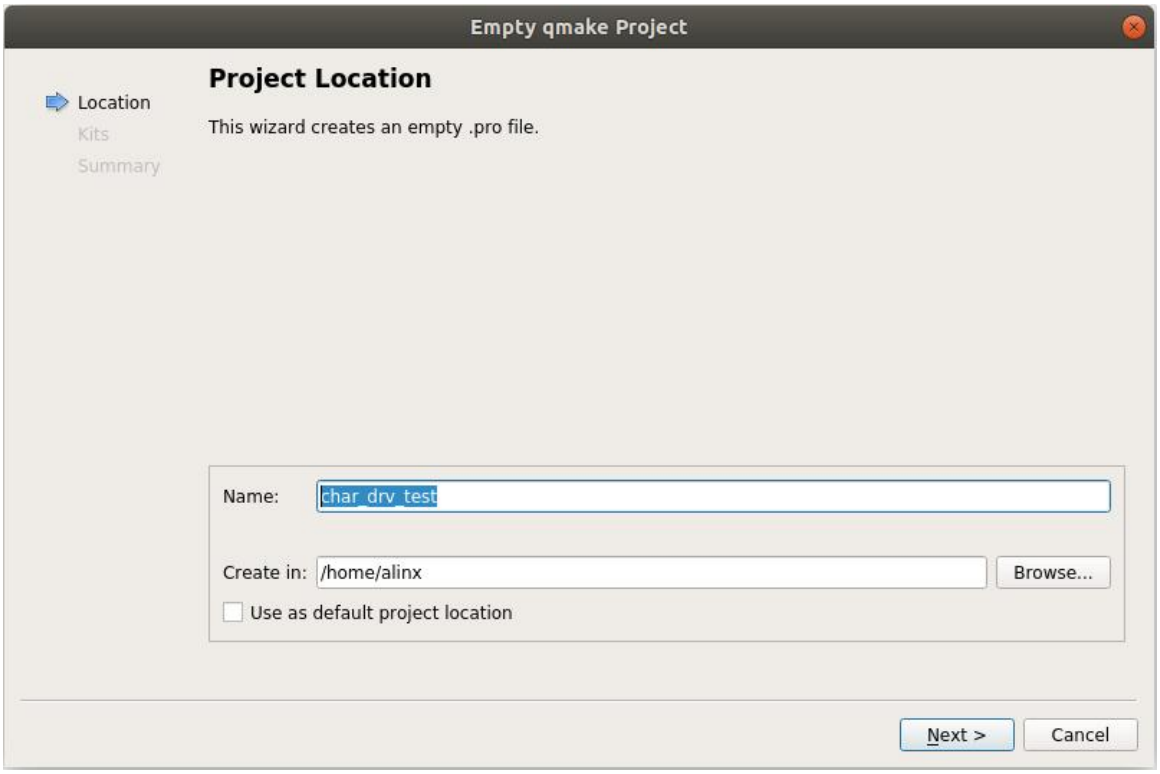
1.3.4 使用 Ubuntu 虚拟机中的 QT 编写测试 APP

驱动代码完成之后，需要测试他的功能。为了方便测试，这里我们使用 Ubuntu 虚拟机中的 QT Creater 来编写测试程序。

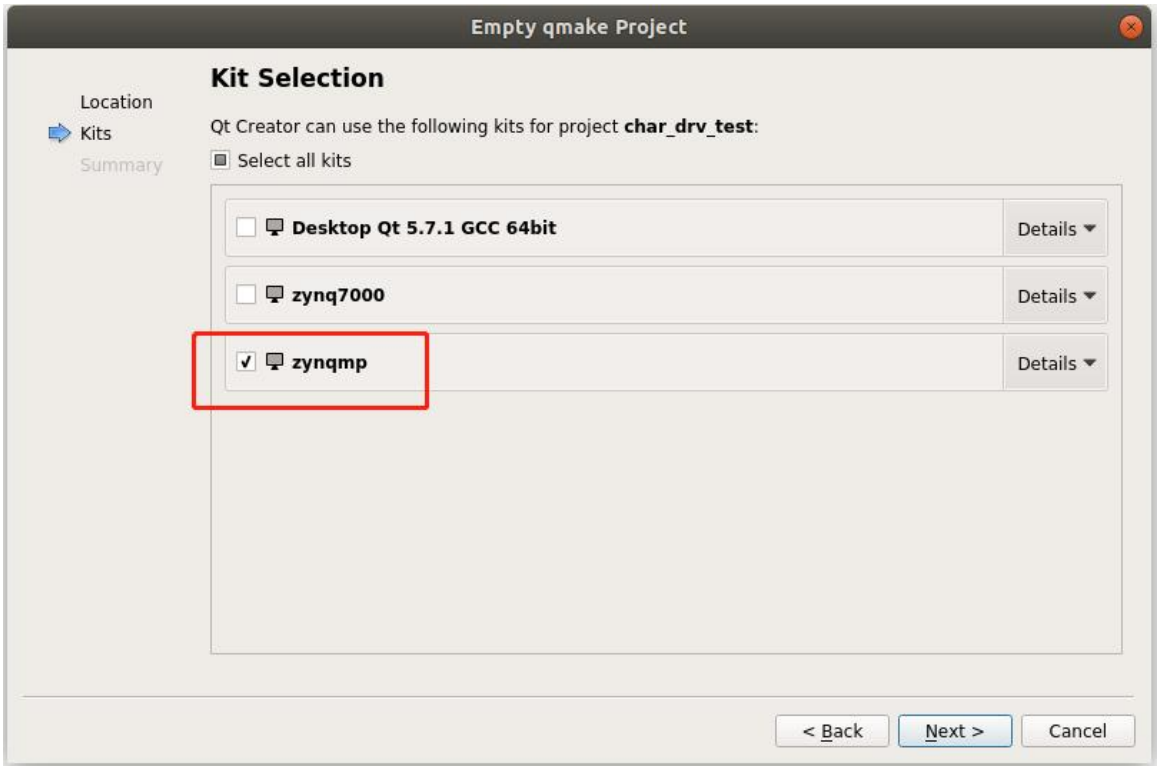
1.打开 QT，点击 New Project 按钮，创建新的空工程如下图：



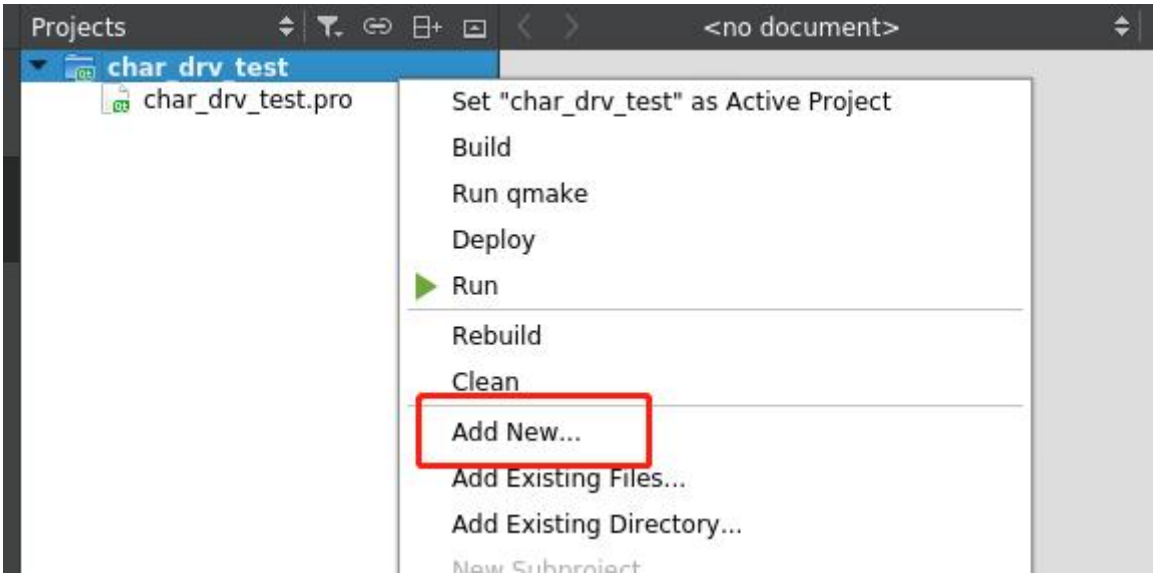
2.这里我把测试工程命名为” char_drv_test”。



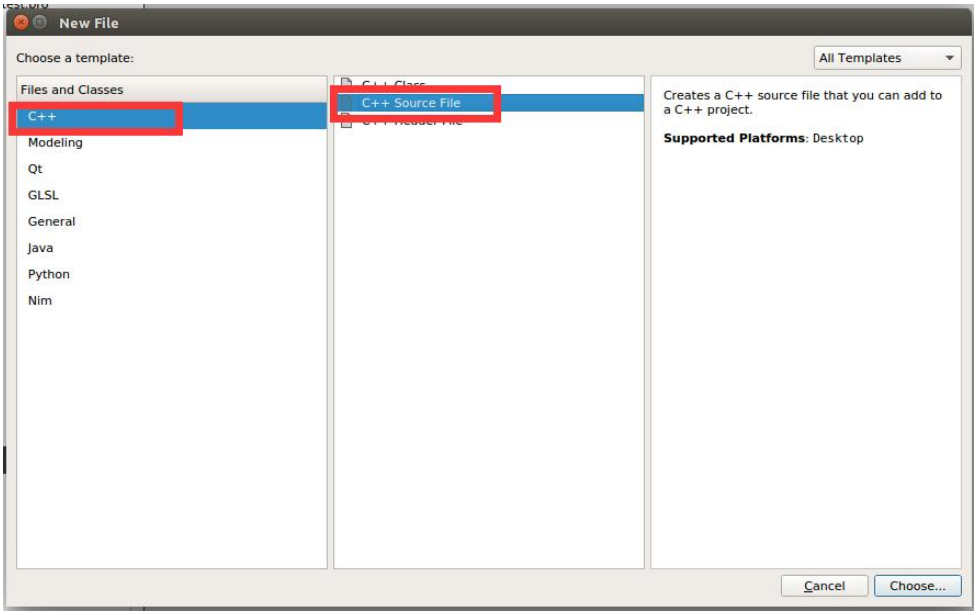
3.选择 zynqmp，编译工具请先参考“Linux 基础教程”的“第六章 QT 和 OPENCV 交叉编译开发环境”，设置 QT 的交叉编工具链。



4.打开新建的工程，在工程目录上右击，点击 Add New 选项。



5.添加 C++ Source File，命名为 main.c。



6.打开 main.c，输入下面的代码，这个程序想要达成的目标是输入 on 时点灯，输入 off 灭灯。

```
1.  #include <stdio.h>
2.  #include <string.h>
3.  #include <unistd.h>
4.  #include <fcntl.h>
5.
6.  int main(int argc, char **argv)
7.  {
8.      int fd;
9.      char buf;
10.
11.     if(3 != argc)
12.     {
13.         printf("none para\n");
14.         return -1;
15.     }
16.
17.     fd = open(argv[1], O_RDWR);
18.     if(fd < 0)
19.     {
20.         printf("Can't open file %s\r\n", argv[1]);
21.         return -1;
22.     }
23.
24.     if(!strcmp("on",argv[2]))
25.     {
26.         printf("ps_led1 on\n");
27.         buf = 1;
28.         write(fd, &buf, 1);
29.     }
30.     else if(!strcmp("off",argv[2]))
31.     {
32.         printf("ps_led1 off\n");
33.         buf = 0;
34.         write(fd, &buf, 1);
35.     }
36.     else
37.     {
38.         printf("wrong para\n");
39.         return -2;
40.     }
41.
42.     close(fd);
43.     return 0;
44. }
```

应用程序中的系统调用函数，与内核驱动中的函数格式稍有区别。

17 行的 open()函数原型为：

int open (const char *__file, int __oflag, ...);

参数说明如下：

__file：设备文件。我们通过运行程序时的第二个参数来输入设备文件名。

__oflag：为文件打开模式，有三项必选其一：

```
#define O_RDONLY        00      //只读
#define O_WRONLY        01      //只写
#define O_RDWR          02      //读写
```

还有其他复选选项，这里用不到先不提。

返回值：如果文件打开成功，则返回文件句柄。

28 行的 write()函数原型为：

ssize_t write (int __fd, const void *__buf, size_t __n);

参数说明如下：

__fd：open()函数返回的文件句柄。

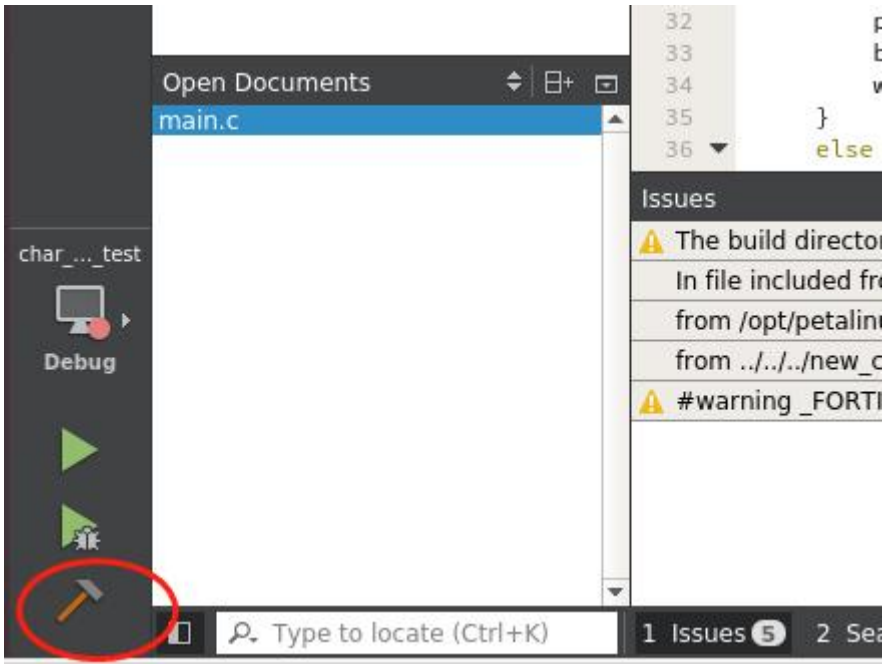
__buf：需要写入的数据首地址。

__n：需要写入的数据长度。

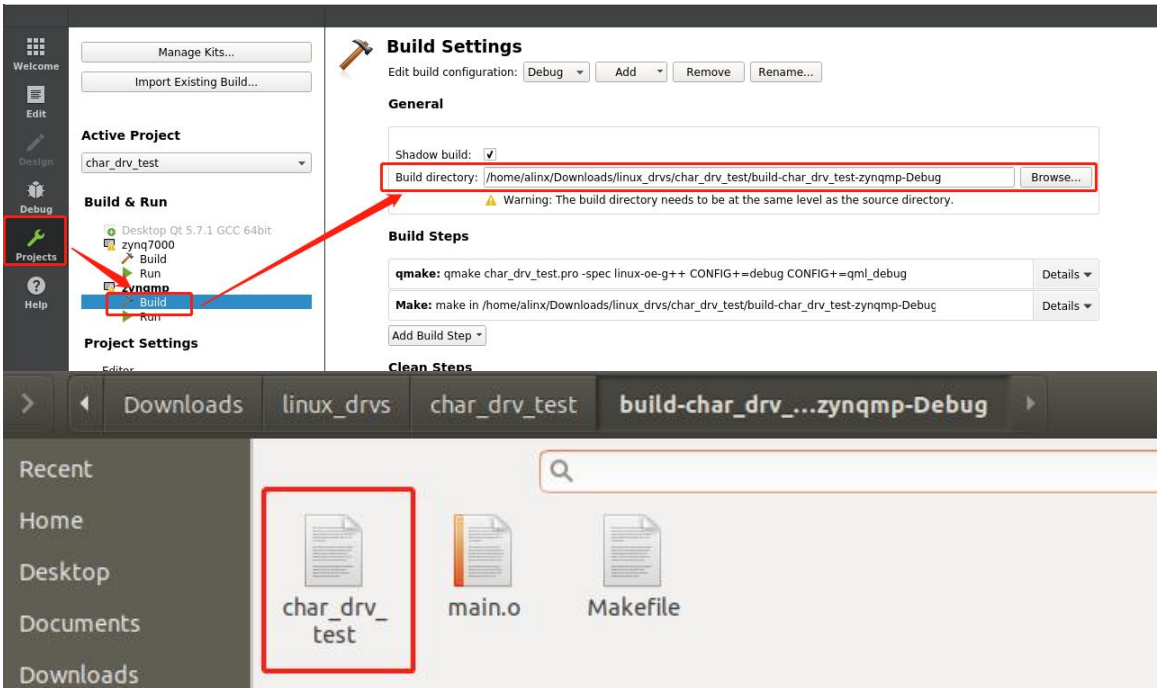
这里 write()函数的调用方式与先前驱动中的实现相对应，仅输入 0 或者 1 两个值，来通知驱动做电灯还是灭灯。

42 行在驱动文件使用完之后调用 close()函数释放文件句柄，输入参数即文件句柄。执行 close()函数后，最终驱动中实现的 relesae()函数会被执行。

7.测试代码完成后，点击 QT 右下角的小锤子编译工程，如下图：



8.编译完成后，在下图的路径中可以找到编译出来的可执行程序，这个路径会有一个默认值，也可以自己设置。



1.3.5 运行测试

1.打开串口工具，使用 sd 卡启动系统。开发板上电，登陆 linux 系统。

2.输入下列命令，用 NFS 服务挂载 ubuntu 的工作路径到开发板的/mnt 路径下，进入/mnt 路径。

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
```

其中 192.168.1.107 是教程中 ubuntu 虚拟机的 ip，/home/alinx/work 是教程中设置的 ubuntu 的工作路径。这两个需要根据实际情况修改调整。

3.把先前得到的驱动模块 ps-led.ko 和可执行文件 char_drv_test 拷贝到工作路径中，也就是这里的/home/alinx/work。

4.加载驱动模块 insmod ps-led.ko。

5.之前我们提到过使用命令 cat /proc/devices 来查看已被使用的设备号，现在我们的设备驱动加载成功了，同样可以用这个命令查看有没有成功注册到设备号。

6.至此我们还缺少设备文件，创建设备文件的命令为:

mknod /dev/xxx type major minor

/dev/xxx：xxx 为设备文件名称，统一放在/dev 路径下，这里我命名为/dev/alinx-led。

type：设备类型，字符设备为 c。

major：主设备号，要与我们调用的驱动一直，应用程序通过 open()函数来关联设备文件，设备文件通过主设备号来关联驱动程序，最终实现应用程序和驱动相互关联。所以这里这里主设备号为 200。

minor：次设备号，一般从零开始，这里填 0 即可。

输入命令：

```
mknod   /dev/alinx-led   c   200   0
```

```
root@petalinux:~# mknod /dev/alinx-led c 200 0
```

7.设备文件也准备好了，接下来就可以运行测试程序了，点亮命令如下：

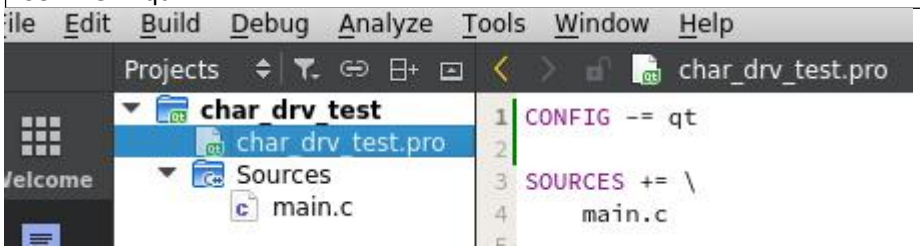
```
./char_drv_test    /dev/alinx-led    on
```

```
root@petalinux:/mnt# ./char_drv_test /dev/alinx-led on
[  525.744516] char dev open
ps_led1 on
[  525.747205] char dev release
```

如果出现这个错误，就在 char_drv_test.pro 里加这一句：

```
root@petalinux:/mnt# ./char_drv_test /dev/alinx-led on
./char_drv_test: error while loading shared libraries: libQt5Gui.so.5: cannot open shared object file: No such file or direc
tory
```

```
CONFIG -= qt
```



8.熄灭 led：

```
./char_drv_test    /dev/alinx-led    off
```

9.卸载驱动模块用下面的命令：

```
rmmod    ./ps-led.ko
```

```
root@petalinux:/mnt# rmmod ps-led.ko
root@petalinux:/mnt#
```

第二章 字符设备的新写法

这一章我们使用的 petalinux 工程是“Linux 基础教程”中“第一章 使用 Petalinux 定制 Linux 系统”制作的。

2.1 设备号管理

在上一章中的实验中，设备号直接在驱动代码中写死了。这样做会带来很多麻烦：

①编译驱动代码前，必须要先查看目标系统中设备号的占用情况；

②更换设备后原先驱动中写死的设备号就可能已被占用；

③原先的驱动注册函数 register_chrdev()输入参数中仅有主设备号而没有次设备号，这意味着一个设备就会占用所有的次设备号，十分浪费。

针对这些问题，Linux 内核提出了新的字符设备注册方法，并由内核来管理设备号。新增两个设备号注册函数。

1.当驱动程序需要给定主设备号时，使用函数来注册设备号：

int register_chrdev_region(dev_t from, unsigned count, const char *name)

输入参数说明：

from：需要申请的起始设备号，dev_t 类型，它取代了原先的主设备号和次设备号，在需要指定主次设备号的情况下，可以通过方法 from=MKDEV(major, minor);来获取他的值。

count：需要申请设备号的个数，一般只要申请一个。

name：设备名。

示例：

```
1.  int   major = 200;    //主设备号指定为 200
2.  int   minor = 0;      //次设备号为 0
3.  dev_t devid = MKDEV(major, minor);    //通过主次设备号获得设备号
4.  /*向内核注册设备号*/
5.  register_chrdev_region(devid, 1, "xxx-dev");
```

2.驱动程序不需要指定主设备号时，使用函数：

int alloc_chrdev_region(dev_t *dev, unsigned baseminor, unsigned count, const char *name)

输入参数说明：

dev：设备号指针，不指定主次设备号的情况下，设备号由内核分配，因此传入指针来获取设备号，注册成功后可以通过方法 major = MAJOR(*dev); 和 minor = MINOR(*dev);分别获取主次设备号，如果不需要用到主次设备号，不获取也可以。

baseminor：次设备号起始地址。

count：需要申请设备号的个数，一般只要申请一个。

name：设备名。

示例：

```
1.  dev_t devid;    //设备号
2.  /* 申请设备号 */
3.  alloc_chrdev_region(&devid, 0, 1, "xxx-dev");
```

3.注销设备号只要使用同一个函数：

void unregister_chrdev_region(dev_t from, unsigned count)

输入参数含义以注册函数一致。

示例：

```
dev_t devid;    //设备号
/* 申请设备号 */
alloc_chrdev_region(&devid, 0, 1, " xxx-dev");
/* 注销设备号 */
unregister_chrdev_region(devid, 1);
```

2.2 新的注册方法

上面提到过注册函数 register_chrdev()存在问题。和新的设备号管理方式相对应，现在摒弃 register_chrdev()，使用一套新的方法来来注册字符设备。

2.2.1 字符设备数据结构

使用 cdev 结构体来定来定义一个字符设备，他定义在 include/linux/cdev.h 中，具体如下：

```
struct cdev {
    struct kobject kobj;
    struct module *owner;
    const struct file_operations *ops;
    struct list_head list;
    dev_t dev;
    unsigned int count;
};
```

重要成员变量：

owner：一般设置为 THIS_MODULE；

ops：设备操作函数指针；

dev：设备号。

2.2.2 cdev 结构体初始化

cdev 结构体变量定义后需要用 cdev_init()函数初始化，函数原型为：

void cdev_init(struct cdev *cdev, const struct file_operations *fops)

cdev：字符设备结构体指针；

fops：设备操作函数集合结构体指针。

使用示例：

```
 /* 字符设备 */
struct cdev ax_cdev = {
    .owner = THIS_MODULE,
};

/* 设备操作函数 */
static struct file_operations ax_fops = {
    .owner = THIS_MODULE,
    .open……
};

/* ax_cdev 变量初始化 */
cdev_init(&ax_cdev, &ax_fops);
```

2.2.3 新的注册和注销函数

初始化字符结构体变量后，便可以使用这个变量来向 Linux 系统注册字符设备。使用新的注册函数 cdev_add，原型为：

int cdev_add(struct cdev *p, dev_t dev, unsigned count)

p：上面初始化后的字符设备结构体变量；

dev：设备号；

count：需要添加的设备数量。

注册函数变了，注销卸载函数也一样，不再使用 unregister_chrdev()函数，改用 cdev_del()函数，原型：

void cdev_del(struct cdev *p)

输入参数即为字符设备结构体变量。

结合上设备号，补充一下 2.2.2 中的示例：

```
1.  /* 字符设备 */
2.  struct cdev ax_cdev = {
3.      .owner = THIS_MODULE,
4.  };
5.
6.  /* 设备操作函数 */
7.  static struct file_operations ax_fops = {
8.      .owner = THIS_MODULE,
9.      .open……
10. };
11.
12. /* 设备号 */
13. cdev_t devid;
14. /* 申请设备号 */
15. alloc_chrdev_region(&devid, 0, 1, " xxx-dev");
16.
17. /* ax_cdev 变量初始化 */
18. cdev_init(&ax_cdev, &ax_fops);
19. /* 注册字符设备 */
20. cdev_add(&ax_cdev, devid, 1);
21. ……
22. /* 卸载字符设备 */
23. cdev_del(&ax_cdev);
```

2.3 自动创建设备文件

2.3.1 mdev

medv 是一个用户程序，是 udev 的简化版。它可以检测并根据系统中硬件设备状态来创建或者删除设备文件。在加载驱动模块后，会自动在/dev 目录下创建设备节点文件，卸载驱动模块后设备节点也会自动删除。接下来看看如何实现。

2.3.2 类的创建和删除

创建设备前需要先创建类，设备是在类下面创建的。类的结构体 struct class 结构体定义在 include/linux/device.h 中，需要使用函数 class_create()来创建。class_create()是个宏定义：

```
1.  #define class_create(owner, name) \
2.  ({ \
3.      static struct lock_class_key __key; \
4.      __class_create(owner, name, &__key); \
5.  })
6.
7.  struct class *__class_create(struct module *owner, const char *name, struct lock_class_key *key)
```

展开后可已看出：

输入参数：

owner：至今出现的 owner 值都是 THIS_MODULE，这里任然不例外；

name：类的名字。

返回值：struct class 类型的结构体指针。

卸载驱动程序时需要删除类，使用函数 class_destroy()，原型如下：

void class_destroy(struct class *cls);

cls 为需要删除的类。

2.2.3 设备节点的创建和删除

创建类后，使用 device_create()函数在类下面创建设备，原型为：

```
struct device *device_create(struct class *class,
                             struct device *parent,
                             dev_t devt,
                             void *drvdata,
                             const char *fmt, ...)
```

参数说明：

class：上节介绍的类，设备会在这个类下创建；

parent：父设备，没有父设备的话填 NULL；

devt：设备号；

drvdata：设备可能会用到的数据，没有的话填 NULL；

fmt：设备名，比如当 fmt=axled 时，创建设备后就会生成/dev/axled 文件。

删除设备函数为：

void device_destroy(struct class *class, dev_t devt)

输入参数含义和上面一样。

自动创建设备节点的实现一般放在驱动入口函数中，结合上一节类的创建以及设备号，自动创建设备节点的实现示例如下：

```
struct class *class;      /* 类 */
struct device *device;    /* 设备 */
dev_t devid;              /* 设备号 */

/* 驱动入口函数 */
static int __init xxx_init(void)
{
    ……
    /* 申请设备号 */
    alloc_chrdev_region(&devid, 0, 1, "xxx-dev");
    ……
    /* 创建类 */
    class = class_create(THIS_MODULE, "xxx");
    /* 创建设备 */
    device = device_create(class, NULL, devid, NULL, "xxx");

    return 0;
}

/* 驱动出口函数 */
static void __exit led_exit(void)
{
    /* 删除设备 */
    device_destroy(class, devid);
    /* 删除类 */
    class_destroy(class);
    /* 注销字符设备 */
    unregister_chrdev_region(devid, 1);
}

module_init(led_init);
module_exit(led_exit);
```

2.4 字符设备新驱动实验

现在了解字符设备驱动的新写法相关的要点，接下来就动手尝试，这章我们要达成的目标和上一章相同，编写开发板上”PS LED”这个 led 的设备驱动，通过驱动程序能点亮、熄灭这个 led。

2.4.1 编写字符设备驱动程序

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/ide.h>
#include <linux/types.h>
#include <linux/cdev.h>
#include <linux/device.h>

#define GPIO_BASE     0x00FF0A0000
#define GPIO_BASE_LEN 0x300
static unsigned int *gpio_addr_vir = 0;

#define GPIO_OEN1     (unsigned int *)(0x248 + (unsigned long)gpio_addr_vir)
#define GPIO_DIRM1    (unsigned int *)(0x244 + (unsigned long)gpio_addr_vir)
#define GPIO_DATA1    (unsigned int *)(0x044 + (unsigned long)gpio_addr_vir)

#define CHAR_DEV_MAJOR 200
#define CHAR_DEV_NAME  "led_dev"

struct alinx_char_dev{
    dev_t          devid;
    struct cdev    cdev;
    struct class   *class;
    struct device *device;
};

static struct alinx_char_dev alinx_char = {
    .cdev = {
        .owner = THIS_MODULE,
    },
};

static int char_dev_open(struct inode *inode_p, struct file *file_p)
{
    printk(KERN_ERR"char dev open\n");
    return 0;
}

static int char_dev_release(struct inode *inode_p, struct file *file_p)
{
    printk("char dev release\n");
    return 0;
}

static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t *l
off_t_p)
{
    int ret = 0;
    char lc_buf = 0;

    ret = copy_from_user(&lc_buf, buf, len);
    if(ret != 0)
    {
        printk("copy_from_user failed\n");
        return -1;
    }

    if(len != 1)
    {
        printk("len err\n");
        return -2;
    }

    if(1 == lc_buf)
    {
        *GPIO_DATA1 |= 0x00000020;
```

```
67.     }
68.     else if(0 == lc_buf)
69.     {
70.         *GPIO_DATA1 &= 0xFFFFFFDF;
71.     }
72.     else
73.     {
74.         printk("data err\n");
75.     }
76.
77.     return 0;
78. }
79.
80. static struct file_operations char_dev_opt = {
81.     .owner = THIS_MODULE,
82.     .open = char_dev_open,
83.     .write = char_dev_write,
84.     .release = char_dev_release,
85. };
86.
87. static int __init char_drv_init(void)
88. {
89.     int ret = 0;
90.
91.     ret = alloc_chrdev_region(&alinx_char.devid, 0, 1, CHAR_DEV_NAME);
92.     if(ret < 0)
93.     {
94.         goto err;
95.     }
96.
97.     cdev_init(&alinx_char.cdev, &char_dev_opt);
98.
99.     ret = cdev_add(&alinx_char.cdev, alinx_char.devid, 1);
100.    if(ret < 0)
101.    {
102.        goto err;
103.    }
104.
105.    alinx_char.class = class_create(THIS_MODULE, "alinx_char_dev");
106.    if(IS_ERR(alinx_char.class))
107.    {
108.        ret = PTR_ERR(alinx_char.class);
109.        goto err;
110.    }
111.
112.    alinx_char.device = device_create(alinx_char.class, NULL, alinx_char.devid, NULL, CHAR_DEV_N
    AME);
113.    if(IS_ERR(alinx_char.device))
114.    {
115.        ret = PTR_ERR(alinx_char.device);
116.        goto err;
117.    }
118.
119.    gpio_addr_vir = ioremap_wc(GPIO_BASE, GPIO_BASE_LEN);
120.    if(NULL == gpio_addr_vir)
121.    {
122.        iounmap(gpio_addr_vir);
123.        ret = -1;
124.        goto err;
125.    }
126.    *GPIO_DIRM1 |= 0x00000020;
127.    *GPIO_OEN1  |= 0x00000020;
128.
129. err:
130.    return ret;
131. }
132.
133. static void __exit char_drv_exit(void)
134. {
135.    iounmap(gpio_addr_vir);
136.
137.    cdev_del(&alinx_char.cdev);
138.
139.    device_destroy(alinx_char.class, alinx_char.devid);
140.
141.    class_destroy(alinx_char.class);
142.
143.    unregister_chrdev_region(alinx_char.devid, 1);
144. }
```

```
145.
146.
147.
148. module_init(char_drv_init);
149. module_exit(char_drv_exit);
150.
151. MODULE_AUTHOR("Alinx");
152. MODULE_ALIAS("gpio_led");
153. MODULE_DESCRIPTION("GPIO LED driver");
154. MODULE_VERSION("v1.0");
155. MODULE_LICENSE("GPL");
```

21~32 行，新建了一个结构体类型，并用这个类型声明了一个变量。把之后会用到的设备号、字符设备、类、设备节点等数据类型都打包进这个结构体中。这样在使用变量时会方便很多，也增加了可读性。并且在后面我们用到私有数据时，这个结构体也会带来遍历，具体在用到的时候再细讲。

87~131 行是关键的地方，在驱动入口函数中，把原先注册字符设备的方法换成本章讲到的申请设备号、初始化并注册字符设备、创建类和设备节点方法。

133~144 行出口函数中就是做与入口函数中注册创建相对应的注销和删除。

2.4.2 编译驱动程序

编译驱动程序得到新的 ps-led.ko 驱动模块。编译方法参考第一章，这里不再赘述。

2.4.3 编写测试 APP

测试程序可以直接使用上一章的。

2.4.4 运行测试

1.给开发板上电，并挂在虚拟机的工作目录到开发板/mnt 路径，并进入./mnt 路径。

2.加载驱动，执行命令：insmod ps-led.ko。

```
root@petalinux:/mnt# insmod ps-led.ko
root@petalinux:/mnt#
```

3.驱动加载成功后查看设备文件有没有创建成功，执行命令：ls /dev/ led_dev

```
root@petalinux:/mnt# ls /dev/led_dev
/dev/led_dev
```

4.设备节点文件已经存在了，可以使用测试 APP 来试试驱动程序了，执行下列命令：./char_drv_test /dev/led_dev on

```
root@petalinux:/mnt# ./char_drv_test /dev/led_dev on
[  183.008554] char dev open
ps_led1 on
[  183.011242] char dev release
```

5.测试一下卸载驱动，执行命令：rmmod ps-led

6.删除设备后，确认设备节点文件有没有被删除：ls /dev/ led_dev

```
root@petalinux:/mnt# rmmod  ps-led.ko
root@petalinux:/mnt# ls /dev/led_dev
ls: /dev/led_dev: No such file or directory
root@petalinux:/mnt#
```

第三章 设备树和 of 函数

这一章我们使用的 petalinux 工程是“Linux 基础教程”中“第一章 使用 Petalinux 定制 Linux 系统”制作的。

3.1 设备树

ARM Linux3.0 以前是没有应用设备树的，那时 ARM Linux 使用者们用 c 语言的数据结构来描述板子上的设备，每一个设备就对应到一个描述文件。ARM 的板子种类不计其数，而 Linux 社区为了保证内核源码的通用性，把这些描述文件全部塞了进去，导致 Linux 内核充满了垃圾代码，震怒了 Linux 之父 Linus。之后 ARM 社区就学习 PowerPC 引入了设备树来替代以前的方法。

设备树除了没有用于代码之外还有其他优势。在驱动对应的硬件有变动时，不需要重新编译内核或驱动程序，只需要单独编译设备树文件，在启动板卡时把设备树编译结果传给内核即可。（遗憾的是，这个优势在 petalinux 中没有得到体现，petalinux 编译出来的结果中，u-boot、内核和设备树被一并打包进了 image.ub 里，而不是单独提供的。所以变更设备树时，往往会使用 petalinux-build 命令把内核整体重新编译。其实通过前面两章可以发现，在更改或者新增驱动时，也是整体编译内核的。实际上单独编译驱动是可以的，但是，为了统一操作步骤就都用 petalinux-build 命令了。）

设备树即树状的设备结构，“树”是个形象的比喻，以系统总线为主干，其他挂在在系统总线上的如 I2C、SPI、GPIO 控制器等设备为分支，而这些挂载在主干上的分支又有他们自身挂载的设备，如 I2C 上挂载了 EEPROM、RTC 等设备，这样的枝干分叉结构就如同树一般。

3.1.1 DTS、DTB 和 DTSI

通常我们编辑的设备树文件扩展名为.dtc(device tree source)，而内核读取的设备树文件为编译之后得到的二进制文件.dtb(device tree binary large object)。DTB 文件我们也可以直接修改，但是二进制文件修改很不方便。这里就重点说 DTS 文件。

DTS 文件内容乍一看有点像 JSON 数据结构，大括号里大括号。当然肯定是有差别的，DTS 有自己的一套语法规则。

先来看一段 DTS 的示例代码：

```
1.  #include "zynq-7000.dtsi"
2.
3.  / {
4.      model = "Zynq ZC702 Development Board";
5.      compatible = "xlnx,zynq-zc702", "xlnx,zynq-7000";
6.
7.      ……
8.
9.      memory@0 {
10.         device_type = "memory";
11.         reg = <0x0 0x40000000>;
12.     };
13.
14.     ……
15.
16.     gpio-keys {
17.         compatible = "gpio-keys";
18.         #address-cells = <1>;
19.         #size-cells = <0>;
```

```
20.          autorepeat;
21.          sw14 {
22.              label = "sw14";
23.              gpios = <&gpio0 12 0>;
24.              linux,code = <108>; /* down */
25.              wakeup-source;
26.              autorepeat;
27.          };
28.
29.          ……
30.      };
31.  };
```

结合这段代码，来大致了解一下 DTS 的语法格式。

第 **1** 行是包含头文件的语句。和 c 语言一样，DTS 文件使用#include 来引用头文件，可以引用.h、.dts 以及.dtsi 文件。".dtsi"文件是专门用于编写 DTS 头文件的文件，语法与 DTS 文件相同，是编写 DTS 头文件的首选。

3.1.2 节点

3.1.2.1 通用节点格式

DTS 文件中，设备用节点来表示，格式为：

```
1.  [label:] node-name[@unit-address] {
2.      [properties definitions]
3.      [child nodes]
4.  };
```

[]中的部分是非必要项。

node-name 是设备节点名称，根节点用"/"表示。第三行的"/"即是设备"Zynq ZC702 Development Board"的根节点。第 **9** 行的 memory、第 **11** 行的 gpio-keys、第 **21** 行的 sw14 都是设备节点名称。

unit-address 一般是指代设备地址或寄存器首地址，如代码中第 **9** 行的 memory@0。如果设备没有设备地址或寄存器就不用写。

label 是设备别称，用于便捷访问节点。如一个名称为 slcr@f8000000 的节点，正常要访问的话，需要用名称 slcr@f8000000 去访问，如果给这个节点加上标签 slcr1: slcr@f8000000，访问节点只需要使用&slcr1 即可。

{}里的是节点的内容，节点内容有两类，[properties definitions]是节点属性，后面再讲解。[child nodes]是这个挂在在这个设备节点上的子节点。子节点的格式和上面介绍的一样。

3.1.2.2 特殊节点 aliases

aliases 节点用于定义别名，作用于 label 标签相似，格式如下：

```
1.    aliases {
2.       ethernet0 = "&gem0";
3.       serial0 = "&uart1";
4.   };
```

之后便可以使用&gem 来访问节点 ethernet。

3.1.3 属性

节点属性[properties definitions]有 4 种形式：

1. [label:]property-name;

属性为空值。如示例代码中 **20** 行和 **26** 行的 autorepeat。

2. [label:]property-name = <arrays of cells>;

用<>括起来的值内容是 32 位数据的合集。如示例代码 11 行的 reg = <0x0 0x40000000>。

3.[label:]property-name = “string”;

用””包含的表示字符串，如第五行的 compatible = "xlnx,zynq-zc702", "xlnx,zynq-7000"。

4. [label:]property-name = [bytestring];

用[]括起来的表示字符序列，这个比较少见，手动举个例子，假设有属性表示为 memory-addr = [0011223344];，这个就等同于 memory-addr = [00 11 22 33 44];他的值是 5 个 byte 型的数组成的序列，并且这个这个 byte 型的数是两位 16 进制数组成的。

属性可以用户自定义，也有很多标准属性，下面介绍几种常见的标准属性。

3.1.3.1 compatible 属性

compatible 属性也叫兼容性，这个属性在我们后面讲平台设备是才会用到，现在可以简单了解一下。他的值是字符串列表，是平台设备框架中设备和驱动关联的关键，在驱动代码中定义一个 of 匹配表来和 compatible 属性对应。如示例代码 17 行，节点 gpio-keys 的 compatible = "gpio-keys"，那么在对应的驱动代码中，就会有以下定义来与之对应：

```
1.  static const struct of_device_id gpiokeys_ids[] = {
2.      { .compatible = "gpio-keys", },
3.      { /* sentinel */ }
4.  };
```

struct of_device_id 是 of 匹配表的数据类型。当驱动程序 of 匹配表中的 compatible 值与设备树节点中的 compatible 对应时，这个节点就会使用这个驱动程序。

根节点中的 compatible 属性比较特殊，表示这个板子兼容哪些平台。一般有两个值，前者表示板子的型号，后者表示使用的芯片。

3.1.3.2 model 属性

model 属性的值也是字符串，一般用来表示板子的名称，与根节点中的 compatible 属性类似。

3.1.3.3 #address-cells、#size-cells 和 reg 属性

#address-cells 属性表示当前节点**子节点**的 reg 属性中，使用多少个 u32 整数来描述地址 address。

#size-cells 属性表示当前节点**子节点**的 reg 属性中，使用多少个 u32 整数来描述大小 length。

reg 属性一般用于描述某个外设的寄存器地址范围信息，格式为 reg = <address1 length1 address2 length2 address3 length3……>。**父节点**中的#address-cells 属性即指代 reg 属性中 address 的大小，#size-cells 即指代 reg 属性中 length 的大小。

这个在示例代码中也没体现，为了便于理解，举个极端一点的例子：

```
1.  ax-parent {
2.      #address-cells = <2>;
3.      #size-cells = <1>;
4.      ……
```

```
5.
6.      ax-son {
7.          reg = <0x00000001 0x00000002 0x00000003
8.                 0x00000004 0x00000005 0x00000006>;
9.          ……
10.     }
11. }
```

子节点 ax-son 的 reg 属性的有 6 个 u32 的数据，父节点 ax-parent 中#address-cells 等于 2，因此子节点 ax-son 的 reg 属性的值中表示 address 的有两个 u32 的数，即 0x00000001、0x00000002 这两个数据都是 address 的值。同理#size-cells 等于 1，所以 length1 的值仅有一个 u32 的数据等于 0x00000003。而后面的三个 u32 数据则是 address2 和 length2 的值。

3.1.3.4 device_type 属性

这个属性现在只能用于 cpu 节点或者 memory 节点。在 cpu 节点中 device_type = “cpu”，在 memory 节点中 device_type = “memory”。

3.1.3.5 phandle 属性

phandle 属性的取值必须是唯一的，他的作用与 label 标签相似，用来引用节点。

```
1.  ax-node-1 {
2.      phandle = <1>;
3.      interrupt-controller;
4.  }
5.
6.  ax-node-2 {
7.      interrupt-parent = <1>;
8.  }
```

在节点 ax-node-1 中有 phandle 属性为<1>，在 ax-node-2 中 interrupt-parent 属性需要指定父节点，赋值为<1>即可。

3.1.4 在文件系统中查看设备树

内核启动时会解析 DTB 文件中的节点信息，并在根文件系统的/proc/devicetree 目录下创建个节点对应的文件夹。

```
root@ax_peta:~# cd /proc/device-tree
root@ax_peta:/proc/device-tree# ls
#address-cells  alinxled        chosen          fpga-full       pmu@f8891000
#size-cells     alinxpwm        compatible      memory          usb_phy@0
aliases         amba            cpus            model
alinxkey        amba_pl         fixedregulator  name
root@ax_peta:/proc/device-tree#
```

进入到对应的节点中，能查看到节点的各个属性。

```
root@ax_peta:/proc/device-tree# cd memory/
root@ax_peta:/proc/device-tree/memory# ls
device_type  name         reg
root@ax_peta:/proc/device-tree/memory# cat device_type
memoryroot@ax_peta:/proc/device-tree/memory#
```

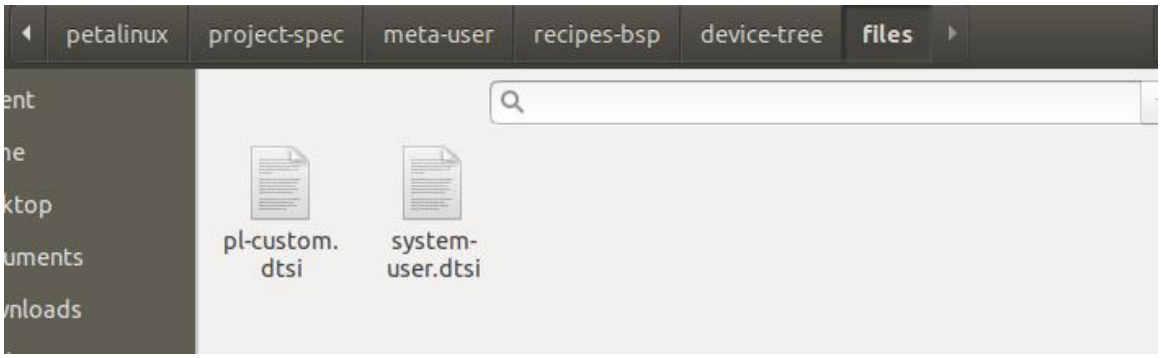
3.1.5 petalinux 中的设备树

petalinux 中有两处设备树。

一处是 petalinux 帮我们生成的设备树，路径是 petalinux 工程中的“components/plnx_workspace/device-tree/device-tree”。这里的设备树编译工程后才会生成并且是无法修改的，只能查看。



需要修改设备树就要到另一处设备树，路径在 petalinux 工程中的“project-spec/meta-user/recipes-bsp/device-tree/files”。这里的两个设备树文件我们都可以修改。



3.2 of 函数

Linux 内核提供了 of 函数来让我们获取设备树中的信息，of 之名来自这些函数的前缀”of_”。of 函数的原型定义在内核目录 include/linux/of.h 中。这节我们介绍一些常用的 of 函数，没有介绍的等用到的时候再去了解也不迟。

3.2.1 查找节点的 of 函数

3.2.1.1 device_node 结构体

device_node 结构体也定义在 include/linux/of.h 中，作为查找节点的 of 函数的返回值，用于给内核描述设备节点。

3.2.1.2 of_find_node_by_name 函数

of_find_node_by_name ()通过节点名查找节点，函数原型为：

```
struct device_node *of_find_node_by_name(struct device_node *from, const char *name);
```

参数说明：

from：从这个节点开始查找，输入 NULL 时从根节点开始查找。

name：目标节点名称。

返回值：找到目标节点返回 device_node 结构体。没有找到时返回 NULL。

3.2.1.3 of_find_node_by_type 函数

of_find_node_by_type ()通过 device_type 属性查找节点，函数原型为：

struct device_node *of_find_node_by_type(struct device_node *from, const char *type);

参数说明：

from：从这个节点开始查找，输入 NULL 时从根节点开始查找。

type：device_type 属性值。

返回值：找到目标节点返回 device_node 结构体。没有找到时返回 NULL。

3.2.1.4 of_find_compatible_node 函数

of_find_compatible_node ()通过 device_type 和 compatible 属性查找节点，函数原型为：

struct device_node *of_find_compatible_node(struct device_node *from, const char *type, const char *compatible);

参数说明：

from：从这个节点开始查找，输入 NULL 时从根节点开始查找。

type：device_type 属性值，输入 NULL 时忽略。

compatible：compatible 属性值。

返回值：找到目标节点返回 device_node 结构体。没有找到时返回 NULL。

3.2.1.5 of_find_node_by_path 函数

of_find_node_by_path ()通过节点路径查找节点，函数原型为：

struct device_node *of_find_node_by_path(const char *path);

参数说明：

path：节点的完整路径，以 3.1.1 节中的示例代码为例，21 行的 sw14 设备的完整路径为 /gpio-keys/sw14。

返回值：找到目标节点返回 device_node 结构体。没有找到时返回 NULL。

3.2.2 提取属性的 of 函数

3.2.2.1 property 结构体

property 结构体同样也定义在 include/linux/of.h 中，作为提取属性的 of 函数的返回值，用于给内核描述节点属性。

3.2.2.2 of_find_property 函数

of_find_property 函数通过设备节点、属性名、属性值的大小查找属性，函数原型：

property *of_find_property(const struct device_node *np, const char *name, int *len);

参数说明：

np：设备节点。

name：目标属性名。

len：目标属性值的长度。

返回值：目标属性。

3.2.2.3 of_property_read_u32_array 函数

of_property_read_u32_array()用于获取有多个值的属性的多个数据，函数名以及输入函数中的 u32 代表目标属性单个值的大小，可替换为 u8、u16、u32。函数原型：

int of_property_read_u32_array(const struct device_node *np,const char *propname, u32 *out_values, size_t size);

参数说明：

np：设备节点。

propname：目标属性名。

out_values：读取到的数据指针，读取到的数据会保存到这个地址中。

size：要读取数据数量。

返回值：

0：读取成功；

-EINVAL：属性不存在；

-ENODATA：属性无数据；

-EOVERFLOW：属性值数据数量小于 size。

3.2.2.4 of_property_read_u32 函数

of_property_read_u32()用于获取只有单个值的属性数据，函数名以及输入函数中的 u32 代表目标属性单个值的大小，可替换为 u8、u16、u32。函数原型：

int of_property_read_u32(const struct device_node *np,const char *propname, u32 out_value);

参数说明：

np：设备节点。

propname：目标属性名。

out_values：目标数据指针，读取到的数据会保存到这个地址中。

返回值：

0：读取成功；

-EINVAL：属性不存在；

-ENODATA：属性无数据。

3.2.2.5 of_property_read_string 函数

of_property_read_string 函数用于读取属性中字符串值，函数原型如下：

int of_property_read_string(struct device_node *np, const char *propname, const char **out_string);

参数说明：

np：设备节点。

proname：目标属性名。

out_string：目标字符串指针，读取到的字符串会保存到该地址。

返回值：返回 0 读取成功。

3.3 设备树下的 led 驱动实验

通过上面两节大概了解设备树和 of 函数之后，光看书面上的东西很难理解深刻，还是得通过实验来深入理解。这节还是使用简单的 led 设备来测试，不会涵盖上面讲的所有，但是在以

后的实验中，会一直用到设备树，所以不用着急，在之后的实验中慢慢去掌握就行了。

3.3.1 修改设备树

打开文件” project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi”, 在根节点下添加以下节点内容：

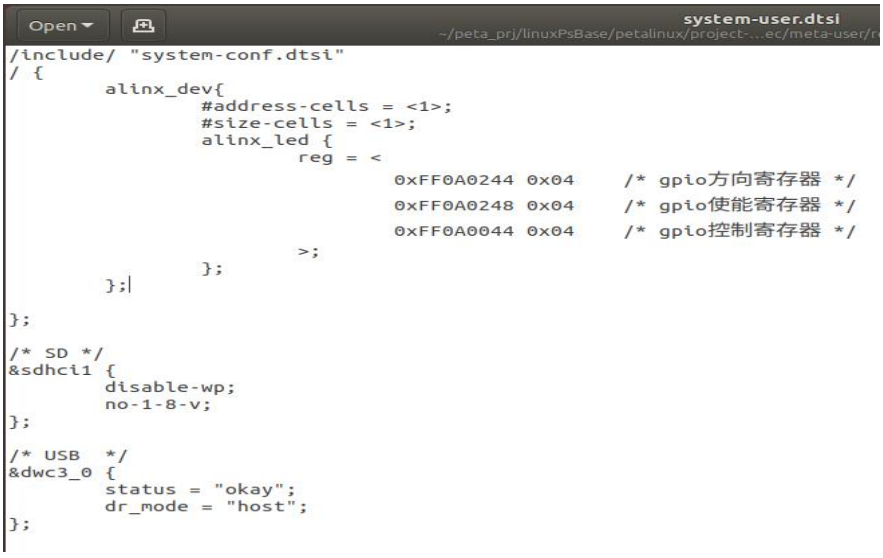
```
/include/ "system-conf.dtsi"
/ {
	alinx_dev{
		#address-cells = <1>;
		#size-cells = <1>;
		alinx_led {
			reg = <
				0xFF0A0244 0x04		/* gpio 方向寄存器  */
				0xFF0A0248 0x04		/* gpio 使能寄存器  */
				0xFF0A0044 0x04		/* gpio 控制寄存器  */
			>;
		};
	};

};

/* SD */
&sdhci1 {
	disable-wp;
	no-1-8-v;
};

/* USB    */
&dwc3_0 {
	status = "okay";
	dr_mode = "host";
};
```

节点名称为 alinxled，兼容性属性值为”alinxled”，reg 中的值即为在前两次实验中用到的 led 相关的寄存器。



因为修改了设备树，petalinux 需要重新编译 ，把新的 image.ub 拷贝到 SD 卡中，BOOT.bin 和 boot.scr 不用替换。

3.3.2 驱动程序

创建和编译驱动程序的方法参考第一章。

```
1.  #include <linux/module.h>
2.  #include <linux/kernel.h>
3.  #include <linux/fs.h>
4.  #include <linux/init.h>
5.  #include <linux/ide.h>
6.  #include <linux/types.h>
7.  #include <linux/cdev.h>
8.  #include <linux/device.h>
9.  #include <linux/of.h>
10.
11. #define CHAR_DEV_MAJOR 200
12. #define CHAR_DEV_NAME  "led_dev"
13.
14. struct alinx_char_dev{
15.     dev_t              devid;
16.     struct cdev        cdev;
17.     struct class       *class;
18.     struct device      *device;
19.     struct device_node *nd;
20. };
21.
22. static struct alinx_char_dev alinx_char = {
23.     .cdev = {
24.         .owner = THIS_MODULE,
25.     },
26. };
27.
28. static u32 *GPIO_DIRM1;
29. static u32 *GPIO_OEN1;
30. static u32 *GPIO_DATA1;
31.
32. static int char_dev_open(struct inode *inode_p, struct file *file_p)
33. {
34.     printk(KERN_ERR"char dev open\n");
35.     return 0;
36. }
37.
38. static int char_dev_release(struct inode *inode_p, struct file *file_p)
39. {
40.     printk("char dev release\n");
41.     return 0;
42. }
43.
44. static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t *l
    off_t_p)
45. {
46.     int ret = 0;
47.     char lc_buf = 0;
48.
49.     ret = copy_from_user(&lc_buf, buf, len);
50.     if(ret != 0)
51.     {
52.         printk("copy_from_user failed\n");
53.         return -1;
54.     }
55.
56.     if(len != 1)
57.     {
58.         printk("len err\n");
59.         return -2;
60.     }
61.
62.     if(1 == lc_buf)
63.     {
64.         *GPIO_DATA1 |= 0x00000020;
65.     }
66.     else if(0 == lc_buf)
67.     {
68.         *GPIO_DATA1 &= 0xFFFFFFDF;
```

```
69.     }
70.     else
71.     {
72.         printk("data err\n");
73.     }
74.
75.     return 0;
76. }
77.
78. static struct file_operations char_dev_opt = {
79.     .owner = THIS_MODULE,
80.     .open = char_dev_open,
81.     .write = char_dev_write,
82.     .release = char_dev_release,
83. };
84.
85. static int __init char_drv_init(void)
86. {
87.     int ret = 0;
88.     u32 reg_data[6] = {0};
89.
90.     alinx_char.nd = of_find_node_by_name(NULL, "alinx_led");
91.     ret = of_property_read_u32_array(alinx_char.nd, "reg", reg_data, 6);
92.     if(ret != 0)
93.     {
94.         goto err;
95.     }
96.
97.     GPIO_DIRM1  = ioremap_wc(reg_data[0], reg_data[1]);
98.     GPIO_OEN1   = ioremap_wc(reg_data[2], reg_data[3]);
99.     GPIO_DATA1  = ioremap_wc(reg_data[4], reg_data[5]);
100.
101.    *GPIO_DIRM1 |= 0x00000020;
102.    *GPIO_OEN1  |= 0x00000020;
103.
104.    ret = alloc_chrdev_region(&alinx_char.devid, 0, 1, CHAR_DEV_NAME);
105.    if(ret < 0)
106.    {
107.        goto err;
108.    }
109.
110.    cdev_init(&alinx_char.cdev, &char_dev_opt);
111.
112.    ret = cdev_add(&alinx_char.cdev, alinx_char.devid, 1);
113.    if(ret < 0)
114.    {
115.        goto err;
116.    }
117.
118.    alinx_char.class = class_create(THIS_MODULE, "alinx_char_dev");
119.    if(IS_ERR(alinx_char.class))
120.    {
121.        ret = PTR_ERR(alinx_char.class);
122.        goto err;
123.    }
124.
125.    alinx_char.device = device_create(alinx_char.class, NULL, alinx_char.devid, NULL, CHAR_DEV_N
    AME);
126.    if(IS_ERR(alinx_char.device))
127.    {
128.        ret = PTR_ERR(alinx_char.device);
129.        goto err;
130.    }
131.
132. err:
133.    return ret;
134. }
135.
136. static void __exit char_drv_exit(void)
137. {
138.    iounmap(GPIO_DIRM1);
139.    iounmap(GPIO_OEN1);
140.    iounmap(GPIO_DATA1);
141.
142.    cdev_del(&alinx_char.cdev);
143.
144.    device_destroy(alinx_char.class, alinx_char.devid);
145.
146.    class_destroy(alinx_char.class);
```

```
147.
148.     unregister_chrdev_region(alinx_char.devid, 1);
149. }
150.
151.
152.
153. module_init(char_drv_init);
154. module_exit(char_drv_exit);
155.
156. MODULE_AUTHOR("Alinx");
157. MODULE_ALIAS("gpio_led");
158. MODULE_DESCRIPTION("GPIO LED driver");
159. MODULE_VERSION("v1.0");
160. MODULE_LICENSE("GPL");
```

90 行使用 of_find_node_by_name 函数通过节点名称获取节点，因为 alinxled 节点挂在在根目录下，所以第一个参数输入 NULL。

91 行，在获取到节点后，再获取节点中的 reg 属性的数据，因为 reg 属性中存放着我们需要的寄存器地址和大小。总共 3 个地址 3 个 size，因此数据总数为 6。

其他的操作与上一章基本相同。

3.3.4 测试程序

测试可以直接使用第一章的。

3.3.5 运行测试

测试方法和第二章相同，只要能成功点亮 led 就成功了。

```
root@petalinux:/mnt# insmod ps-led.ko
[  427.716328] char_drv_init 0
root@petalinux:/mnt# ./char_drv_test /dev/led_dev on
[  458.953962] char dev open
ps_led1 on
[  458.956651] char dev release
root@petalinux:/mnt# ./char_drv_test /dev/led_dev off
[  484.168537] char dev open
ps_led1 off
[  484.171233] char dev release
root@petalinux:/mnt#
```

另外在系统运行之后，查看一下/proc/device-tree 路径中有没有我们添加的 alinxled 节点，并核对内容。

第四章 pinctrl 和 gpio 子系统

在前面的实验中，我们操作设备都是直接操作寄存器的。虽然上一章使用了设备树，但也只是把寄存器信息放到设备树中，本质上都是一样的。直接操作寄存器来控制硬件资源对于编码的人来说很方便，然而对于阅读代码的人来说想必是很崩溃的。而且在设备树中写入大量的寄存器地址，不仅不易于阅读，在使用时也很容易出错。比如上一章中，仅是控制 led 就用了 4 个寄存器，而且寄存器地址在设备树中的顺序也没有规则，全是自定的，在驱动程序中读取地址也无法对应和确认，最终我们在设置寄存器时就很容易错位。

pinctrl 子系统来可以应对这个问题，同时针对我们实验中用到的 led 也就是 gpio 操作来说还需要结合 gpio 子系统。

这一章我们使用的 petalinux 工程是“Linux 基础教程”中“第一章 使用 Petalinux 定制 Linux 系统”制作的。

4.1 pinctrl 子系统

Linux 内核提供 pinctrl 子系统来统一芯片厂商的 pin 脚管理方法，他会帮我们完成以下工作：

1. 从设备树中获取 pin 脚信息；
2. 设置 pin 脚的复用功能；
3. 配置 pin 脚的电器特性。

使用 pinctrl 子系统，只需要在设备树中配置好相关节点信息，剩余的工作全由 pinctrl 子系统完成，我们只要做甩手掌柜就行了。

4.1.1 zynqmp 平台 pinctrl 子系统的设备树节点和属性

不同的芯片 pin 脚配置方法有差别，相应的芯片厂家的 pinctrl 子系统实现也会有差别，但我们需要关心的仍然只有设备树的写法。直接来看一段 zynqmp 设备树中 pinctrl 示例：

```
1.  firmware {
2.      zynqmp_firmware: zynqmp-firmware {
3.          compatible = "xlnx,zynqmp-firmware";
4.          method = "smc";
5.          pinctrl0: pinctrl {
6.              compatible = "xlnx,zynqmp-pinctrl";
7.              pinctrl_uart1_default: uart1-default {
8.                  mux {
9.                      groups = "uart0_4_grp";
10.                     function = "uart0";
11.                 };
12.                 conf {
13.                     groups = "uart0_4_grp";
14.                     slew-rate = <SLEW_RATE_SLOW>;
15.                     io-standard = <IO_STANDARD_LVCMOS18>;
16.                 };
17.                 conf-rx {
18.                     pins = "MIO18";
19.                     bias-high-impedance;
20.                 };
21.                 conf-tx {
22.                     pins = "MIO19";
23.                     bias-disable;
```

```
24.                     schmitt-cmos = <PIN_INPUT_TYPE_CMOS>;
25.                 };
26.             };
27.         };
28.     };
29. };
```

firmware 这个节点在根节点下。我们需要关心的节点是 pinctrl0，是 firmware\zynqmp_firmware 下的子节点。至于 firmware 和 zynqmp_firmware 这两个节点我们不用去关心，我们可以猜测这个两个节点是 xilinx 针对 zynqmp 实现的一个固件驱动，pinctrl 子系统也在此基础上实现，有兴趣的话可以根据 compatible 属性去内核中寻找对应的源码实现。

pinctrl 相关的部分是第五行开始的 pinctrl0 节点。pinctrl 驱动源码同样可以通过 compatible 属性找到，我们只关心如何修改设备树，pinctrl0 是 pinctrl 子系统的节点，使用 pinctrl 子系统的设备都在这个节点下添加子节点，子节点的格式如示例中的 pinctrl_uart1_default 节点。这个节点下面没有属性，而是有 4 个子节点：mux、conf、conf-rx、conf-tx。其中 mux 是配置 pin 脚的复用功能，其他三个是配置 pin 脚的电气特性，conf 中设置通用的特性，conf-rx 中设置输入引脚和相关特性，conf-tx 中设置输出引脚和相关特性。

mux 节点中，有两个必要属性：

groups：使用这个复用功能的 pin 脚分组列表。可选的值参考内核源码中 \Documentation\devicetree\bindings\pinctrl\xlnx,zynqmp-pinctrl.txt 文档，搜索关键词”Valid values for groups are:”。至于应该选择哪个分组，需要与使用的 pins(也就是 conf 节点中的 pins 属性)相对应，以例子中的 uart0_4_grp 和 MIO18、MIO19 为例，我们在寄存器手册中能找到他们的对应关系。

MIO_PIN_18 (IOU_SLCR) Register Bit-Field Summary

Field Name	Bits	Type	Reset Value	Description
Reserved	31:8	rw	0x0	reserved
L3_SEL	7:5	rw	0x0	Level 3 Mux Select: 0: GPIO [18] input/output bank 0. 1: CAN0 RX input. 2: I2C0 SCL input/output clock. 3: LPD SWDT clock output. 4: SPI1 MISO input/output. 5: TTC2 clock input 6: UART0 RxD input. 7: reserved

在 MIO18 的服用选项里，有 UART0，所以这里的 groups 选择的分组前缀是 uart0。那么 uart0_4_grp 中的标号 4 是哪儿来的，实际上他就是一个顺序标号而已，从 MIO_PIN_0 一直看到 MIO_PIN_18，会发现，MIO18 刚好是(从 0 开始的)4 号 uart0。

function：选择 pin 脚分组的复用功能。可选的值可选的值参考内核源码中 \Documentation\devicetree\bindings\pinctrl\xlnx,zynqmp-pinctrl.txt 文档，搜索关键词” Valid values for function are:”。function 的选择很明了，这里 MIO 用作 uart0，那 function 就选 uart0。

conf 节点可以分为多组，因为相同复用功能的 pin 组里每个 pin 的电器特性可能会不同，比如示例中的 uart0，有一个 pin 脚用作输出不需要设置偏置，另一个用做输入需要设置成高阻。conf 节点的名字必须要包含”conf”这个关键词，其他的关键词仅用于区分分组含义，比如示例中

的 conf-rx 是指这个 conf 节点适用于配置输入 pin 脚的，命名为 conf-in 也是同样的效果，甚至把示例中 rx 和 tx 交换，对使用也没用影响，仅用于含以上的区分。

conf 节点的必要属性为 **pin** 和 **group** 二选一。设置 pin 组时使用属性 group，如示例中第 10 行的 conf 节点，设置单个 pin 或多个不在一个 group 中的 pin 时使用属性 **pin**，如 15 行的 conf-rx 和 19 行的 conf-tx。

group 属性设上面 mux 节点中的一样。

pin：使用这个 conf 的 pin 脚列表。可选的值为：MIO0 – MIO77。

conf 节点还有一些可选属性：

io-standard：设置 io 电平标准，可选值为：IO_STANDARD_LVCMOS33、IO_STANDARD_LVCMOS18。

bias-disable、bias-high-impedance、bias-pull-up：设置 pin 脚偏置，不需要赋值，只需要在对应的 conf 节点中包含需要的偏置属性即可。

slew-rate：设置转换速率，等于 0 时为 slow，等于 1 时为 fast。

low-power-disable、low-power-enable：使能低功耗模式，在对应的 conf 节点中包含需要的偏置属性即可。

schmitt-cmos：选择使用 schmitt 或者 cmos，可选值为 PIN_INPUT_TYPE_SCHMITT、PIN_INPUT_TYPE_CMOS。

还有其他的可选属，可以在内核源码目录 Documentation/devicetree/bindings/pinctrl/pinctrl-binding.txt 文件中，搜索”== Generic pin configuration node content ==”词条查看。

4.1.2 pinctrl 子系统的使用

在 pinctrl 节点中添加了设备子节点后，系统还不会去初始化这个设备，我们需要在客户设备(client device)节点中去调用他。举个例子：

```
1.  device {
2.      pinctrl-names = "active", "idle";
3.      pinctrl-0 = <&pinctrl_uart1_default>;
4.      pinctrl-1 = <&state_1_node_a &state_1_node_b>;
5.  };
```

这个设备树中的 pinctrl_uart1_default， state_1_node_a， state_1_node_b 都是 pinctl 节点。

device 是客户设备节点，是调用 pinctrl 节点的设备。

pinctrl-0 是必选属性，是这个设备对应 pin 的 pinctrl 节点引用，如果这个设备对应了多个 pinctrl 节点，那就增加属性 **pinctrl-0~n**。如果一个 pin 对应了多个 pinctrl，就如第 4 行的 pinctrl-1 属性，附上多个 pinctrl 节点。

pinctrl-names 是下面 pinctrl0~n 的名称列表，命名是自定的但要尽量易读并贴合实际。

4.2 gpio 子系统

pinctrl 子系统中，如果把功能配置成 gpio 的话，就需要用到 gpio 子系统了。gpio 子系统帮我们实现了 gpio 初始化，并提供了一些接口函数给我们操作 gpio。我们要做的同样只是在设备

树中配置好信息，然后了解一下接口函数的用法即可。

4.2.1 设备树中的 gpio 子系统

在 petalinux 工程的“components/plnx_workspace/device-tree/device-tree/zynqmp.dtsi”中，能找到 petalinux 帮我生成的 gpio 节点，如下：

```
1.  gpio: gpio@ff0a0000 {
2.      compatible = "xlnx,zynqmp-gpio-1.0";
3.      status = "disabled";
4.      #gpio-cells = <0x2>;
5.      interrupt-parent = <&gic>;
6.      interrupts = <0 16 4>;
7.      interrupt-controller;
8.      #interrupt-cells = <2>;
9.      reg = <0x0 0xff0a0000 0x0 0x1000>;
10.     gpio-controller;
11.     power-domains = <&zynqmp_firmware PD_GPIO>;
12. };
```

interrupt-parent、interrupts、interrupt-controller、#interrupt-cells：这四个属性是中断相关的，等之后讲中断的时候再说。

#gpio-cells：指代引用这个 gpio 节点时需要两个 cell，他的值必须是 2，即引用节点时需要有两个 cell，第一个 cell 为 gpio 编号，第二个 cell 用于指顶可选参数，第二个 cell 在 zynqmp 中没有用到填 0 即可。

compatible：兼容性，必须为” xlnx, zynqmp-gpio-1.0”。

gpio-controller：指定该节点为 gpio 控制器。

reg：这组 gpio 的寄存器首地址和范围。

4.2.1.1 设备节点中引用 gpio 节点

和 pinctl 类似，我们需要在设备节点中应用 gpio 节点，这个也有固定的格式，举例如下

```
1.  alinxled {
2.      compatible = "alinx-led";
3.      pinctrl-names = "default";
4.      pinctrl-0 = <&pinctrl_led_default>;
5.      led-gpios = <&gpio 44 GPIO_ACTIVE_HIGH>;
6.  };
```

第五行就是我们引用 gpio 节点的方法，属性名称“led-gpios”后面的“-gpios”是固定的，“-”前面的部分可以自定义，这里 gpio 用于 led 所以前面就叫 led。属性的值有三个，第一个是应用 gpio 节点，第二个是 gpio 号，第三个值是 io 的默认状态，这是个宏定义，所以我们的设备树中要应用这个宏定义所在的头文件。

4.2.2 gpio 接口函数

gpio 子系统提供接口函数来代替我们直接对寄存器的操作把驱动程序再次分离分层。这节我们介绍一些常用的 gpio 接口函数。在 2020.1 对应的 linux 内核中，gpio 子系统有一套新的接口，旧的接口仍然可以使用，这里我们介绍新的。

4.2.2.1 gpiod_get_index()

gpiod_get_index()函数用于获取获取 gpio 描述符，函数原型如下：

static inline struct gpio_desc *__must_check
gpiod_get_index(struct device *dev,
const char *con_id,
unsigned int idx,
enum gpiod_flags flags)

返回值：struct gpio_desc *：这个结构体是新的 gpio 接口中新增的，用于表示一个 gpio，也是用这个描述符来设置和控制 gpio。

输入参数：

dev：对应的设备，他的 of_node 成员必须赋值，且对应的设备树节点中必须要有“xxx-gpios”这个属性。

con_id：是一个字符串，必须要是对应的设备树节点中必须要有“xxx-gpios”这个属性中“xxx”。

idx：需要获取的 gpio 在节点中的下标，从 0 开始。

flags：gpio 默认状态。

4.2.2.2 gpiod_direction_output()

设置 gpio 的输入输出方向，函数原型:

int gpiod_direction_output(struct gpio_desc *desc, int value)

desc：gpio 描述符。

value：0 或 1，至于 0 和 1 哪个表示输出那个表是输入和 **gpiod_get_index** 函数的参数 flags 有关。只有两个逻辑，比较简单，研究一下就能明白了。

4.2.2.3 gpiod_set_value()

设置 gpio 的输出状态。两个参数，第一个参数是 gpio 描述符，第二个参数是输出的状态，用 0 和 1 表示。

4.2.2.4 gpiod_put()

注销 gpio 描述符。只有一个输入参数就是 gpio 描述符。

4.3 实验

4.3.1 设备树

打开文件”system-conf.dtsi”，在根节点下添加以下节点内容：

```
#include <dt-bindings/gpio/gpio.h>
#include <dt-bindings/pinctrl/pinctrl-zynqmp.h>

/include/ "system-conf.dtsi"
/ {
    alinxled {
        compatible = "alinx-led";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_led_default>;
        led-gpios = <&gpio 31 GPIO_ACTIVE_HIGH>;
    };
};

/* SD */
&sdhci1 {
    disable-wp;
    no-1-8-v;
};

```

```
20. /* USB  */
21. &dwc3_0 {
22.     status = "okay";
23.     dr_mode = "host";
24. };
25.
26. &gpio {
27.     status = "okay";
28. };
29.
30. &pinctrl0 {
31.     status = "okay";
32.     pinctrl_led_default: led-default {
33.         mux {
34.             groups = "gpio0_31_grp";
35.             function = "gpio0";
36.         };
37.
38.         conf {
39.             pins = "MIO31";
40.             bias-disable;
41.             slew-rate = <SLEW_RATE_SLOW>;
42.             io-standard = <IO_STANDARD_LVCMOS18>;
43.         };
44.     };
45. };
```

第 **1~2** 行包含了两个头文件，IO_STANDARD_LVCMOS33、SLEW_RATE_SLOW、GPIO_ACTIVE_HIGH 这些定义都在这个头文件中，不包含的话便宜会出错。

第 **26~28** 行是 gpio 节点的引用，需要修改 status 属性为”okey”。如果保持 disable 状态，可能会导致 gpio 相关的 of 函数无法使用。

第 **30~45** 行是 pinctrl 节点，同样需要修改 status 属性。然后添加子节点 pinctrl_led_default: led-default，在这个节点下配置实验 led 所用到的 pin 脚。MIO31 在分组 gpio0_31_grp，founction 就是 gpio0。因为另有一个引脚，所以只要一个 conf 节点就行了，转换率设置为 SLEW_RATE_SLOW，电平标准为 IO_STANDARD_LVCMOS33，偏置失能，配置目标为”MIO31”。

第 **6~11** 行则是我们的 led 设备节点，对于 gpio 子系统和 pinctrl 子系统来说，就是客户设备(client device)，通过属性 pinctrl-0 = <&pinctrl_led_default>调用 pin 节点，通过属性 led-gpios = <&gpio 31 GPIO_ACTIVE_HIGH>来绑定 gpio。这个属性的名字 led-gpios 的前缀 led，稍后我们要通过他来获取 gpio 标号。

4.3.2 驱动代码

创建和编译驱动程序的方法参考第一章。

```
1.  #include <linux/module.h>
2.  #include <linux/kernel.h>
3.  #include <linux/fs.h>
4.  #include <linux/init.h>
5.  #include <linux/ide.h>
6.  #include <linux/types.h>
7.  #include <linux/cdev.h>
8.  #include <linux/device.h>
9.  #include <linux/of.h>
10. #include <linux/gpio/consumer.h>
11.
12. #define CHAR_DEV_MAJOR 200
13. #define CHAR_DEV_NAME  "led_dev"
14.
15. struct alinx_char_dev{
16.     dev_t              devid;
17.     struct cdev        cdev;
18.     struct class       *class;
19.     struct device      *device;
20.     struct device_node *nd;
21.     struct gpio_desc   *gpio_d;
```

```
};

static struct alinx_char_dev alinx_char = {
    .cdev = {
        .owner = THIS_MODULE,
    },
};

static int char_dev_open(struct inode *inode_p, struct file *file_p)
{
    printk("char dev open\n");
    return 0;
}

static int char_dev_release(struct inode *inode_p, struct file *file_p)
{
    printk("char dev release\n");
    return 0;
}

static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t *l
off_t_p)
{
    int ret = 0;
    char lc_buf = 0;

    ret = copy_from_user(&lc_buf, buf, len);
    if(ret != 0)
    {
        printk("copy_from_user failed\n");
        return -1;
    }

    if(len != 1)
    {
        printk("len err\n");
        return -2;
    }

    if(1 == lc_buf)
    {
        gpiod_set_value(alinx_char.gpio_d, 1);
    }
    else if(0 == lc_buf)
    {
        gpiod_set_value(alinx_char.gpio_d, 0);
    }
    else
    {
        printk("data err\n");
    }

    return 0;
}

static struct file_operations char_dev_opt = {
    .owner = THIS_MODULE,
    .open = char_dev_open,
    .write = char_dev_write,
    .release = char_dev_release,
};

static int __init char_drv_init(void)
{
    int ret = 0;

    ret = alloc_chrdev_region(&alinx_char.devid, 0, 1, CHAR_DEV_NAME);
    if(ret < 0)
    {
        goto err;
    }

    cdev_init(&alinx_char.cdev, &char_dev_opt);

    ret = cdev_add(&alinx_char.cdev, alinx_char.devid, 1);
    if(ret < 0)
    {
        goto err;
    }
```

```
100.
101.    alinx_char.class  = class_create(THIS_MODULE, "alinx_char_dev");
102.    if(IS_ERR(alinx_char.class))
103.    {
104.        ret = PTR_ERR(alinx_char.class);
105.        goto err;
106.    }
107.
108.    alinx_char.device = device_create(alinx_char.class, NULL, alinx_char.devid, NULL, CHAR_DEV_N
    AME);
109.    if(IS_ERR(alinx_char.device))
110.    {
111.        ret = PTR_ERR(alinx_char.device);
112.        goto err;
113.    }
114.
115.    alinx_char.device->of_node = of_find_node_by_path("/alinxled");
116.    alinx_char.gpio_d = gpiod_get_index(alinx_char.device, "led", 0, GPIOD_OUT_HIGH);
117.    gpiod_direction_output(alinx_char.gpio_d, 1);
118.
119. err:
120.    return ret;
121. }
122.
123. static void __exit char_drv_exit(void)
124. {
125.    gpiod_put(alinx_char.gpio_d);
126.
127.    cdev_del(&alinx_char.cdev);
128.
129.    device_destroy(alinx_char.class, alinx_char.devid);
130.
131.    class_destroy(alinx_char.class);
132.
133.    unregister_chrdev_region(alinx_char.devid, 1);
134. }
135.
136. module_init(char_drv_init);
137. module_exit(char_drv_exit);
138.
139. MODULE_AUTHOR("Alinx");
140. MODULE_ALIAS("gpio_led");
141. MODULE_DESCRIPTION("GPIO LED driver");
142. MODULE_VERSION("v1.0");
143. MODULE_LICENSE("GPL");
```

相比之前的驱动，我们把寄存器相关的操作都去掉了。

21 行在设备结构体中加入 gpio 描述符成员；

115~117 行在驱动入口函数中调用 gpiod_get_index 和 gpiod_direction_output 初始化 gpio，在这之前需要先获取设备树节点赋值給“device->of_node”；

42 行的 write 函数中调用 gpiod_set_value 来控制 gpio 输出高低电平，控制 led 亮灭；

125 行在驱动出口函数中，调用 gpiod_put 函数释放 gpio 描述符。

4.3.3 测试代码

测试可以直接使用第一章的。

4.3.4 运行测试

测试方法和第二章相同，只要能成功点亮 led 就成功了。

```
root@petalinux:/mnt# insmod ps-led.ko
root@petalinux:/mnt# ./char_drv_test /dev/led_dev off
[   91.517861] char dev open
ps_led1 off
[   91.520546] char dev release
root@petalinux:/mnt#
```

第五章 并发的处理

所谓并发就是指多个任务同时访问同一个资源，内存就是资源。内存出错，很有可能导致系统崩溃，所以必须要极力避免并发问题的发生。

并发访问产生的原因众多，比如多线程访问、抢占、中断等等，这些都是系统机制很难避免。要避免并发访问带来的问题，不如去识别出会被同时访问的**共享资源**以及会访问共享资源的程序即**临界区**，并做好保护。当然很多时候识别出需要保护的资源并非易事，这我们只能从不断的积累中去总结。

这一章我们使用的 petalinux 工程是“Linux 基础教程”中“第一章 使用 Petalinux 定制 Linux 系统”制作的。

5.3Linux 对并发的处理

识别出共享资源后，我们就可以使用 Linux 内核提供的保护共享资源机制来避免并发访问。这节我们介绍几种内核提供的并发处理方法。

5.3.1 原子操作

原子操作是指不能被分割或打断的操作，Linux 内核提供了原子变量和系列函数来实现原子操作。文件 include/linux/types.h 中，原子变量定义如下：

```
1.  typedef struct {
2.      int counter;
3.  } atomic_t;
```

定义原子变量的方式为：

```
1.  atomic_t a = ATOMIC_INIT(x);
```

ATOMIC_INIT(x)是用于原子变量初始化的宏定义，x 是我们赋予的初值，不需要的初值的话可以省略。

原子变量的读写操作也要内核提供的接口函数，除了上面说的 ATOMIC_INIT(x)外，还有些常用的接口函数如下：

int atomic_read(atomic_t *v)：读取 v 并且返回。

void atomic_set(atomic_t *v, int i)：把 i 写入 v。

void atomic_add(int i, atomic_t *v)：v 的值减去 i。

void atomic_sub(int i, atomic_t *v)：v 的值加上 i。

void atomic_inc(atomic_t *v)：v 自增。

void atomic_dec(atomic_t *v)：v 自减。

int atomic_inc_return(atomic_t *v)：v 自增并返回。

int atomic_dec_return(atomic_t *v)：v 自减并返回。

当然还不止这些，除了对整型的操作，内核还提供了原子的位操作，等需要的时候再去了解吧。

具体用法我们放到之后的实验中再去分析。

5.3.2 锁机制

原子操作只能对整形变量和位起到保护作用，很多场景都不适用。比如需要保护结构多变的结构体变量时就需要用到其他的机制，比如这节要讲的锁机制。内核中的锁也用很多种，先介绍一种常用的自旋锁。

自旋锁的机制是，某个线程要访问共享资源时，需要先获取相应的锁即上锁，只要不释放这个锁即解锁，别的线程就无法获取锁也就无法访问共享资源。此时需要却没有获取到自旋锁的线程就会一直处于等带状态(阻塞)。线程的等待状态会浪费很多处理器资源，所以占用自旋锁的临界区要尽量的轻便，更不能调用或存在阻塞的函数或逻辑。

锁是一个形象比喻，说到底，也就是个结构体，定义如下：

```
typedef struct spinlock {
    union {
        struct raw_spinlock rlock;
#ifdef CONFIG_DEBUG_LOCK_ALLOC
# define LOCK_PADSIZE (offsetof(struct raw_spinlock, dep_map))
        struct {
            u8 __padding[LOCK_PADSIZE];
            struct lockdep_map dep_map;
        };
#endif
    };
} spinlock_t;
```

同样的有了数据结构，还得有相应的操作函数，常用的自旋锁接口函数有：

int spin_lock_init(spinlock_t *lock)：自旋锁初始化。

void spin_lock(spinlock_t *lock)：获取自旋锁。

void spin_unlock(spinlock_t *lock)：释放自旋锁。

int spin_trylock(spinlock_t *lock)：获取自旋锁，没有获取到则返回 0。

int spin_is_locked(spinlock_t *lock)：检查自旋锁是否已被获取，返回 0 则已被获取，其他值则未被获取。

void spin_lock_irq(spinlock_t *lock)：禁用本地中断，并获取自旋锁。

void spin_unlock_irq(spinlock_t *lock)：恢复本地中断，并释放自旋锁。

void spin_lock_irqsave(spinlock_t *lock,unsigned long flags)：保存中断状态，禁用本地中断，并获取自旋锁，flags 为中断状态。

void spin_unlock_irqrestore(spinlock_t*lock, unsigned long flags)：恢复中断至保存的状态，恢复本地中断，并获释放旋锁。

使用自旋锁，需要注意：

1.使用自旋锁的临界区必定不能进入休眠。自旋锁被成功获取后，内核会停用抢占机制。假如现在有个临界区 A 获取了自旋锁，然后进入休眠主动放弃了 CPU 使用权，线程 B 开始运行。线程 B 的临界区也想访问共享数据，但是自旋锁已被占用，B 线程的临界区就一直处于等待自旋锁的状态，而此时内核抢占被禁止，临界区 A 无法获得主动权释放锁，从而发生了死锁。

2.临界区内要避免中断抢占。假设现在有线程获取了自旋锁，在释放之前，被中断抢占了，如果中断也需要获取共享数据，也去申请自旋锁，那中断就会进入等待状态，那就大事不妙了，直接死锁。这里要避免终端抢占的办法，就是使用上面介绍的 spin_lock_irq 函数和

spin_lock_irqsave 函数来申请自旋锁，禁用本地中断。

3.临界区要尽量短。原因上面有解释过了。

使用示例：

```
1.  spinlock_t lock;
2.  spin_lock (&gpioled.lock);  //上锁
3.  /* 临界区 */
4.  spin_unlock (&gpioled.lock);  //解锁
```

上锁和解锁中间就是临界区。

5.3.3 信号量

与自旋锁相比，信号量有两个优势：

1.信号量可以让等待信号量的线程进入休眠，减少 CPU 的占用；

2.信号量支持对多个线程同时访问共享资源。

信号量结构体定义如下：

```
1.  struct semaphore {
2.      raw_spinlock_t lock;
3.      unsigned int count;
4.      struct list_head wait_list;
5.  };
```

元素 count 即指信号量支持同时访问共享资源的线程数。

常用的信号量接口函数有：

void sema_init(struct semaphore *sem, int val)：初始化信号量，设置信号值(同时访问个数)为 val。

void down(struct semaphore *sem)：获取信号量，失败时会进入休眠且不可以被信号打断，不能用于中断，当前版本的 linux 已经不推荐用这个函数来上锁了。

int down_trylock(struct semaphore *sem)：获取信号量，不会进入休眠，成功返回 0。

int down_interruptible(struct semaphore *sem)：获取信号量，失败时会进入休眠但可以被信号打断，不能用于中断，休眠被打断时返非 0 值。

void up(struct semaphore *sem)：释放信号量。

使用信号量要注意的地方：

1.线程休眠后会切换线程，如果占用信号量的临界区很短，导致频繁切换线程，也会带来大量的开销，因此与自旋锁相反，**信号量不适用于共享资源使用时间很短的场合**；

2.中断不能休眠，因此**中断中不能使用会进入休眠的方式申请信号量**；

3.信号量**允许多个线程同时访问共享资源时(count > 1)，不可用于互斥访问**(互斥访问指一次只能有一个线程访问共享资源)。

使用示例：

```
1.  struct semaphore sem;
2.  sema_init(&sem, 1);
3.  down_interruptible(&sem);
4.  /* 临界区 */
5.  up(&sem);
```

5.4 实验

5.4.1 驱动代码

以上一章的驱动代码为例，首先识别出共享资源。led 设备的驱动，那共享资源就是 led 设备了，如果有多个应用程序来调用这个驱动，最终就是 led 设备会被多个应用程序操作。再进一步的说，就是 led 设备操作文件，在操作这个设备时，不能让别的应用程序再操作。而临界区就是从占用这个设备开始到释放这个设备为止。

这一章有三个机制，但重复的代码很多，为了简便，我把三个机制都写在一个代码里。用宏定义开关来隔开。代码如下：

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/ide.h>
#include <linux/types.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/gpio/consumer.h>

#define CHAR_DEV_MAJOR 200
#define CHAR_DEV_NAME  "led_dev"

#define ATOMIC_T_ON
//#define SPINKLOCK_T_ON
//#define SEMAPHORE_ON

struct alinx_char_dev{
    dev_t               devid;
    struct cdev         cdev;
    struct class        *class;
    struct device       *device;
    struct device_node *nd;
    struct gpio_desc    *gpio_d;

#ifdef ATOMIC_T_ON
    atomic_t            lock;
#endif

#ifdef SPINKLOCK_T_ON
    spinlock_t          lock;
    int                 src_state;
    unsigned long       irq_state;
#endif

#ifdef SEMAPHORE_ON
    struct semaphore    lock;
#endif
};

static struct alinx_char_dev alinx_char = {
    .cdev = {
        .owner = THIS_MODULE,
    },
};

static int char_dev_open(struct inode *inode_p, struct file *file_p)
{
    int ret = 0;

#ifdef ATOMIC_T_ON
    if(1 == atomic_add_return(1, &alinx_char.lock))
    {
        printk("char dev open\n");
    }
    else
```

```
58.     {
59.         atomic_set(&alinx_char.lock, 1);
60.         printk("%s resource busy\n", CHAR_DEV_NAME);
61.             ret = -1;
62.     }
63. #endif
64.
65. #ifdef SPINKLOCK_T_ON
66.     spin_lock_irqsave(&alinx_char.lock, alinx_char.irq_state);
67.     if(alinx_char.src_state)
68.     {
69.             printk("%s resource busy\n", CHAR_DEV_NAME);
70.             ret = -1;
71.     }
72.     else
73.     {
74.         alinx_char.src_state = 1;
75.         printk("char dev open\n");
76.     }
77.     spin_unlock_irqrestore(&alinx_char.lock, alinx_char.irq_state);
78. #endif
79.
80. #ifdef SEMAPHORE_ON
81.     ret = down_interruptible(&alinx_char.lock);
82.     if(ret)
83.     {
84.             printk("%s resource busy\n", CHAR_DEV_NAME);
85.             ret = -1;
86.     }
87.     else
88.     {
89.         printk("char dev open\n");
90.     }
91. #endif
92.
93.     return ret;
94. }
95.
96. static int char_dev_release(struct inode *inode_p, struct file *file_p)
97. {
98. #ifdef ATOMIC_T_ON
99.     atomic_set(&alinx_char.lock, 0);
100. #endif
101.
102. #ifdef SPINKLOCK_T_ON
103.     spin_lock_irqsave(&alinx_char.lock, alinx_char.irq_state);
104.     alinx_char.source_status = 0;
105.     spin_unlock_irqrestore(&alinx_char.lock, alinx_char.irq_state);
106. #endif
107.
108. #ifdef SEMAPHORE_ON
109.     up(&alinx_char.lock);
110. #endif
111.
112.     printk("char dev release\n");
113.     return 0;
114. }
115.
116. static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t *l
    off_t_p)
117. {
118.     int ret = 0;
119.     char lc_buf = 0;
120.
121.     ret = copy_from_user(&lc_buf, buf, len);
122.     if(ret != 0)
123.     {
124.         printk("copy_from_user failed\n");
125.         return -1;
126.     }
127.
128.     if(len != 1)
129.     {
130.         printk("len err\n");
131.         return -2;
132.     }
133.
134.     if(1 == lc_buf)
135.     {
```

```
136.         gpiod_set_value(alinx_char.gpio_d, 1);
137.     }
138.     else if(0 == lc_buf)
139.     {
140.         gpiod_set_value(alinx_char.gpio_d, 0);
141.     }
142.     else
143.     {
144.         printk("data err\n");
145.     }
146.
147.     return 0;
148. }
149.
150. static struct file_operations char_dev_opt = {
151.     .owner = THIS_MODULE,
152.     .open = char_dev_open,
153.     .write = char_dev_write,
154.     .release = char_dev_release,
155. };
156.
157. static int __init char_drv_init(void)
158. {
159.     int ret = 0;
160.
161.     ret = alloc_chrdev_region(&alinx_char.devid, 0, 1, CHAR_DEV_NAME);
162.     if(ret < 0)
163.     {
164.         goto err;
165.     }
166.
167.     cdev_init(&alinx_char.cdev, &char_dev_opt);
168.
169.     ret = cdev_add(&alinx_char.cdev, alinx_char.devid, 1);
170.     if(ret < 0)
171.     {
172.         goto err;
173.     }
174.
175.     alinx_char.class  = class_create(THIS_MODULE, "alinx_char_dev");
176.     if(IS_ERR(alinx_char.class))
177.     {
178.         ret = PTR_ERR(alinx_char.class);
179.         goto err;
180.     }
181.
182.     alinx_char.device = device_create(alinx_char.class, NULL, alinx_char.devid, NULL, CHAR_DEV_N
    AME);
183.     if(IS_ERR(alinx_char.device))
184.     {
185.         ret = PTR_ERR(alinx_char.device);
186.         goto err;
187.     }
188.
189.     alinx_char.device->of_node = of_find_node_by_path("/alinxled");
190.     alinx_char.gpio_d = gpiod_get_index(alinx_char.device, "led", 0, GPIOD_OUT_HIGH);
191.     gpiod_direction_output(alinx_char.gpio_d, 1);
192.
193. #ifdef ATOMIC_T_ON
194.     atomic_set(&alinx_char.lock, 0);
195. #endif
196.
197. #ifdef SPINKLOCK_T_ON
198.     spin_lock_init(&alinx_char.lock);
199.     alinx_char.src_state = 0;
200. #endif
201.
202. #ifdef SEMAPHORE_ON
203.     sema_init(&alinx_char.lock, 1);
204. #endif
205.
206. err:
207.     return ret;
208. }
209.
210. static void __exit char_drv_exit(void)
211. {
212.     gpiod_put(alinx_char.gpio_d);
213.
```

```
214.     cdev_del(&alinx_char.cdev);
215.
216.     device_destroy(alinx_char.class, alinx_char.devid);
217.
218.     class_destroy(alinx_char.class);
219.
220.     unregister_chrdev_region(alinx_char.devid, 1);
221. }
222.
223. module_init(char_drv_init);
224. module_exit(char_drv_exit);
225.
226. MODULE_AUTHOR("Alinx");
227. MODULE_ALIAS("gpio_led");
228. MODULE_DESCRIPTION("GPIO LED driver");
229. MODULE_VERSION("v1.0");
230. MODULE_LICENSE("GPL");
```

与上一章相比的改动主要集中在：

1.结构体定义中增加成员；

2.驱动入口函数中，添加初始化操作；

3.open 函数中加锁；

4.release 函数中释放锁。

通过同定义开关分成了三部分，分别对应原子操作、自旋锁、信号量，他们的用法都比较相似。按顺序来分析。

原子操作：

看宏定义 ATOMIC_T_ON 相关的部分，思路是：

a：**28** 行定义一个原子变量，并把它作为共享资源使用状态的标志，0 为未被占用，其他值则已被占用。

b：**194** 行在驱动入口函数中初始化原子变量为 0，即共享资源未被占用。

c：**53** 行在 open 函数中，使用“atomic_add_return”函数先判断资源占用状态，“atomic_add_return”函数会使原子变量加 1，如果返回值不是 1，也就是原子变量不为 0，则资源已被占用，返回 busy 并将原子变量再置为 1。否则资源是空闲的，然后 open 函数返回 0，表示调用共享资源设备节点成功。

d：**160** 行在 release 函数中，应用程序释放了共享资源，此时把原子变量置为 0，共享资源恢复为空闲的状态。

自旋锁：

再看宏定义 SPINKLOCK_T_ON 相关的部分。

前面介绍过自旋锁的临界区要尽量短，正常情况下，从应用程序调用 open 到应用程序调用 close 之间都是临界区。这中间的时间不由驱动来决定，有可能会拉的很长，所有在 open 函数中上锁，在 release 函数中解锁就有临界区很长的风险。可以借助上面原子变量实验中用到的标志 flag 的思想来实现：

a：**32** 行定义一个自旋锁，同时 **33** 行定义一个标志 src_state 来记录资源的使用状态，0 为未被占用，其他值则已被占用。**34** 行定一个 irq_state 无符号长整型变量在存放中断状态。

b：**66** 行在驱动入口函数中用 spin_lock_irqsave 函数初始化自旋锁并且暂停中断保存中断状态。然后 67 行判断资源占用状态，如果不为 0 资源被占用，否则资源未被占用，把资源状态置为 1。

c：**77** 行直接解锁。这样一来，及时资源被占用了，线程也不会一直等待了。

d：**103** 行 release 函数中，获取锁后，设置资源占用状态为 0，然后就解锁。

信号量：

信号量的用法就很简单了，因为信号量的临界区要长，所以我们放心的在 open 中上锁，在 release 中解锁，就不需要用标志了。当然临界区到底有多长，还是由应用程决定的。

5.4.2 测试代码

测试代码在上一章的基础上稍作修改。新建 QT 工程名为”axledlong_test”，新建 main.c，输入下列代码：

```
1.  #include <stdio.h>
2.  #include <string.h>
3.  #include <unistd.h>
4.  #include <fcntl.h>
5.
6.  int main(int argc, char **argv)
7.  {
8.      int fd;
9.      char buf;
10.     int count;
11.
12.     if(3 != argc)
13.     {
14.         printf("none para\n");
15.         return -1;
16.     }
17.
18.     fd = open(argv[1], O_RDWR);
19.     if(fd < 0)
20.     {
21.         printf("Can't open file %s\r\n", argv[1]);
22.         return -1;
23.     }
24.
25.     if(!strcmp("on",argv[2]))
26.     {
27.         printf("ps_led1 on\n");
28.         buf = 1;
29.         write(fd, &buf, 1);
30.     }
31.     else if(!strcmp("off",argv[2]))
32.     {
33.         printf("ps_led1 off\n");
34.         buf = 0;
35.         write(fd, &buf, 1);
36.     }
37.     else
38.     {
39.         printf("wrong para\n");
40.         return -2;
41.     }
42.
43.     count = 20;
44.     while(count --)
45.     {
46.         sleep(1);
47.     }
48.
49.     close(fd);
50.     return 0;
51. }
```

在 43 到 47 行添加代码，用 20 秒的 sleep 模拟以下对资源 20 秒的占用。其他和之前一样。

5.4.3 运行测试

加载驱动的步骤和前面一样。

用下列命令来打开测试程序：

```
./char_test_conc   /dev/led_dev   off &
./char_test_conc   /dev/led_dev   on
```

“&”表示在后台运行 app，可以使用 top 命令查看后台正在运行的程序。

可以看出，先执行 app 关闭 led，这个线程就占用了设备节点。再执行 on，app 返回了 resource busy，而不是直接打开 led。等输出” char dev release”后，说明先执行的 app 线程已经释放了资源，再执行 on，led 就被打开了。

自旋锁的现象与原子变量的实验现象是一致的。信号量则会略有区别，因为信号量的实现中，会休眠去等待资源，因此 on 时不会直接返回错误，而是会等输出” char dev release”后，再自动去打开 led。

```
root@petalinux:/mnt# insmod ps-led.ko
root@petalinux:/mnt# ./char_test_conc /dev/led_dev off &
[1] 802
root@petalinux:/mnt# [  182.617274] char dev open
ps_led1 off
./char_test_conc /dev/led_dev on
[  187.930735] led_dev resource busy
Can't open file /dev/led_dev
root@petalinux:/mnt# [  192.620048] char dev release
```

第六章 定时器

本章简单介绍 Linux 内核提供的定时器及其用法。

这一章我们使用本教程“pinctrl 和 gpio 子系统”一章的 petalinux 工程。

6.1 Linux 内核中的定时器

内核中定时器的实现依赖于硬件定时器。硬件定时器提供固定频率的时钟源并产生中断，系统使用这个中断来计时。而内核中的定时器功能，则是在这个计时的基础上实现的。

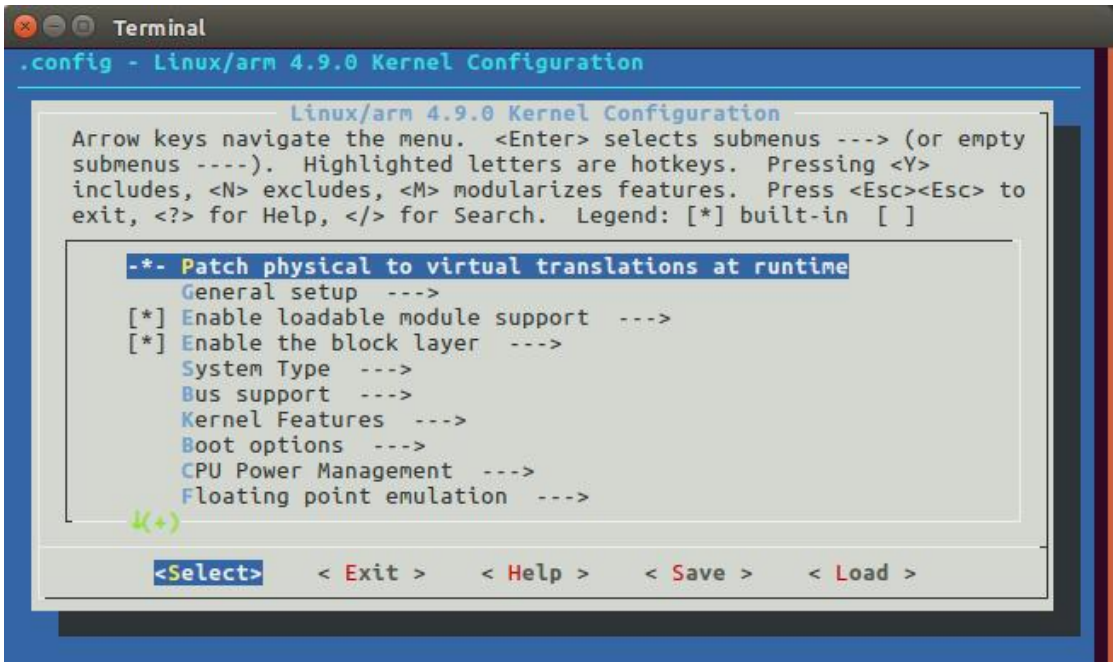
6.1.1 系统频率

产生系统中断的频率就是系统频率，也叫系统节拍即每秒的节拍数。系统节拍是可设置的，在 petalinux 中设置方法为：

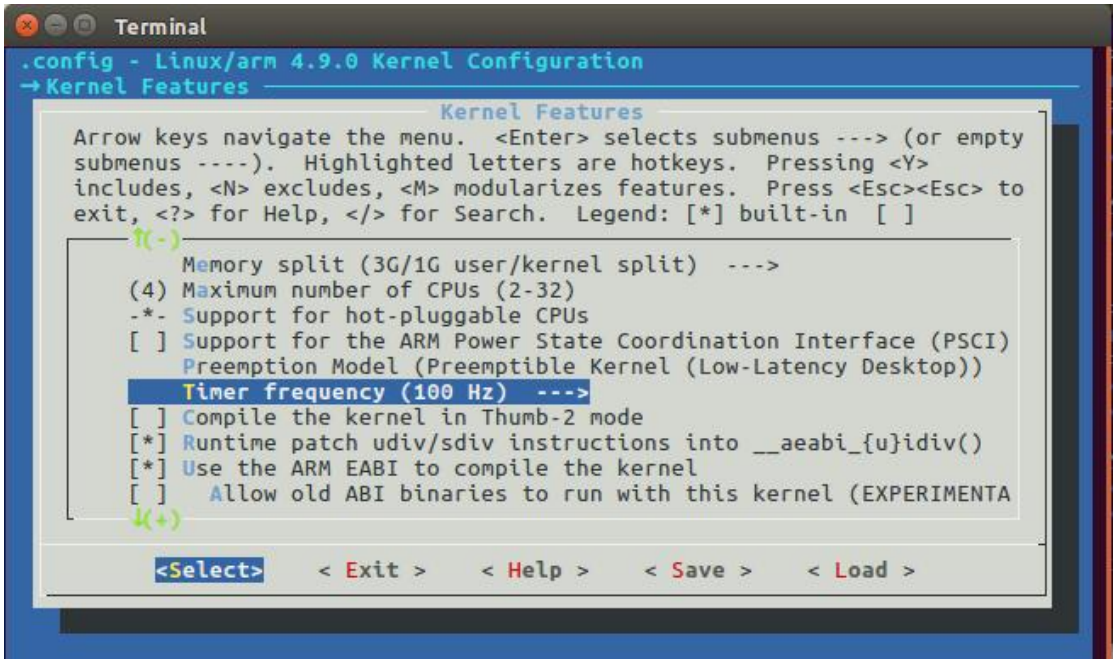
在 petalinux 工程目录中打开终端输入命令：

```
petalinux-config -c kernel
```

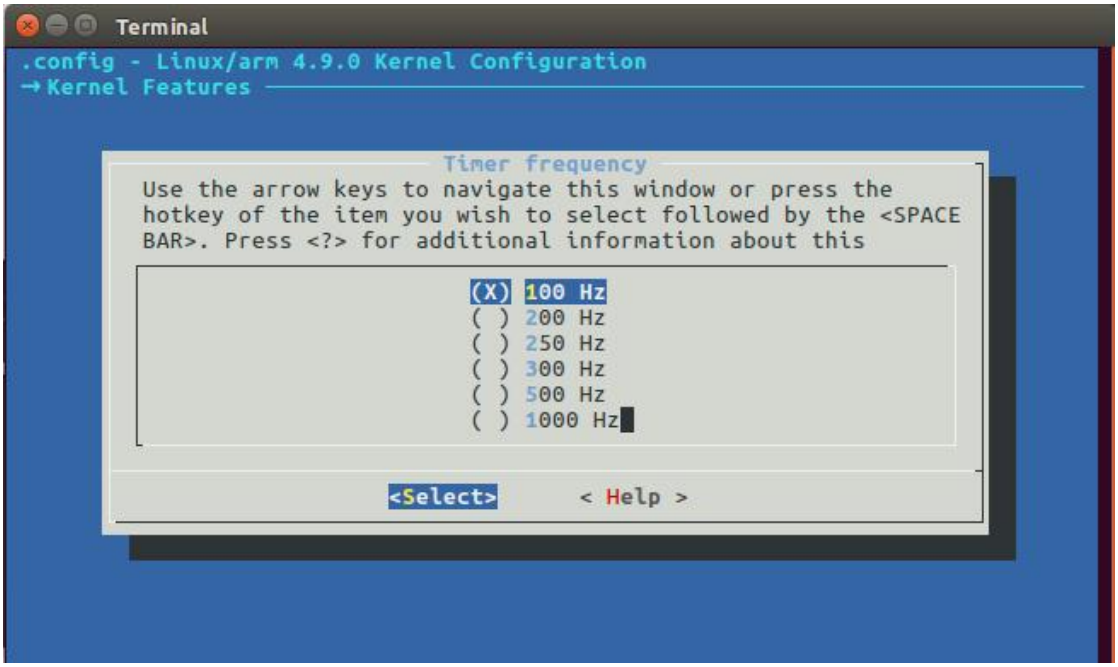
等待一段时间后，会弹出交互界面如下图：



用上下方向键移动选项置”Kernel Features”，按回车进入，如下图：



再按下方向键选择选项”Timer frequency(100 Hz)”按回车(这个选项被藏在很下面，不在一开始的界面里)，就能看到系统频率的选项：



这里默认选择的是 100Hz，每秒 100 个节拍。

系统频率越高，时钟精度也越高，100Hz 的时间进度能达到 10ms，而 1000Hz 能达到 1ms。同时中断产生的频率增加，cpu 的负担也就越大这需要根据实际情况去取舍，我们的实验使用 100Hz 就足够了。

系统时钟定义在 include/asm-generic/param.h 中，为宏定义 HZ，他最终会对应到我们设置的系统时钟值。

6.1.2 节拍数

系统计时的方式就是记录节拍数。系统运行的时间=节拍数/系统频率，单位为秒。

内核中用全局变量 jiffies 来记录节拍书，定义在文件 include/linux/jiffies.h 中：

```
extern u64 __jiffy_data jiffies_64;
extern unsigned long volatile __jiffy_data jiffies;
```

jiffies_64 用于 64 位的系统，jiffies 用于 32 位系统。既然是用来计数的变量，就会有溢出的风险。64 位且不提，32 位的 jiffies 在 100Hz 的系统频率下，500 天左右就会溢出回绕，在 1000Hz 的频率下 50 天左右就会回绕。回绕会带来什么问题，举个例子：

```
1.  unsigned long time_mark = jiffies + HZ * 5;
2.
3.  /* 业务处理 */
4.
5.  if(time_mark > jiffies)
6.  {
7.      /* 未超时 */
8.  }
9.  else
10. {
11.     /* 超时 */
12. }
```

这个例子中，先设定一个时间 time_mark 为当前节拍数再加上 5 秒的节拍，也就是 5 秒后系统节拍 jiffies 会到达的值。这个示例代码的目的是如果业务处理的时间超过 5 秒就执行超时的处理。极端情况下，比如此时 jiffies 为 0xFFFFFDA7，time_mark 则等于 0xFFFFFF9B，如果业务处理时间用了 7 秒，jiffies 会因为超过最大值 0xFFFFFFFF 发生反转，从 0 开始重新计数变成 0x64，这个值远远小于 time_mark，明明超时了，却进入了未超时的条件。

对此，内核的通过把数据强转无符号数强转成有符号数巧妙的规避了回绕的问题，不用传统的比较运算符去比较，而使用下面几个宏来代替：

```
#define time_after(unknown,known) ((long)(known) - (long)(unknown)<0)
#define time_before(unkonwn,known) ((long)(unknown) - (long)(known)<0)
#definetime_after_eq(unknown,known) ((long)(unknown) - (long)(known)>=0)
#define time_before_eq(unknown,known) ((long)(known)
-(long)(unknown)>=0)
```

以 time_after_eq 为例，参数 unknow 一般指 jiffies，know 通常是我们设定的时间，如上面例子中的 time_mark。after_eq 即大于等于，如果 unknow 大于等于 know，即 jiffies 大于 time_mark，则返回真，对于上面的例程来说就是超时了。

6.1.3 内核定时器及相关函数

6.1.3.1 timer_list

Linux 内核中定时器用数据结构 timer_list 来表示。定义在 include/linux/timer.h 中：

```
1.   struct timer_list {
2.      /*
3.       * All fields that change during normal runtime grouped to the
4.       * same cacheline
5.       */
6.      struct hlist_node   entry;
7.      unsigned long       expires;
8.      void            (*function)(unsigned long);
9.      unsigned long       data;
10.     u32         flags;
11.
12. #ifdef CONFIG_TIMER_STATS
13.     int         start_pid;
14.     void            *start_site;
15.     char            start_comm[16];
16. #endif
17. #ifdef CONFIG_LOCKDEP
18.     struct lockdep_map  lockdep_map;
```

```
19. #endif
20. }
```

重要的成员介绍：

expires：设置定时器超时时间点的节拍数，如果我们需要定义一个 10s 的定时器，那 expires 就应该等于 jiffies+10*HZ。为了方便，内核还提供了毫秒、微妙、纳秒和 jiffies 互换的系列函数：

```
/*毫秒互换*/
int jiffies_to_msecs(const unsigned long j)
long msecs_to_jiffies(const unsigned int m)
/*微秒互换*/
int jiffies_to_usecs(const unsigned long j)
long usecs_to_jiffies(const unsigned int u)
/*纳秒互换*/
u64 jiffies_to_nsecs(const unsigned long j)
unsigned long nsecs_to_jiffies(u64 n)
```

function：定时器超回调函数指针。超时后的需要做的操作就在这个函数中实现。新版的内核中这个函数的输入参数改为 struct timer_list *，zynqmp 系列驱动实验中内核版本使用用的是 4.19，也就是新的内核。

data：用于设置 function 的输入参数。

PS：新的内核中删除了 9~15 行的成员。

6.1.3.2 定时器相关函数

1. void init_timer(struct timer_list *timer)

初始化定时器，输入参数 timer 即需要初始化的定时器。这个函数在新的内核中被删除了，使用宏 DEFINE_TIMER(_name, _function)或 timer_setup(timer, callback, flags)来代替。

2.void add_timer(struct timer_list *timer)

向内核注册定时器，注册后定时器就开始运行，输入参数 timer 即需要注册的定时器。

3.int del_timer(struct timer_list * timer)

删除定时器，不管定时器是否正在运行，正在运行的 timer 会直接停止计时。输入参数 timer 即需要删除的定时器。删除的定时器若正在运行，则返回 1，否则返回 0。

4.int del_timer_sync(struct timer_list *timer)

删除定时器，正在运行的 timer 会等到使用完成后才删除。输入参数 timer 即需要删除的定时器。删除的定时器若正在运行，则返回 1，否则返回 0。

5.int mod_timer(struct timer_list *timer, unsigned long expires)

修改定时器的 expires 即超时时间，如果定时器不在运行，则激活该定时器。输入参数 timer 为需要修改的定时器。expires 即为修改后的超时时间。

6.2 实验

实验目标是，使用定时器定让 led 闪烁，应用程序可以通过输入参数设置 led 闪烁的周期。

6.2.1 驱动程序

```
#include <linux/module.h>
#include <linux/kernel.h>
```

```
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/ide.h>
#include <linux/types.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/gpio/consumer.h>
#include <linux/timer.h>

#define CHAR_DEV_MAJOR 200
#define CHAR_DEV_NAME  "led_dev"

struct alinx_char_dev{
     dev_t             devid;
     struct cdev       cdev;
     struct class      *class;
     struct device     *device;
     struct device_node *nd;
     struct gpio_desc   *gpio_d;
     struct semaphore   lock;
     struct timer_list  timer;
     unsigned long      timer_cnt;
     char              led_state;
};

static struct alinx_char_dev alinx_char = {
     .cdev = {
          .owner = THIS_MODULE,
     },
};

void timer_function(struct timer_list *timer)
{
     alinx_char.led_state = !alinx_char.led_state;
     gpiod_set_value(alinx_char.gpio_d, alinx_char.led_state);
     mod_timer(&alinx_char.timer, jiffies_64 +
msecs_to_jiffies(alinx_char.timer_cnt));
}

static int char_dev_open(struct inode *inode_p, struct file *file_p)
{
     int ret = 0;

     ret = down_interruptible(&alinx_char.lock);
     if(ret)
     {
          printk("%s wait resource break\n", CHAR_DEV_NAME);
          ret = -1;
     }
     else
     {
          printk("char dev open\n");
     }

     return ret;
}

static int char_dev_release(struct inode *inode_p, struct file *file_p)
{
     up(&alinx_char.lock);
     if(&alinx_char.lock != 1)
     {
          printk("char dev failed\n");
     }
     else
     {
          printk("char dev release\n");
     }

     return 0;
}

static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len,
loff_t *loff_t_p)
{
     int ret = 0;

     ret = copy_from_user(&alinx_char.timer_cnt, buf, len);
```

```
    if(ret != 0)
    {
        printk("copy_from_user failed\n");
        return -1;
    }

    mod_timer(&alinx_char.timer, jiffies_64 +
msecs_to_jiffies(alinx_char.timer_cnt));

    return 0;
}

static struct file_operations char_dev_opt = {
    .owner = THIS_MODULE,
    .open = char_dev_open,
    .write = char_dev_write,
    .release = char_dev_release,
};

static int __init char_drv_init(void)
{
    int ret = 0;

    ret = alloc_chrdev_region(&alinx_char.devid, 0, 1, CHAR_DEV_NAME);
    if(ret < 0)
    {
        goto err;
    }

    cdev_init(&alinx_char.cdev, &char_dev_opt);

    ret = cdev_add(&alinx_char.cdev, alinx_char.devid, 1);
    if(ret < 0)
    {
        goto err;
    }

    alinx_char.class  = class_create(THIS_MODULE, "alinx_char_dev");
    if(IS_ERR(alinx_char.class))
    {
        ret = PTR_ERR(alinx_char.class);
        goto err;
    }

    alinx_char.device = device_create(alinx_char.class, NULL, alinx_char.devid,
NULL, CHAR_DEV_NAME);
    if(IS_ERR(alinx_char.device))
    {
        ret = PTR_ERR(alinx_char.device);
        goto err;
    }

    alinx_char.device->of_node = of_find_node_by_path("/alinxled");
    alinx_char.gpio_d = gpiod_get_index(alinx_char.device, "led", 0,
GPIOD_OUT_HIGH);
    gpiod_direction_output(alinx_char.gpio_d, 1);

    timer_setup(&alinx_char.timer, timer_function, 0);
    add_timer(&alinx_char.timer);

    sema_init(&alinx_char.lock, 1);

err:
    return ret;
}

static void __exit char_drv_exit(void)
{
    del_timer(&alinx_char.timer);

    gpiod_put(alinx_char.gpio_d);

    cdev_del(&alinx_char.cdev);

    device_destroy(alinx_char.class, alinx_char.devid);

    class_destroy(alinx_char.class);

    unregister_chrdev_region(alinx_char.devid, 1);
```

```
}

module_init(char_drv_init);
module_exit(char_drv_exit);

MODULE_AUTHOR("Alinx");
MODULE_ALIAS("gpio_led");
MODULE_DESCRIPTION("GPIO LED driver");
MODULE_VERSION("v1.0");
MODULE_LICENSE("GPL");
```

在 126~127 行驱动入口函数中，初始化一个 timer。

在 write 函数中，从用户空间获取时间间隔然后调用 mod_timer 启动定时器。

在定时器回调函数中反转 led 状态并且再次启动 timer。

6.2.2 测试程序

```
1.  #include "stdio.h"
2.  #include "unistd.h"
3.  #include "sys/types.h"
4.  #include "sys/stat.h"
5.  #include "fcntl.h"
6.  #include "stdlib.h"
7.  #include "string.h"
8.  #include "linux/ioctl.h"
9.
10. int main(int argc, char *argv[])
11. {
12.     int fd, ret;
13.     char *filename;
14.     unsigned int interval_new, interval_old = 0;
15.
16.     /* 验证输入参数个数 */
17.     if(argc != 2)
18.     {
19.         printf("Error Usage!\r\n");
20.         return -1;
21.     }
22.
23.     /* 打开输入的设备文件，获取文件句柄 */
24.     filename = argv[1];
25.     fd = open(filename, O_RDWR);
26.     if(fd < 0)
27.     {
28.         /* 打开文件失败 */
29.         printf("can not open file %s\r\n", filename);
30.         return -1;
31.     }
32.
33.     while(1)
34.     {
35.         /* 等待输入 led 闪烁的时间间隔 */
36.         printf("Input interval:");
37.         scanf("%d", &interval_new);
38.
39.         /* 时间间隔更新了 */
40.         if(interval_new != interval_old)
41.         {
42.             interval_old = interval_new;
43.             /* 写入新的时间间隔 */
44.             ret = write(fd, &interval_new, sizeof(interval_new));
45.             if(ret < 0)
46.             {
47.                 printf("write failed\r\n");
48.             }
49.             else
50.             {
51.                 printf("interval refreshed!\r\n");
52.             }
53.         }
54.         else
55.         {
56.             printf("same interval!");
57.         }
```

```
58.     }
59.     close(fd);
60. }
```

应用程序很简单，就是调用 write 函数把时间间隔传到驱动层就可以了。

6.3.3 内核修改

```
petalinux-config -c kernel
```

进入内核“Kernel Features”：

```
    *** Compiler: aarch64-xilinx-linux-gcc (GCC) 9.2.0 ***
    General setup  --->
[*] Support DMA32 zone
    Platform selection  --->
    Kernel Features  --->
    Boot options  --->
```

找到“Timer frequency”，选择 1000Hz，保存退出，编译工程，打包文件到 SD 卡中。

6.3.4 运行测试

加载驱动，运行应用程序输入不同的时间间隔作为参数，观察 led 的变化。

```
root@petalinux:/mnt# insmod ps-led.ko
root@petalinux:/mnt# ./linux_timer /dev/led_dev
[   88.977754] char dev open
Input interval:1000
interval refreshed!
Input interval:2000
interval refreshed!
Input interval:4000
interval refreshed!
Input interval:100
interval refreshed!
Input interval:4000
interval refreshed!
Input interval:
```

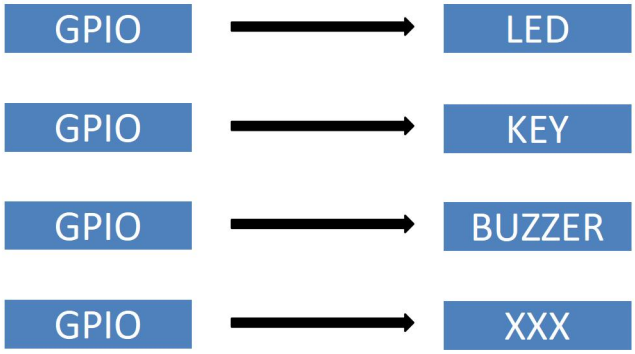
第七章 platform 平台设备

前面一直都是使用字符设备框架来编写简单的IO读写驱动，但是遇到复杂的总线和设备时，仅有字符设备框架是没法应对的。比如，当 SPI 总线挂载了多个设备时，这些设备的驱动该如何实现，SPI 的驱动如何实现，他们之间又如何关联？强行去写也许可以实现功能，但最终的代码会很不环保，复用性很难保证，不符合 Linux 的设计思想。这时就需要用到驱动分离的思想。对此，Linux 内核提供了 platform 设备框架。

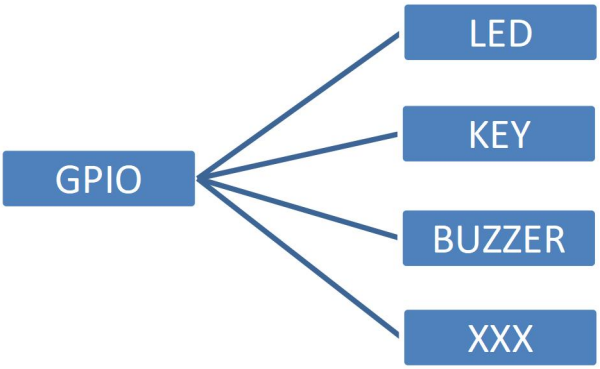
7.1 驱动分离

在说 platform 之前，先简单说说驱动分离的思想。

有没有想过这个问题，我们前面写的点亮 LED 实验中的驱动，到底是 IO 的驱动还是 LED 的驱动呢？暂且结合起来看作是 IO_LED 驱动，那么后来我们又用到按键，又有了 IO_KEY 驱动，如果现在要使用蜂鸣器了，就会出现 IO_BUZZER 驱动，每多一个设备，就会多一个 IO_XXX 驱动。

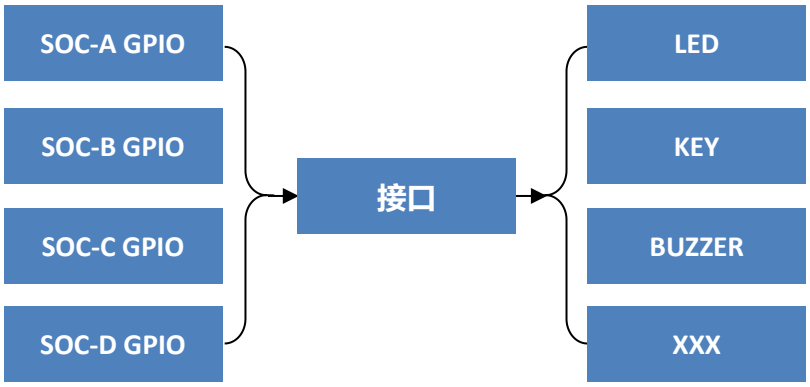


但是这些 IO_XXX 驱动中，IO 的部分几乎是一样的，为了保证代码的简介，把 IO 的部分单独拎出来，写成一个 IO 驱动，然后设备的驱动就只剩下设备自身的内容，只要去调用 IO 驱动中的接口即可。

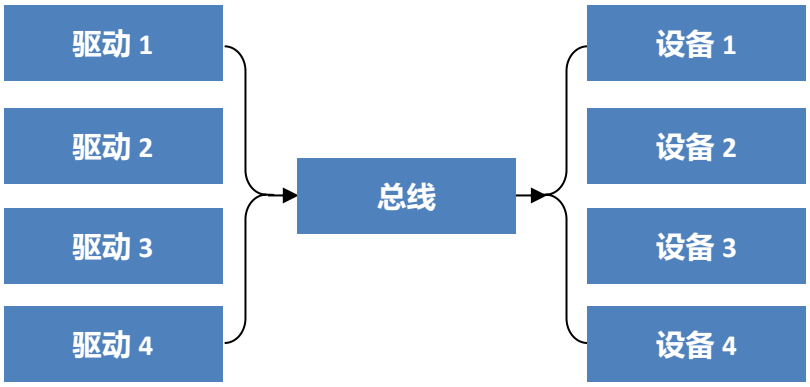


问题又来了，Linux 的一大卖点是易与移植，现在需要在其他的硬件平台上运行使用这些设备驱动又会变成什么样子呢。不同厂家 SOC 上的 GPIO 底层的实现都有所差别，那每出现一个

新的 SOC，就需要重写一个 GPIO 驱动。当然这是不可避免的，但是这些 GPIO 驱动中，还是有共同的部分，即 IO 和设备之间的接口，把这些 GPIO 中的接口部分再拎出来，单独写成一个驱动。



这样就形成了驱动的分离，一边是 SOC 的硬件资源，另一边是用户设备，他们通过统一的接口来连接。驱动分离的思想带来了很多好处，Linux 长久的发展中，省去了很多冗余的代码，SOC 厂家根据统一的接口提供 SOC 片上外设的驱动，设备厂家也根据结构统一的接口提供设备的驱动，用户只需要选择好 SOC 和外设，就能很方便的关联到一起。这样的思想也延续到了单个 SOC 中，片上驱动(之后称为驱动)和设备驱动(之后称为设备)通过总线协议来关联。



当我们往内核中添加驱动时，总线就会查找对应的设备，添加设备时就去查找对应的驱动。这就是 Linux 中的 bus、driver、device 模型，platform 就是这种模型的实现。

7.2 platform 模型

SPI 等总线设备很容易去对应 bus、driver、device 模型，但是 SOC 上并不是所有资源都有总线的概念，比如 GPIO。为了对应 bus、driver、device 模型，platform 模型定义了 platform_bus 虚拟总线、platform_driver 和 platform_device 分别来对应模型总线、驱动和设备。有一点要说明的是，platform 平台设备模型虽然名字里有”平台设备”，但他并不是独立于 Linux 三种设备之外的新的设备类型，他强调的模型二字，只是一种框架，在使用中，是无法和字符设备、块设备、网络设备脱离关系的。

7.2.1 platform_bus

内核中使用 bus_type 结构体表示总线，定义在文件 include/linux/device.h 中：

```
1.  struct bus_type {
2.    ……
3.      int (*match)(struct device *dev, struct device_driver *drv);
4.    ……
5.  };
```

成员中的 match 函数就是驱动和设备匹配的关键，他的两个输入参数一个是 drv 一个是 dev 也就是驱动和设备，每个总线类型都必须实现这个函数。

platform 虚拟总线也是 bus_type 类型的变量，在文件 drivers/base/platform.c 中定义如下：

```
1.  struct bus_type platform_bus_type = {
2.      .name       = "platform",
3.      .dev_groups = platform_dev_groups,
4.      .match      = platform_match,
5.      .uevent     = platform_uevent,
6.      .pm         = &platform_dev_pm_ops,
7.  };
```

看一下其中 match 函数的实现 platform_match 函数，也在这个文件中：

```
1.  static int platform_match(struct device *dev, struct device_driver *drv)
2.  {
3.      struct platform_device *pdev = to_platform_device(dev);
4.      struct platform_driver *pdrv = to_platform_driver(drv);
5.
6.      /* When driver_override is set, only bind to the matching driver */
7.      if (pdev->driver_override)
8.          return !strcmp(pdev->driver_override, drv->name);
9.
10.     /* Attempt an OF style match first */
11.     if (of_driver_match_device(dev, drv))
12.         return 1;
13.
14.     /* Then try ACPI style match */
15.     if (acpi_driver_match_device(dev, drv))
16.         return 1;
17.
18.     /* Then try to match against the id table */
19.     if (pdrv->id_table)
20.         return platform_match_id(pdrv->id_table, pdev) != NULL;
21.
22.     /* fall-back to driver name match */
23.     return (strcmp(pdev->name, drv->name) == 0);
24. }
```

platform_match 函数提供了 4 种匹配方式。

11~12 行，设备树下会使用的 OF 匹配表匹配方式，match 的输入参数之一 drv 的数据类型中有一个成员变量 of_match_table，of_match_table 又有一个成员为 compatible，如果这个 compatible 能和设备树中的 compatible 属性相匹配，驱动代码中的 probe 就会被调用。

如果没有使用设备树，一般会使用 23 行的第四种匹配方式，直接比较驱动和设备中 name 成员。

platform 总线是不需要驱动层面去管理的，也就是说对于驱动开发，了解其中原理即可。

7.2.2 platform_driver

1) 定义并初始化 platform_driver

platform 驱动用 platform_driver 结构体来表示，在头文件 include/linux/platform_device.h 中：

```
1.  struct platform_driver {
2.      int (*probe)(struct platform_device *);
```

```
3.      int (*remove)(struct platform_device *);
4.      void (*shutdown)(struct platform_device *);
5.      int (*suspend)(struct platform_device *, pm_message_t state);
6.      int (*resume)(struct platform_device *);
7.      struct device_driver driver;
8.      const struct platform_device_id *id_table;
9.      bool prevent_deferred_probe;
10. };
```

a) 成员 probe

probe 函数前面提到过，当设备和驱动匹配成功时，就会执行这个函数。在这个函数中调用原先在驱动入口函数中调用的内容，如字符设备中调用的 cdev_init 函数。

b) 成员 remove

remove 函数在驱动或对应设备注销时会执行，与 probe 相对的，这个函数中调用原先在驱动出口函数中调用的内容，如字符设备中的 cdel_del 函数以及其他初始化内容。

c) 成员 driver

driver 是 struct device_driver 类型，device_driver 是基本的设备驱动类型，platform_driver 是在 device_driver 基础上扩展的，所以需要包含这个类型的元素，以使用他的成员。

device_driver 结构体中有个成员变量

```
const struct of_device_id*of_match_table;
```

of_match_table 也就是上面提到的 OF 匹配表匹配方式用到的成员，of_device_id 结构体中有名为 compatible 的成员，设备树中的 compatible 就是和这个成员来比较的。

of_device_id 结构体中还有名为 name 的成员，需要与 platform 设备中的 name 字段相同。

d) 成员 id_table

id_table 是用于匹配设备的，是上面介绍的 platform_match 函数提供的第三种匹配方式需要用到的成员。

2) platform_driver 注册和注销

定义好 struct platform_driver 后，需要在驱动入口函数中调用下面的函数来注册 platform 驱动，取代原先的初始化内容：

```
int platform_driver_register (struct platform_driver*driver);
```

注册成功返回 0，失败返回负数。

在出口函数中做相应的注销操作取代原先的注销内容：

```
void platform_driver_unregister(struct platform_driver *drv);
```

7.2.3 platform_device

platform 设备用结构体 platform_device 来表示。在支持设备树的内核中，可以使用设备树代替 platform_device，但是 platform_device 仍然保留使用，我们先完整的了解一下 platform 的完整流程，之后再去结合设备树。

1) platform_device 结构体

platform_device 结构体定义在 include/linux/platform_device.h 中，内容如下：

```
1.  struct platform_device {
2.      const char  *name;
3.      int     id;
4.      bool        id_auto;
5.      struct device   dev;
6.      u32     num_resources;
7.      struct resource *resource;
8.
9.      const struct platform_device_id *id_entry;
```

```
10.      char *driver_override; /* Driver name to force a match */
11.
12.      /* MFD cell pointer */
13.      struct mfd_cell *mfd_cell;
14.
15.      /* arch specific additions */
16.      struct pdev_archdata    archdata;
17.  };
```

成员 name 用于和驱动匹配，需要和 platform_driver 中的 name 相同。

成员 id 表示当前设备在这类设备中的编号，只有一个这种类型的设备是 id 赋值-1。

成员 num_resources 表示资源数量。

成员 resource 是资源的数组，struct resource 定义如下：

```
1.   struct resource {
2.       resource_size_t start;
3.       resource_size_t end;
4.       const char *name;
5.       unsigned long flags;
6.       struct resource *parent, *sibling, *child;
7.   };
```

start 表示资源的起始地址，end 表示资源的结束地址。name 是资源名称，flags 表示资源类型，资源类型的宏定义在头文件 include/linux/ioport.h 的 29~105 行。

2) platform_device 注册和注销

在声明初始化 platform_device 结构体后，使用下面的方法注册 platform_device：

```
int platform_device_register(struct platform_device *pdev);
```

注销设备时，使用下面的方法注销：

```
void platform_device_unregister(struct platform_device *pdev);
```

3) 获取资源的方法

当 platform 设备设置好资源后，platform 驱动就可以通过下面的函数来获取资源信息：

```
struct resource *platform_get_resource(struct platform_device *dev,unsigned int
type, unsigned int num)
```

参数说明：

dev：目标 platform 设备。

type：也就是上面说的 platform_device 结构体成员 resource 结构体的 flags 成员。

num：指定 type 的资源的下标。

返回值：资源的信息，成功时返回 resource 结构体类型指针，失败时返回 NULL。

7.3 实验

这一章我们用””ps_hello”一章做的工程启动开发板，用 platform 架构来实现简单的点亮 led 实验。

7.3.1 驱动程序

驱动程序分为驱动和设备两个部分。两个程序通过设备结构体中的.name 字段匹配。匹配成功后在驱动程序中的 probe 函数中就可以获取设备程序中的资源了。

1) 先完成驱动的代码

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
```

```
#include <linux/init.h>
#include <linux/ide.h>
#include <linux/types.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/platform_device.h>

#define CHAR_DEV_MAJOR 200
#define CHAR_DEV_NAME  "led_dev"

struct alinx_char_dev{
     dev_t            devid;
     struct cdev       cdev;
     struct class      *class;
     struct device     *device;
     struct device_node *nd;
};

static struct alinx_char_dev alinx_char = {
     .cdev = {
          .owner = THIS_MODULE,
     },
};

static u32 *GPIO_DIRM1;
static u32 *GPIO_OEN1;
static u32 *GPIO_DATA1;

static int char_dev_open(struct inode *inode_p, struct file *file_p)
{
     printk(KERN_ERR"char dev open\n");
     return 0;
}

static int char_dev_release(struct inode *inode_p, struct file *file_p)
{
     printk("char dev release\n");
     return 0;
}

static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t *loff_t_p)
{
     int ret = 0;
     char lc_buf = 0;

     ret = copy_from_user(&lc_buf, buf, len);
     if(ret != 0)
     {
          printk("copy_from_user failed\n");
          return -1;
     }

     if(len != 1)
     {
          printk("len err\n");
          return -2;
     }

     if(1 == lc_buf)
     {
          *GPIO_DATA1 |= 0x00000020;
     }
     else if(0 == lc_buf)
     {
          *GPIO_DATA1 &= 0xFFFFFFDF;
     }
     else
     {
          printk("data err\n");
     }

     return 0;
}

static struct file_operations char_dev_opt = {
     .owner = THIS_MODULE,
     .open = char_dev_open,
     .write = char_dev_write,
```

```
    .release = char_dev_release,
};

static int gpio_leds_probe(struct platform_device *dev)
{
    int ret = 0;
    struct resource *led_rec[10];
    int rec_size[10];

    int i = 0;
    for(i = 0; i < dev->num_resources; i ++)
    {
        led_rec[i]  = platform_get_resource(dev, IORESOURCE_MEM, i);
        rec_size[i] = resource_size(led_rec[i]);
    }

    GPIO_DIRM1  = ioremap_wc(led_rec[0]->start, rec_size[0]);
    GPIO_OEN1   = ioremap_wc(led_rec[1]->start, rec_size[1]);
    GPIO_DATA1  = ioremap_wc(led_rec[2]->start, rec_size[2]);

    *GPIO_DIRM1 |= 0x00000020;
    *GPIO_OEN1  |= 0x00000020;

    ret = alloc_chrdev_region(&alinx_char.devid, 0, 1, CHAR_DEV_NAME);
    if(ret < 0)
    {
        goto err;
    }

    cdev_init(&alinx_char.cdev, &char_dev_opt);

    ret = cdev_add(&alinx_char.cdev, alinx_char.devid, 1);
    if(ret < 0)
    {
        goto err;
    }

    alinx_char.class = class_create(THIS_MODULE, "alinx_char_dev");
    if(IS_ERR(alinx_char.class))
    {
        ret = PTR_ERR(alinx_char.class);
        goto err;
    }

    alinx_char.device = device_create(alinx_char.class, NULL, alinx_char.devid, NULL, CHAR_DEV_NAME);
    if(IS_ERR(alinx_char.device))
    {
        ret = PTR_ERR(alinx_char.device);
        goto err;
    }

err:
    return ret;
}

static int gpio_leds_remove(struct platform_device *dev)
{
    iounmap(GPIO_DIRM1);
    iounmap(GPIO_OEN1);
    iounmap(GPIO_DATA1);

    cdev_del(&alinx_char.cdev);

    device_destroy(alinx_char.class, alinx_char.devid);

    class_destroy(alinx_char.class);

    unregister_chrdev_region(alinx_char.devid, 1);

    return 0;
}

static struct platform_driver led_driver = {
    .driver = {
        .name = "alinx-led",
    },
    .probe  = gpio_leds_probe,
    .remove = gpio_leds_remove,
};
```

```
static int __init char_drv_init(void)
{
     return platform_driver_register(&led_driver);
}

static void __exit char_drv_exit(void)
{
     platform_driver_unregister(&led_driver);
}

module_init(char_drv_init);
module_exit(char_drv_exit);

MODULE_AUTHOR("Alinx");
MODULE_ALIAS("gpio_led");
MODULE_DESCRIPTION("GPIO LED driver");
MODULE_VERSION("v1.0");
MODULE_LICENSE("GPL");
```

2) 再完成设备的部分

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/ide.h>
#include <linux/types.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/platform_device.h>

#define GPIO_DIRM_1      0xFF0A0244
#define GPIO_OEN_1       0xFF0A0248
#define GPIO_DATA_1      0xFF0A0044

static void led_release(struct device *dev)
{
     printk("led device released\r\n");
}

static struct resource led_resource[3] ={
     {
          .start = GPIO_DIRM_1,
          .end = GPIO_DIRM_1 + 3,
          .flags = IORESOURCE_MEM, // include/linux/ioport.h
     },
     {
          .start = GPIO_OEN_1,
          .end = GPIO_OEN_1 + 3,
          .flags = IORESOURCE_MEM,
     },
     {
          .start = GPIO_DATA_1,
          .end = GPIO_DATA_1 + 3,
          .flags = IORESOURCE_MEM,
     },
};

static struct platform_device led_dev = {
     .name = "alinx-led",
     .id   = 1,
     .dev = {
          .release = led_release,
     },
     .num_resources = 3,
     .resource = led_resource,
};

static int __init char_drv_init(void)
{
     return platform_device_register(&led_dev);
}

static void __exit char_drv_exit(void)
{
     platform_device_unregister(&led_dev);
}
```

```
module_init(char_drv_init);
module_exit(char_drv_exit);

MODULE_AUTHOR("Alinx");
MODULE_ALIAS("gpio_led");
MODULE_DESCRIPTION("GPIO LED driver");
MODULE_VERSION("v1.0");
MODULE_LICENSE("GPL");
```

7.3.2 测试程序

可以使用第一章的测试程序。

7.3.3 运行测试

加载驱动模块和设备模块，运行测试程序，板子上的 ps_led1 会被点亮。

```
root@petalinux:/mnt# insmod ps-led-plt-drv.ko
root@petalinux:/mnt# insmod ps-led-plt-dev.ko
root@petalinux:/mnt# ./char_drv_test /dev/led_dev on
[  158.888174] char dev open
ps_led1 on
[  158.890846] char dev release
root@petalinux:/mnt# ./char_drv_test /dev/led_dev off
[  164.552552] char dev open
ps_led1 off
[  164.555243] char dev release
root@petalinux:/mnt#
```

如果我们把设备删除，设备文件也就不见了，再重新加载，又存在了，说明 probe 函数就是在 dev 和 drv 相匹配的时候执行的。

```
root@petalinux:/mnt# rmmod ps_led_plt_dev.ko
[  154.906658] led device released
root@petalinux:/mnt#
```

第八章 platform 与设备树

8.1 设备树下的 platform

在设备树引入之后，platform 结构中的 platform_device 就可以用设备树去代替了。设备树下的 platform 驱动相较于原始的 platform 驱动，只需要把 platform_device 中描述的设备信息放到设备树中，同时修改 paltform_drvier 中对资源的读取方法即可。

1) 在设备树中描述设备信息

使用 platform_device 时，我们可以通过 name 字段或者 id_table 来匹配设备和驱动，当 platform_device 变成设备树时，根据上一章在 platform_bus 中的介绍，则使用 of_match_table 方法来匹配。of_match_table 对于设备树来说，要做的事，就是保证设备节点的 compatible 属性和 platform_driver 中的 compatible 保持一致。

```
1.  alinxled {
2.      compatible = "alinx-led";
3.      pinctrl-names = "default";
4.      pinctrl-0 = <&pinctrl_led_default>;
5.      alinxled-gpios = <&gpio0 0 0>;
6.  };
```

除了这点要注意之外，设备树的写法就没有什么特别的了。

2) of_match_table

paltform_drvier 中的 compatible 属性设置在上一章中已经介绍过了，compatible 位于 paltform_drvier->device_driver->of_device_id->compatible，保证和设备树中的 compatible 字段一致即可。of_device_id 结构体在 paltform_drvier 结构体中的成员名为 of_match_table，of 匹配表。初始化示例如下：

```
1.  static const struct of_device_id led_of_match[] = {
2.      /* compatible 字段和设备树中保持一致 */
3.      { .compatible = "alinx-led" },
4.      {/* Sentinel */}
5.  };
```

需要注意 of_device_id 的最后一个成员必须为空。

还有一点，即使我们不是用 name 字段做匹配工作，device_driver 结构体中的 name 字段还是需要保留，否则在加载驱动时可能会出错。

3) 修改 paltform_drvier 读取设备信息的方法

既然 platform_device 换成了设备树，那 paltform_drvier 读取设备信息的方式肯定得换成 of 函数了。回顾一下第四章中的 of 函数内容，就可以动手来实现设备树下的 platform 驱动代码了。

8.2 实验

这一章的实验在上一章的基础上，用设备树代替 platform_device 的情况下，使用 platform 架构来实现简单的点亮 led 实验。

8.2.1 设备树

这里我们使用第四章，pinctrl 子系统和 gpio 子系统下的设备树，写法都是一样的，直接用就行了。只要注意设备节点中的 compatible 属性，要和 platform_driver 中的 compatible 保持一致。

8.2.2 驱动程序

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/ide.h>
#include <linux/types.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/platform_device.h>
#include <linux/gpio/consumer.h>

#define CHAR_DEV_MAJOR 200
#define CHAR_DEV_NAME  "led_dev_%d"

struct alinx_char_dev{
     struct cdev        cdev;
     struct device      *device;
     struct gpio_desc   *gpio_d;
     struct semaphore   lock;
};

int dev_probe_times = 0;
dev_t devid_cnt = 0;
struct class *pclass;

static int char_dev_open(struct inode *inode_p, struct file *file_p)
{
     struct alinx_char_dev *alinx_char;
     int ret = 0;

     alinx_char = container_of(inode_p->i_cdev, struct alinx_char_dev, cdev);
     file_p->private_data = alinx_char;

     ret = down_interruptible(&alinx_char->lock);
     if(ret)
     {
          ret = -1;
     }
     else
     {
          printk("char dev open\n");
     }

     return ret;
}

static int char_dev_release(struct inode *inode_p, struct file *file_p)
{
     struct alinx_char_dev *alinx_char = file_p->private_data;

     up(&alinx_char->lock);
     printk("char dev release\n");
     return 0;
}

static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t
*loff_t_p)
{
     struct alinx_char_dev *alinx_char = file_p->private_data;
     int ret = 0;
     char lc_buf = 0;

     ret = copy_from_user(&lc_buf, buf, len);
     if(ret != 0)
     {
          printk("copy_from_user failed\n");
          return -1;
```

```
    }

    if(len != 1)
    {
        printk("len err\n");
        return -2;
    }

    if(1 == lc_buf)
    {
        gpiod_set_value(alinx_char->gpio_d, 1);
    }
    else if(0 == lc_buf)
    {
        gpiod_set_value(alinx_char->gpio_d, 0);
    }
    else
    {
        printk("data err\n");
    }

    return 0;
}

static struct file_operations char_dev_opt = {
    .owner = THIS_MODULE,
    .open = char_dev_open,
    .write = char_dev_write,
    .release = char_dev_release,
};

static int ax_led_probe(struct platform_device *pdev)
{
    int ret = 0;
    struct alinx_char_dev *alinx_char;

    alinx_char = devm_kzalloc(&pdev->dev, sizeof(struct alinx_char_dev), GFP_KERNEL);
    if(alinx_char == NULL)
    {
        return -1;
    }

    platform_set_drvdata(pdev, alinx_char);

    cdev_init(&alinx_char->cdev, &char_dev_opt);

    ret = cdev_add(&alinx_char->cdev, devid_cnt, 1);
    if(ret < 0)
    {
        goto err;
    }

    alinx_char->device = device_create(pclass, NULL, devid_cnt, NULL, CHAR_DEV_NAME,
dev_probe_times);
    if(IS_ERR(alinx_char->device))
    {
        ret = PTR_ERR(alinx_char->device);
        goto err;
    }
    dev_probe_times ++;
    devid_cnt ++;

    alinx_char->device->of_node = pdev->dev.of_node;
    alinx_char->gpio_d = gpiod_get_index(alinx_char->device, "led", 0, GPIOD_OUT_HIGH);
    gpiod_direction_output(alinx_char->gpio_d, 1);

    sema_init(&alinx_char->lock, 1);

err:
    return ret;
}

static int ax_led_remove(struct platform_device *pdev)
{
    struct alinx_char_dev *alinx_char = platform_get_drvdata(pdev);

    gpiod_put(alinx_char->gpio_d);

    cdev_del(&alinx_char->cdev);
```

```
    device_destroy(pclass, devid_cnt);

    devm_kfree(&pdev->dev, alinx_char);

    devid_cnt --;
    dev_probe_times --;

    return 0;
}

static const struct of_device_id led_of_dev[] = {
    { .compatible = "alinx-led", },
    {}
};

static struct platform_driver led_driver = {
    .driver =
    {
        .name = "alinx-led",
        .of_match_table = led_of_dev,
    },
    .probe = ax_led_probe,
    .remove = ax_led_remove,
};

static int __init char_drv_init(void)
{
    int ret = 0;
    ret = alloc_chrdev_region(&devid_cnt, 0, 256, CHAR_DEV_NAME);
    if(ret < 0)
    {
        return -1;
    }

    pclass = class_create(THIS_MODULE, "alinx_char_dev");
    if(IS_ERR(pclass))
    {
        return -1;
    }

    return platform_driver_register(&led_driver);
}

static void __exit char_drv_exit(void)
{
    class_destroy(pclass);

    unregister_chrdev_region(devid_cnt, 256);

    platform_driver_unregister(&led_driver);
}

module_init(char_drv_init);
module_exit(char_drv_exit);

MODULE_AUTHOR("Alinx");
MODULE_ALIAS("gpio_led");
MODULE_DESCRIPTION("GPIO LED driver");
MODULE_VERSION("v1.0");
MODULE_LICENSE("GPL");
```

和上一章相比就是把设备程序中的内容转移到了设备树中。驱动程序中新增了一个.of_match_table 字段来匹配设备。

probe 函数中获取资源的方法也换成了从设备树中获取资源的接口。

8.2.3 测试程序

可以使用第一章的测试程序。

8.2.4 运行测试

测试方法参考第一章。

```
root@petalinux:~# cd /mnt/
root@petalinux:/mnt# ls
char_drv_test         linux-nio             pl-key.ko             ps-led-plt-drv.ko
char_drv_test_conc    linux-signal          ps-led-plt-dev.ko     ps-led.ko
gpio-in               linux-timer           ps-led-plt-drv-dt.ko
root@petalinux:/mnt# insmod ps-led-plt-drv-dt.ko
root@petalinux:/mnt# ./char_drv_test /dev/led_dev_0 off
[  176.272131] char dev open
ps_led1 off
[  176.274830] char dev release
root@petalinux:/mnt# ./char_drv_test /dev/led_dev_0 on
[  183.176415] char dev open
ps_led1 on
[  183.179103] char dev release
root@petalinux:/mnt# ./char_drv_test /dev/led_dev_0 off
[  187.304327] char dev open
ps_led1 off
[  187.307016] char dev release
root@petalinux:/mnt#
```

第九章 MISC 设备驱动

做了这么多实验之后，一直没有离开过字符设备。比较这些章节的驱动代码不难发现，字符设备框架中，有很大一部分的代码是相同的。对此，Linux 内核提出了 MISC 驱动模型，把这些共性的代码封装起来。

9.1 MISC 设备简介

MISC 意为杂散设备，诸如蜂鸣器、adc、led 这些难以分类的设备，都可以用 MISC 框架来实现驱动。

MISC 驱动模型实际上是对字符设备驱动模型的一次封装，他的本质是还是字符设备。MISC 的源码目录为/drivers/char/misc.c。相比一般的字符设备驱动框架，MISC 这层封装带来的好处有以下几点：

1) 节省主设备号

在文件/drivers/char/misc.c 的 214 行，在 misc_register 函数中，使用宏 MISC_MAJOR 表示主设备号来注册次设备号，MISC_MAJOR 宏定义在文件 include/uapi/linux/major.h 的第 25 行，值为 10，也就是说 MISC 设备的主设备号固定为 10。使用 MISC 框架注册的字符设备，都会在主设备号 10 下分配次设备号(部分次设备号已被内核占用)。而普通字符设备框架注册时必须要分配一个主设备号，因此相对于一般的字符设备框架，使用 MISC 框架能节省主设备号。

2) 简洁使用方便

一般的字符设备驱动框架需要用 alloc_chrdev_region()、cdev_init()、cdev_add()、class_create()、device_create()一串函数来完成设备的创建，但是 MISC 框架使用 misc_register()函数对设备的创建过程做了很好的优化和封装，我们只需调用这一个函数就可以完成设备的创建，方便很多。

3) 驱动分层

MISC 也是对 Linux 驱动分层思想的一种体现，给难以分类的设备提供了归属类型，也方便了他们的使用。

9.2 MISC 框架的使用

MISC 框架的使用对于一般字符设备框架来说只是做一些替换。方法如下。

1) 创建并初始化 miscdevice 结构体

miscdveice 表示 misc 设备，定义在文件 include/linux/miscdevice.h 中，如下：

```
1.  struct miscdevice  {
2.      int minor;
3.      const char *name;
4.      const struct file_operations *fops;
5.      struct list_head list;
6.      struct device *parent;
7.      struct device *this_device;
8.      const struct attribute_group **groups;
9.      const char *nodename;
10.     umode_t mode;
11. };
```

结构体成员中没有主设备号，因为 misc 设备的主设备号都固定为 10。声明了 miscdevice 结构体变量之后需要初始化三个成员变量：

a) 第 2 行的 minor，子设备号。使用 misc 框架注册的设备以子设备号来区分。内核中已经占用了部分 misc 的子设备号，在文件 include/linux/miscdevice.h 的 14~54 行中可查看已被占用的子设备号(不同的内核源码行号可能不一样)。如果需要在系统中查看已被占用的子设备号，可以 ls 一下/sys/dev/char 这个路径如下图。ls 结果中的如第一项的 10:1，冒号前的 10 为主设备，冒号后的 1 为此次设备号，也就是说主设备号 10 下的次设备号 1 已被占用。

```
root@ax_peta:/sys/dev/char# ls
10:1    10:61   1:1     1:8     254:0   4:11    4:17    4:22    4:28    4:33    4:39    4:44    4:5     4:55    4:60    4:9     7:128
 90:2
10:130  10:62   1:11    1:9     254:1   4:12    4:18    4:23    4:29    4:34    4:4     4:45    4:50    4:56    4:61    5:0     7:129
 90:3
10:20   10:63   1:3     246:0   254:2   4:13    4:19    4:24    4:3     4:35    4:40    4:46    4:51    4:57    4:62    5:1     89:0
 90:4
10:237  116:33  1:4     247:0   4:0     4:14    4:2     4:25    4:30    4:36    4:41    4:47    4:52    4:58    4:63    5:2     89:1
 90:5
10:59   13:63   1:5     248:0   4:1     4:15    4:20    4:26    4:31    4:37    4:42    4:48    4:53    4:59    4:7     7:0     90:0
 90:6
10:60   189:0   1:7     253:0   4:10    4:16    4:21    4:27    4:32    4:38    4:43    4:49    4:54    4:6     4:8     7:1     90:1
 90:7
```

b) 第 3 行的 name，设备名称。将在/dev 目录中生成的节点名。

c) 第 4 行的 fops，设备操作函数集。和字符设备中实现的 fops 相同。

2) 使用 misc_register 函数注册设备代替原先的注册过程

声明并初始化完成 miscdevice 后，使用 misc_register 函数来注册 misc 设备，函数定义在文件/drivers/char/misc.c 中，原型如下：

```
int misc_register(struct miscdevice * misc)
```

misc 即为 miscdevice 结构体变量。

注册成功返回 0，失败返回负数。

3) 使用 misc_deregister 函数注销设备代替原先的注销过程

与注册相对的就有注销函数，代替一般字符设备在驱动出口函数中一系列的注销操作，函数原型：

```
void misc_deregister(struct miscdevice *misc)
```

misc 即为 miscdevice 结构体变量。

9.3 实验

本章的实验在上一章的基础上修改，在 platform 框架下结合 misc 框架和把一般的字符设备框架修改成 misc 框架的方法几乎没有差别。

9.3.1 驱动代码

```
1.  #include <linux/module.h>
2.  #include <linux/kernel.h>
3.  #include <linux/fs.h>
4.  #include <linux/init.h>
5.  #include <linux/ide.h>
6.  #include <linux/types.h>
7.  #include <linux/cdev.h>
8.  #include <linux/device.h>
9.  #include <linux/of.h>
10. #include <linux/gpio/consumer.h>
11. #include <linux/miscdevice.h>
12.
13. #define CHAR_DEV_MAJOR 200
14. #define CHAR_DEV_NAME  "led_dev"
```

```
15.
16. struct alinx_char_dev{
17.     struct gpio_desc   *gpio_d;
18.     struct semaphore   lock;
19. };
20.
21. static struct alinx_char_dev alinx_char;
22.
23. static int char_dev_open(struct inode *inode_p, struct file *file_p)
24. {
25.     int ret = 0;
26.     ret = down_interruptible(&alinx_char.lock);
27.     if(ret)
28.     {
29.         printk("%s wait resource break\n", CHAR_DEV_NAME);
30.         ret = -1;
31.     }
32.     else
33.     {
34.         printk("char dev open\n");
35.     }
36.
37.     return ret;
38. }
39.
40. static int char_dev_release(struct inode *inode_p, struct file *file_p)
41. {
42.     up(&alinx_char.lock);
43.     printk("char dev release\n");
44.     return 0;
45. }
46.
47. static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t *l
    off_t_p)
48. {
49.     int ret = 0;
50.     char lc_buf = 0;
51.
52.     ret = copy_from_user(&lc_buf, buf, len);
53.     if(ret != 0)
54.     {
55.         printk("copy_from_user failed\n");
56.         return -1;
57.     }
58.
59.     if(len != 1)
60.     {
61.         printk("len err\n");
62.         return -2;
63.     }
64.
65.     if(1 == lc_buf)
66.     {
67.         gpiod_set_value(alinx_char.gpio_d, 1);
68.     }
69.     else if(0 == lc_buf)
70.     {
71.         gpiod_set_value(alinx_char.gpio_d, 0);
72.     }
73.     else
74.     {
75.         printk("data err\n");
76.     }
77.
78.     return 0;
79. }
80.
81. static struct file_operations char_dev_opt = {
82.     .owner = THIS_MODULE,
83.     .open = char_dev_open,
84.     .write = char_dev_write,
85.     .release = char_dev_release,
86. };
87.
88. static struct miscdevice led_misc = {
89.     .name = CHAR_DEV_NAME,
90.     .minor = MISC_DYNAMIC_MINOR, //ls /sys/dev/char
91.     .fops = &char_dev_opt,
92. };
```

```
93.
94. static int __init char_drv_init(void)
95. {
96.     int ret = 0;
97.
98.     ret = misc_register(&led_misc);
99.     if(ret < 0) {
100.        printk("misc device register failed\r\n");
101.        ret = -EFAULT;
102.        goto err;
103.    }
104.
105.    led_misc.this_device->of_node = of_find_node_by_path("/alinxled");
106.    alinx_char.gpio_d = gpiod_get_index(led_misc.this_device, "led", 0, GPIOD_OUT_HIGH);
107.    gpiod_direction_output(alinx_char.gpio_d, 1);
108.
109.    sema_init(&alinx_char.lock, 1);
110.
111. err:
112.    return ret;
113. }
114.
115. static void __exit char_drv_exit(void)
116. {
117.    gpiod_put(alinx_char.gpio_d);
118.
119.    misc_deregister(&led_misc);
120. }
121.
122.
123.
124. module_init(char_drv_init);
125. module_exit(char_drv_exit);
126.
127. MODULE_AUTHOR("Alinx");
128. MODULE_ALIAS("gpio_led");
129. MODULE_DESCRIPTION("GPIO LED driver");
130. MODULE_VERSION("v1.0");
131. MODULE_LICENSE("GPL");
```

misc 驱动比较简单，在驱动入口函数中把原先字符设备中的系列操作替换成 misc_register 接口就行了。

9.3.2 测试代码

可以使用第一章的测试程序。

9.3.3 运行测试

测试方法和上一章一致。

```
root@petalinux:~# cd /mnt/
root@petalinux:/mnt# insmod ps-led.ko
root@petalinux:/mnt#
root@petalinux:/mnt# ./char_drv_test /dev/led_dev off
[  118.304427] char dev open
ps_led1 off
[  118.307116] char dev release
root@petalinux:/mnt# ./char_drv_test /dev/led_dev on
[  122.924602] char dev open
ps_led1 on
[  122.927291] char dev release
root@petalinux:/mnt#
```

第十章 pwm 驱动

pwm 在嵌入式设备中使用很多，常用于控制电机、控制振动器件、调节背光、呼吸灯等等。这一章就来学习 zynqmp 平台上 Linux 系统中的 pwm 驱动的实现方法。

本章的 petalinux 工程需要用 course2“vitis 应用教程”中的“15_custom_pwm_ip”vivado 工程来制作。

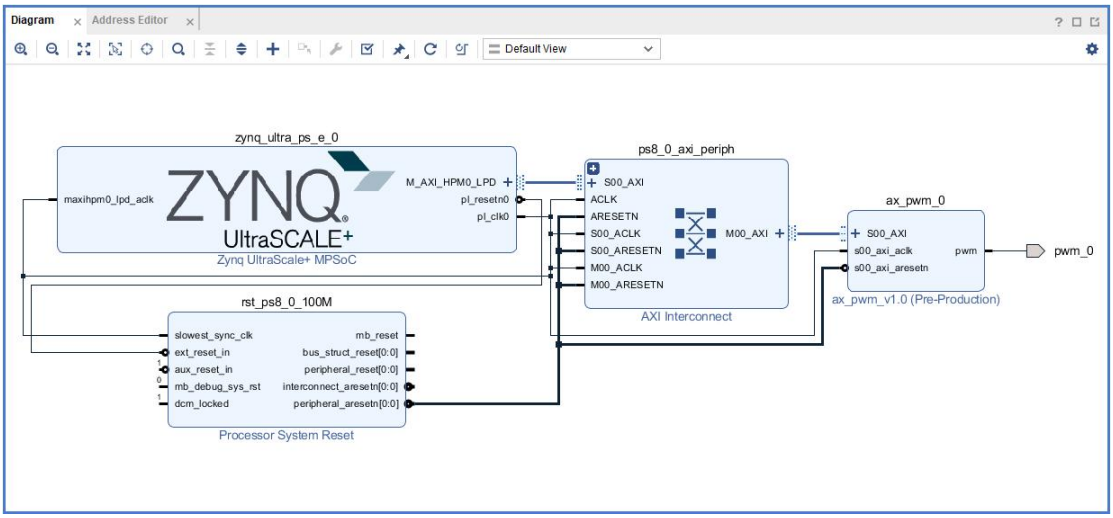
10.1 zynqmp 上的 pwm 实现

10.1.1 修改 vivado 工程

zynqmp 平台上输出 pwm 需要借助 pl 端(fpga)的资源。前面使用的硬件描述文件中，并没有设置 pwm 输出，所以需要在这个 vivado 工程基础上修改。vivado 工程参考源码包中的 01_ps_base.zip。

修改的方法参考前面学习的 vivado 自定义 IP 的方法。这里面以 pwm 为例介绍了自定义 IP 的制作方法。这里我们需要做的是就是在十八章的 vivado 工程基础上，添加十一章中制作的 pwm 自定义 ip。添加自定义 ip 的方法在第十一章中也有介绍。

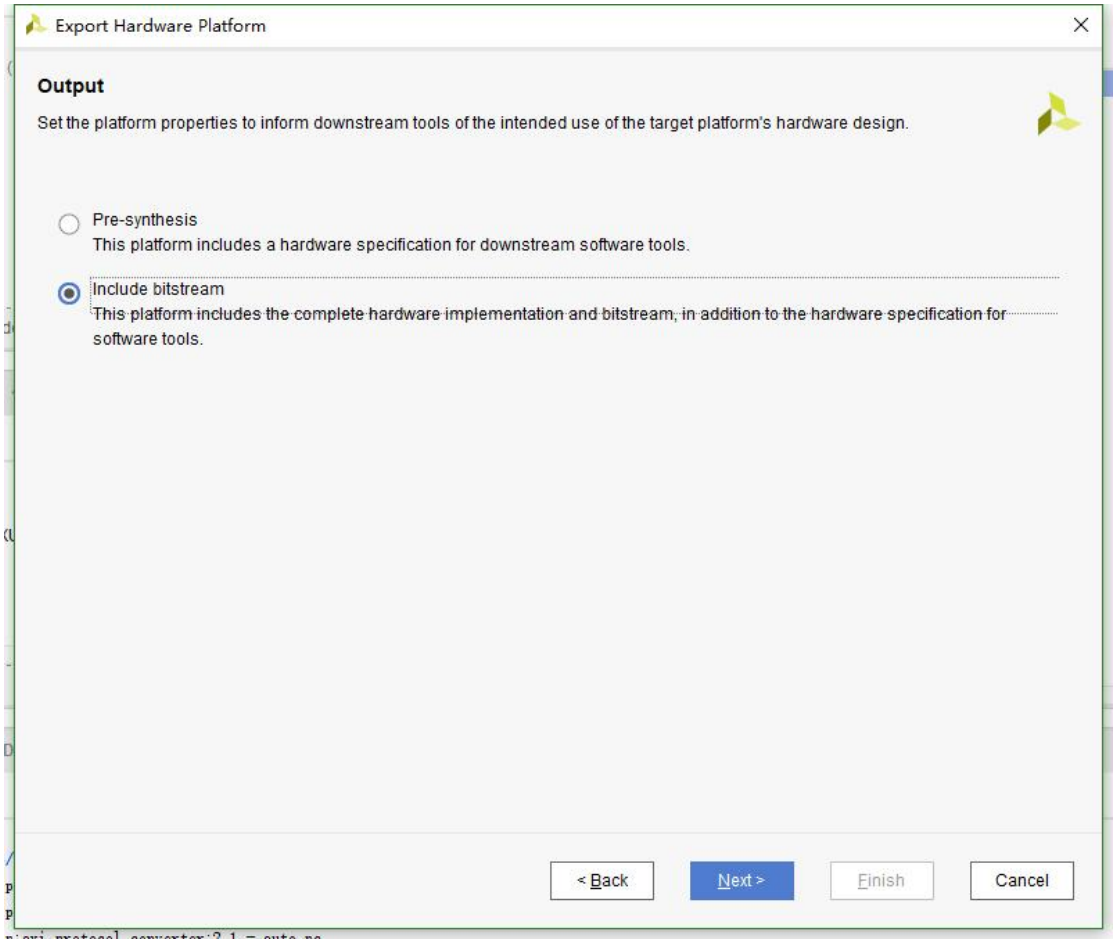
整个过程这里就不详细说了，添加 pwm 的 ip 之后结果如下：



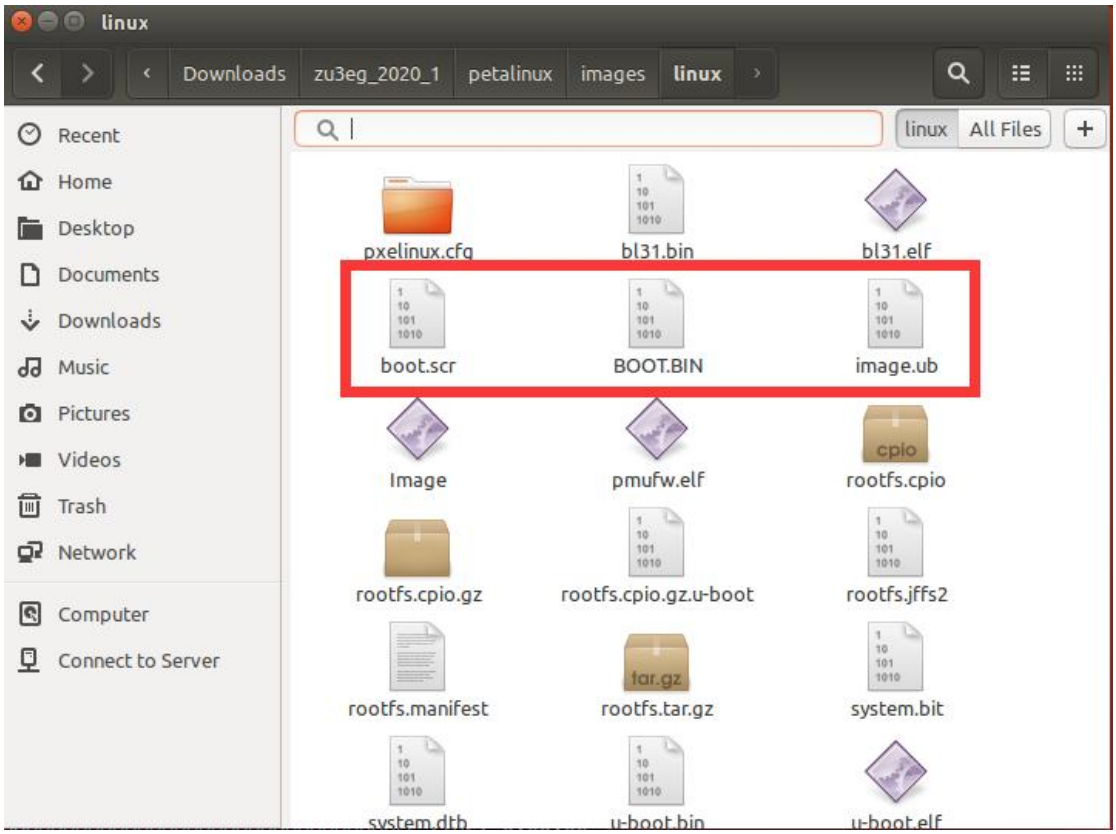
ip 添加完成后，需要修改 xdc 更改引脚约束，这里我们准备使用 pwm 来控制开发板上 PL_LED 这个 led。新建 pwm.xdc 文件，添加下面的内容约束引脚。

```
set_property IOSTANDARD LVCMOS33 [get_ports pwm_0]
set_property PACKAGE_PIN E10 [get_ports pwm_0]
```

修改完成后编译生成 bitstream，并导出硬件描述文件。注意导出是，要把 bitstream 包含进去。

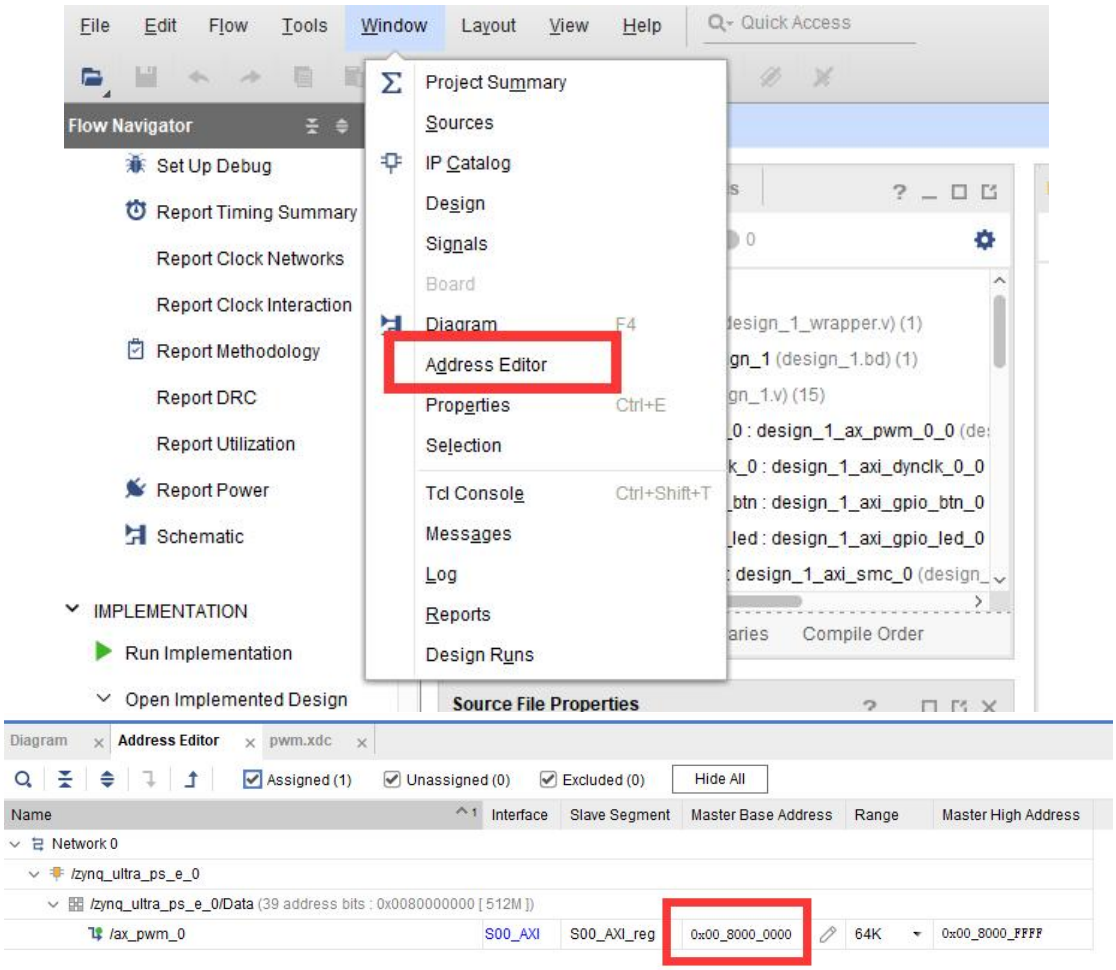


然后使用新获得的.xsa 文件重新配置内核，编译后重新打包 BOOT.BIIN，替换 sd 卡中原先的 boot.scr、BOOT.BIN、image.nb 三个文件。



10.1.2 控制 pwm 输出的方法

控制 pwm 输出有两个关键参数，一是频率，二是占空比。在上面我们配置好硬件信后，该如何去操作 pwm 的这两个参数呢。自定义 IP 可以理解为配置 fpga 的寄存器资源变成我们想要的设备，添加 IP 后，我们就可以像操作 ARM 资源一样，通过操作紧寄存器来操 fpga 的资源。那首先我们就要找到 pwm 设备的寄存器首地址。在菜单栏 Window 选项中打开 Address Editor 窗口。如图下：



就能找到 ip 对应的寄存器范围，这个寄存器首地址是 vivado 工程自动分配的，所以每个人的可能会不一样，当然也可以手动修改。

光是找到了寄存器首地址，还是不知道寄存器的具体设置方法，这里我们参考一下下面这段代码，如下图：

```
unsigned int duty;
int main()
{
    init_platform();
    print("Hello World\n\r");
    //pwm out period = frequency(pwm_out) * (2 ** N) / frequency(clk);
    AX_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR, AX_PWM_S00_AXI_SLV_REG0_OFFSET, 17179);//200hz
    while (1) {
        for (duty = 0x8fffffff; duty < 0xffffffff; duty = duty + 100000) {
            AX_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR, AX_PWM_S00_AXI_SLV_REG1_OFFSET, duty);
            usleep(100);
        }
    }
    cleanup_platform();
    return 0;
}
```

AX_PWM_mWriteReg 函数用于设置寄存器的值，第一个参数是首地址，第二个参数是偏移量，第三个参数是目标值。

第一次调用 AX_PWM_mWriteReg 函数是设置频率。XPAR_AX_PWM_0_S00_AXI_BASEADDR 的值就是我们上面在 Address Editor 界面中查到的首地址，看一下偏移量 AX_PWM_S00_AXI_SLV_REG0_OFFSET 的值为 0。也就是说设置频率的寄存器就是首地址了。

第二次调用是设置占空比，首地址没变，偏移量 AX_PWM_S00_AXI_SLV_REG1_OFFSET 的值为 4，也就是首地址加 4。

10.2 Linux 中的 pwm 框架

在 Linux 内核提供了一套完整的 pwm 框架。但是在 zynqmp 上使用这套框架需要注意，zynqmp 上的 pwm 是借助 fpga 资源来实现的，自定义 ip 的寄存器设置可能会因人而异，因此要使用这套框架，需要保证自定义 ip 的一致。再者如果只是简单的 pwm 应用，如本章实验的 led 呼吸灯，使用这个框架就显得有些大材小用了。虽然在本章的实验中不会这个框架，但我们还是简单的介绍一下，了解一下其中的结构与设计思路。

pwm 框架还是融入了驱动的分层思想，把 pwm 驱动分成了 pwm_device(设备)和 pwm_chip(芯片)两个部分，pwm_chip 意为 pwm 的芯片，它可以输出单路或者多路 pwm 波形，而 pwm_device 即为使用 pwm_chip 其中一路或多路的设备，以本章例程 led 呼吸灯为例，led 就是 pwm_device，我们自定义的 pwmip 就是 pwm_chip。pwm 的框架一般会和 platform 框架结合使用。接下来我们分别介绍他们的实现方法。

10.2.1 pwm_device

一个使用 pwm 的设备，需要关注四个参数：使能、极性、频率、占空比，对此 pwm 框架提供了以下方法来与之对应：

1) int pwm_enable(struct pwm_device *pwm);和 void pwm_disable(struct pwm_device *pwm);
   pwm_enable 函数用使能 pwm 输出，pwm_disable 用于失能 pwm 输出。
2) int pwm_set_polarity(struct pwm_device *pwm, enum pwm_polarity polarity);
   pwm_set_polarity 用于设置极性，可选参数有 PWM_POLARITY_NORMAL(普通)和 PWM_POLARITY_INVERSED(反转)两种。
3) int pwm_config(struct pwm_device *pwm, int duty_ns, int period_ns);
   pwm_config 函数用于设置占空比和频率，参数 duty_ns 为占空比，period_ns 为频率。

这些接口函数定义在文件 include/linux/pwm.h 中，他们都需要一个 struct pwm_device 类型的指针作为操作句柄。

pwm_device 结构体则是指代使用 pwm 的设备，我们要使用这些接口，需要先获取一个 pwm_device 结构体。

获取 pwm_device 结构体的方法也定义在文件 include/linux/pwm.h 中。如下：

1) struct pwm_device *pwm_get(struct device *dev, const char *con_id);和
   struct pwm_device *of_pwm_get(struct device_node *np, const char *con_id);
   这两个方法将从指定设备(参数 dev)的设备树节点或者直接从设备树节点(参数 np)中去获取 pwm_device。对设备树的格式要求如下：

```
bl: backlight {
pwms = <&pwm 0 5000000 PWM_POLARITY_INVERTED>;
pwm-names = "backlight";
}
```

bl: backlight 即为设备节点名称。节点中的两个属性都是必须的

pwms 指代这个设备用到的 pwm 列表，第一个参数&pwm 实际是引用了 pwm_chip 节点，具体在讲 pwm_chip 的时候再说。第二个参数指代 platform_device 的设备号，具体根据实际情况来定，不一定会用到。第三个参数 500000 是 pwm 的默认周期，单位是纳秒。第四个参数是可选字段，表示极性。

pwm-names 是与 pwms 相对应的 pwm 设备名称。"backlight"就是与&pwm 0 5000000 PWM_POLARITY_INVERTED 相对应。

函数参数 con_id 用于指定相匹配的 pwm 设备，如果 NULL，就会返回 pwms 中的第一个设备，如果指定名称如"backlight"，就会返回对应的设备。

2) void pwm_put(struct pwm_device *pwm);

与获取 get 函数相对的 put，用于释放 pwm_device。

pwm_device 需要关注的事情就这些，pwm 设备调用了接口函数后，最终就会调用 pwm_chip 中的操作函数，接下来我们再看 pwm_chip。

10.2.1 pwm_chip

pwm_chip 关在作用在于，它提供了 pwm_device 接口函数对应的操作函数集，pwm_chip 结构体定义在 include/linux/pwm.h 中，如下：

```
struct pwm_chip {
        struct device                *dev;
        struct list_head             list;
        const struct pwm_ops         *ops;
        int                          base;
        unsigned int                 npwm;
        struct pwm_device            *pwms;
        struct pwm_device *          (*of_xlate)(struct pwm_chip *pc, const struct
of_phandle_args *args);
        unsigned int                 of_pwm_n_cells;
        bool                         can_sleep;
};
```

dev 为 pwm chip 对应的设备，一般由 pwm driver 对应的 platform 驱动指定且必须提供。

ops 为 pwm_device 中接口函数对应的操作函数集，必须提供。

npwm 为 pwm chip 可以支持的 pwm_device 个数，必须提供。

pwms 为该 pwm chip 中 pwm device 的数组，kernel 会自行分配，不需要手动设置。

重点介绍一下 struct pwm_ops 结构体指正 ops。struct pwm_ops 定义如下：

```
struct pwm_ops {
        int (*request)(struct pwm_chip *chip, struct pwm_device *pwm);
        void (*free)(struct pwm_chip *chip, struct pwm_device *pwm);
        int (*config)(struct pwm_chip *chip, struct pwm_device *pwm, int duty_ns, int
period_ns);
        int (*set_polarity)(struct pwm_chip *chip, struct pwm_device *pwm, enum
pwm_polarity polarity);
        int (*enable)(struct pwm_chip *chip, struct pwm_device *pwm);
        void (*disable)(struct pwm_chip *chip, struct pwm_device *pwm);
```

```
#ifdef CONFIG_DEBUG_FS
        void (*dbg_show)(struct pwm_chip *chip, struct seq_file *s);
#endif
        struct module *owner;
};
```

request 和 free 函数已经弃用了。

函数与 pwm_device 接口函数中的 pwm_config 函数相对应，配置 pwm_device 的频率、占空比。必须提供。

enable、disable 函数与 pwm_device 接口函数中的 pwm_enable、pwm_disable 函数相对应，用于使能/禁止 pwm 信号输出。必须提供。

set_polarity 函数与 pwm_device 接口函数中的 pwm_set_polarity 函数对应，设置 pwm 信号的极性。可选。

定义一个 pwm_chip 之后，需要实现 ops 中至少 config、enable、disable 三个函数。初始化完成后，使用下面的函数向内核注册 pwm_chip：

```
int pwmchip_add(struct pwm_chip *chip);
```

相对的注销使用函数：

```
int pwmchip_remove(struct pwm_chip *chip);
```

在实现 config 等函数时，需要通过设备树获取设备信息，pwm_chip 在设备树中的节点格式并不固定，添加相关信息即可。如下:

```
pwm: pwm@43C20000 {
compatible = "test-pwm";
reg = <0x43C20000 0x100>;
};
```

上面再 pwm_device 的节点中有对 pwm_chip 的引用"pwms = <&pwm 0 5000000 PWM_POLARITY_INVERTED>;"。这里的&pwm，实际上就是引用如"pwm: pwm@43C20000"这样的 pwm_chip0 节点。引用的目的也就是把 device 和 chip 相关联。

专用的 pwm 芯片厂家会提供 pwm_chip 的驱动，但是 xlinx 斌没有提供，也许是考虑到自定义 ip 的多样性，因此在 zynqmp 平台上使用 pwm 框架，还需要驱动开发人员去实现 pwm_chip，比较麻烦。

10.2.1 示例

设备树：

```
1.  pwm: pwm@43C20000 {
2.          compatible = "alinx-pwm";
3.          reg = <0x43C20000 1>;
4.          #pwm-cells = <2>;
5.  };
6.
7.  pwm-led {
8.      compatible = "pwm-led";
9.      pwms = <&pwm 0 5000000>;
10. };
```

pwm_chip：

结合 platform 框架，从设备树中获取信息，注册 pwm_chip。

```
1.  struct ax_pwm_chip {
2.      struct pwm_chip chip;
3.  };
```

```
4.
5.  struct ax_pwm_chip ax_pwm;
6.
7.  static int ax_pwm_config(struct pwm_chip *chip, struct pwm_device *pwm, int duty_ns, int period_ns)
8.  {
9.      return 0;
10. }
11.
12. static int ax_pwm_enable(struct pwm_chip *chip, struct pwm_device *pwm)
13. {
14.     return 0;
15. }
16.
17. static void ax_pwm_disable(struct pwm_chip *chip, struct pwm_device *pwm)
18. {
19.
20. }
21.
22. static const struct pwm_ops ax_pwm_ops = {
23.     .owner = THIS_MODULE,
24.     .config = ax_pwm_config,
25.     .enable = ax_pwm_enable,
26.     .disable = ax_pwm_disable,
27. };
28.
29. static int ax_pwm_probe(struct platform_device *pdev)
30. {
31.     int err;
32.
33.     ax_pwm.chip.dev = &pdev->dev;
34.     ax_pwm.chip.ops = &ax_pwm_ops;
35.     ax_pwm.chip.npwm = 1;
36.
37.     err = pwmchip_add(&ax_pwm.chip);
38.     if (err < 0)
39.     {
40.         return err;
41.     }
42.
43.     return 0;
44. }
45.
46. static int ax_pwm_remove(struct platform_device *pdev)
47. {
48.     int err;
49.
50.     err = pwmchip_remove(&ax_pwm.chip);
51.     if (err < 0)
52.     {
53.         return err;
54.     }
55.
56.     return 0;
57. }
58.
59. static const struct of_device_id of_ax_pwm_match[] = {
60.     { .compatible = "alinx-pwm", },
61.     { /* Sentinel */ },
62. };
63.
64. static struct platform_driver ax_pwm_driver = {
65.     .driver = {
66.         .name = "alinx-pwm",
67.         .of_match_table = of_ax_pwm_match,
68.     },
69.     .probe = ax_pwm_probe,
70.     .remove = ax_pwm_remove,
71. };
72.
73. module_platform_driver(ax_pwm_driver);
```

pwm_device：

结合 misc 设备框架，注册 pwm_device。

```
1.  #define PWM_ON  0x100001
2.  #define PWM_OFF 0x100002
3.
4.  struct pwm_device *ax_pwm_dev;
5.
6.  static long ax_pwm_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
7.  {
```

```
    int ret;
    switch(cmd) {
        case PWM_ON:
                ret = pwm_config(ax_pwm_dev, 200000, 500000);
                pwm_enable(ax_pwm_dev);
            break;

        case PWM_OFF:
                ret = pwm_config(ax_pwm_dev, 0, 500000);
                pwm_disable(ax_pwm_dev);
            break;
    }
    return 0;
}

static struct file_operations ax_pwm_fops = {
    .owner = THIS_MODULE,
    .unlocked_ioctl = ax_pwm_ioctl,
};

static struct miscdevice pwm_misc = {
    .minor = MISC_DYNAMIC_MINOR,
    .name = "ax-pwm",
    .fops = &ax_pwm_fops
};
static int ax_pwm_init(void)
{
    int ret;
    struct device_node *nd;

    nd = of_find_node_by_path("/pwm-led");

    ax_pwm_dev = of_pwm_get(nd, "pwm-led");

    misc_register(&pwm_misc);
    return 0;
}

static void ax_pwm_exit(void)
{
    pwm_put(ax_pwm_dev);
}
module_init(ax_pwm_init);
module_exit(ax_pwm_exit);
```

10.3 实验

本章实验结合 platform 和 misc 框架，实现简单的 led 呼吸灯。

10.3.1 原理图

在底板原理图中找到 PL_LED 连接的连接器引脚。



再到核心板原理图中通过连接器引脚找到 pl 对应的引脚也就是 E10，是我们上面修改 vivado 工程时，xdc 文件中约束的引脚。



10.3.2 设备树

打开 system-user.dtsi 文件修改如下：

```
/include/  "system-conf.dtsi"
/ {
};

/*  SD  */
&sdhci1  {
      disable-wp;
```

```
        no-1-8-v;
};

/*  USB   */
&dwc3_0 {
        status = "okay";
        dr_mode = "host";
};
```

不需要手动添加 pwm 节点，petalinux 工程编译之后已经帮我们生成了。在“components/plnx_workspace/device-tree/device-tree”中，如下

Open ▾ pl.dtsi ~/peta_prj/linuxPsBase/petalinux/components/plnx_workspace/device-tree/device-tree

```
/*
 * CAUTION: This file is automatically generated by Xilinx.
 * Version:
 * Today is: Tue Apr 12 07:23:58 2022
 */

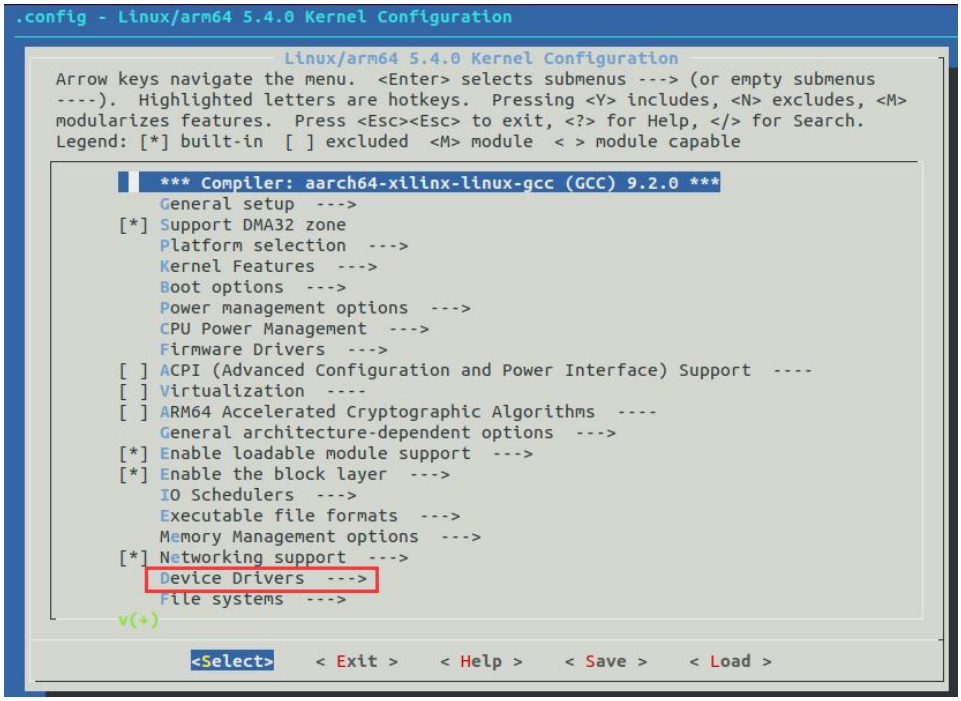

/ {
        amba_pl: amba_pl@0 {
                #address-cells = <2>;
                #size-cells = <2>;
                compatible = "simple-bus";
                ranges ;
                ax_pwm_0: ax_pwm@80000000 {
                        clock-names = "s00_axi_aclk";
                        clocks = <&zynqmp_clk 71>;
                        compatible = "xlnx,ax-pwm-1.0";
                        reg = <0x0 0x80000000 0x0 0x10000>;
                        xlnx,s00-axi-addr-width = <0x4>;
                        xlnx,s00-axi-data-width = <0x20>;
                };
        };
};
```

只需要保证驱动中的 compatible 字段和这里一致就行了。

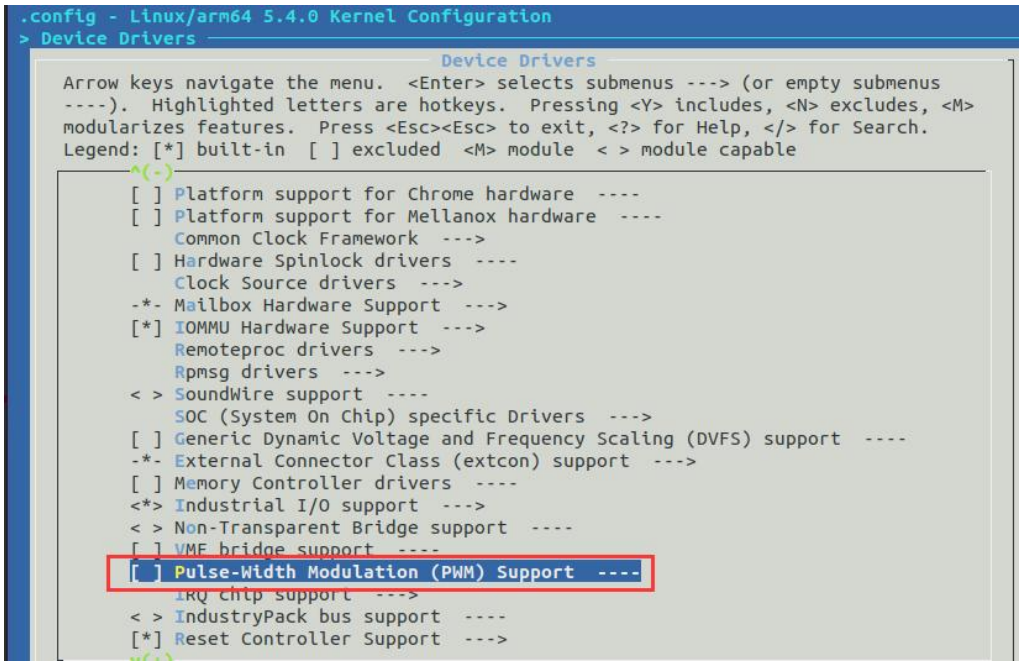
10.3.3 内核修改

```
petalinux-config -c kernel
```

进入内核，选择“Device Driver”



找到“Pulse-width Modulation (PWM) Support”，按空格键选上，保存退出。



编译 petalinux 工程，打包 BOOT.bin 等三个文件到 SD 卡中。

10.3.4 驱动程序

```
#include <linux/err.h>
#include <linux/platform_device.h>
#include <linux/pwm.h>
#include <linux/module.h>
#include <linux/io.h>

#define PWM_PERI 0x80000000
#define PWM_DUTY 0x80000004

struct ax_pwm_chip {
     struct pwm_chip chip;
     unsigned int *prei;
     unsigned int *duty;
};

static int ax_pwm_config(struct pwm_chip *chip, struct pwm_device *pwm,
                    int duty_ns, int period_ns)
{
     struct ax_pwm_chip *ax_pwm;
     int ret = 0;
     ax_pwm = container_of(chip, struct ax_pwm_chip, chip);

     *ax_pwm->prei = period_ns;
     *ax_pwm->duty = 0x8FFFFFFF + (0x4000 - duty_ns) * ((0xFFFFFFFF - 0x8FFFFFFF) / 0x4000);

     printk("pwm config : period_ns = %d, duty_ns = %d\n", period_ns, duty_ns);
     return ret;
}

static int ax_pwm_enable(struct pwm_chip *chip, struct pwm_device *pwm)
{
     int ret = 0;
     printk("pwm en\n");
     return ret;
}

static void ax_pwm_disable(struct pwm_chip *chip, struct pwm_device *pwm)
{
     printk("pwm dis\n");
}

static const struct pwm_ops ax_pwm_ops = {
     .config = ax_pwm_config,
     .enable = ax_pwm_enable,
```

```
	.disable = ax_pwm_disable,
	.owner = THIS_MODULE,
};

static int ax_pwm_probe(struct platform_device *pdev)
{
	struct ax_pwm_chip *ax_pwm;
	int err;

	ax_pwm = devm_kzalloc(&pdev->dev, sizeof(*ax_pwm), GFP_KERNEL);
	if (ax_pwm == NULL)
	{
		return -ENOMEM;
	}


	ax_pwm->chip.dev = &pdev->dev;
	ax_pwm->chip.ops = &ax_pwm_ops;
	ax_pwm->chip.base = pdev->id;
	ax_pwm->chip.npwm = 1;


	err = pwmchip_add(&ax_pwm->chip);
	if (err < 0)
	{
		return err;
	}


	ax_pwm->prei = ioremap_wc(PWM_PERI, 4);
	if(ax_pwm->prei == NULL)
		return -1;
	ax_pwm->duty = ioremap_wc(PWM_DUTY, 4);
	if(ax_pwm->duty == NULL)
		return -1;

	dev_dbg(&pdev->dev, "pwm probe successful\n");
	platform_set_drvdata(pdev, ax_pwm);

	return 0;
}

static int ax_pwm_remove(struct platform_device *pdev)

{
	struct ax_pwm_chip *ax_pwm = platform_get_drvdata(pdev);
	int err;

	err = pwmchip_remove(&ax_pwm->chip);
	if (err < 0)
		return err;

	dev_dbg(&pdev->dev, "pwm driver removed\n");

	return 0;
}

static const struct of_device_id pwm_of_match[] = {
    { .compatible = "xlnx,ax-pwm-1.0" },
    {/* Sentinel */}
};

static struct platform_driver ax_pwm_driver = {
	.driver = {
		.name = "ax-pwm",
		.of_match_table = pwm_of_match,
	},
	.probe = ax_pwm_probe,
	.remove = ax_pwm_remove,
};
module_platform_driver(ax_pwm_driver);

MODULE_LICENSE("GPL");
```

enable 函数和 disable 函数中应为我的 ip 不需要使能使能，空着就行，但一定要实现。

在 config 函数中设置频率和占空比，占空比这里需要换算。

10.3.5 运行测试

加载本章的驱动。

执行下面的命令

```
echo 0 > /sys/class/pwm/pwmchip0/export
echo 17179 > /sys/class/pwm/pwmchip0/pwm0/period
echo 0x2000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
```

这三条命令分别是导出 pwm0，设置 pwm0 的频率，设置 pwm0 的占空比。重复执行最后一天条命令，设置不同的占空比来查看 led 呼吸的现象。

```
root@petalinux:/mnt# insmod ./ax_pwm.ko
root@petalinux:/mnt# echo 0 > /sys/class/pwm/pwmchip0/export
root@petalinux:/mnt# echo 17179 > /sys/class/pwm/pwmchip0/pwm0/period
[  378.079101] pwm config : period_ns = 17179, duty_ns = 0
root@petalinux:/mnt# echo 0x100 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
[  482.135009] pwm config : period_ns = 17179, duty_ns = 256
```




```
root@petalinux:/mnt# echo 0x4000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
[  498.155015] pwm config : period_ns = 17179, duty_ns = 16384
```



第十一章 I2C 驱动

I2C 是常用的串行接口总线，Linux 内核中也提供了完善的 I2C 驱动框架。本章就来了解一下 Linux 内核中提供的 I2C 驱动框架。

这一章我们使用的 petalinux 工程是“Linux 基础教程”中“第一章 使用 Petalinux 定制 Linux 系统”制作的。

11.1 I2C 驱动框架

在 platform 驱动框架和 pwm 驱动框架中，都提到过驱动的分离，也就是控制器或总线和设备的分离。I2C 也是相似的结构，分为 I2C 总线和 I2C 设备。总线是芯片本身的 I2C 资源，而设备则是 I2C 外接的用户设备如 RTC、EEPROM 等。我们分别来看他们的实现方法。

11.1.1 I2C 控制器驱动

内核中使用结构体 i2c_adapter 来表示 I2C 控制器，i2c_adapter 结构体定义在文件 include/linux/i2c.h 中。如下：

```
546.struct i2c_adapter {
547.    struct module *owner;
548.    unsigned int class;       /* classes to allow probing for */
549.    const struct i2c_algorithm *algo; /* the algorithm to access the bus */
550.    void *algo_data;
551.
552.    /* data fields that are valid for all devices   */
553.    const struct i2c_lock_operations *lock_ops;
554.    struct rt_mutex bus_lock;
555.    struct rt_mutex mux_lock;
556.
557.    int timeout;            /* in jiffies */
558.    int retries;
559.    struct device dev;      /* the adapter device */
560.
561.    int nr;
562.    char name[48];
563.    struct completion dev_released;
564.
565.    struct mutex userspace_clients_lock;
566.    struct list_head userspace_clients;
567.
568.    struct i2c_bus_recovery_info *bus_recovery_info;
569.    const struct i2c_adapter_quirks *quirks;
570.};
```

549 行的 const struct i2c_algorithm 型指针成员变量 algo，是 I2C 设备访问总线的接口函数的合集，是 I2C 设备和 I2C 控制器通讯的方法。

i2c_algorithm 结构体定义如下：

```
407.struct i2c_algorithm {
408.    /* If an adapter algorithm can't do I2C-level access, set master_xfer
409.       to NULL. If an adapter algorithm can do SMBus access, set
410.       smbus_xfer. If set to NULL, the SMBus protocol is simulated
411.       using common I2C messages */
412.    /* master_xfer should return the number of messages successfully
413.       processed, or a negative value on error */
414.    int (*master_xfer)(struct i2c_adapter *adap, struct i2c_msg *msgs,
415.        int num);
```

```
416.    int (*smbus_xfer) (struct i2c_adapter *adap, u16 addr,
417.        unsigned short flags, char read_write,
418.        u8 command, int size, union i2c_smbus_data *data);
419.
420.    /* To determine what the adapter supports */
421.    u32 (*functionality) (struct i2c_adapter *);
422.
423.#if IS_ENABLED(CONFIG_I2C_SLAVE)
424.    int (*reg_slave)(struct i2c_client *client);
425.    int (*unreg_slave)(struct i2c_client *client);
426.#endif
```

414 行的 master_xfer 函数就是用于与 I2C 设备通讯的函数。

416 行的 smbus_xfer 函数是 smbus 的传输函数。

I2C 总线驱动的实现首先要定义并初始化一个 i2c_adapter 结构体型的变量，初始化包括实现 i2c_algorithm 中的函数，至少要实现 master_xfer 函数。

初始化完成后，使用下面的函数向内核注册 i2c_algorithm：

```
int i2c_add_adapter(struct i2c_adapter *adapter)
int i2c_add_numbered_adapter(struct i2c_adapter *adap)
```

这两个函数的区别是前者使用动态总线号，后者使用静态总线号。输入参数为初始化后的 i2c_add_adapter 结构体变量。返回 0 注册成功，返回负值注册失败。

相对的注销函数为：

```
void i2c_del_adapter(struct i2c_adapter * adap)
```

输入参数 adap 为需要删除的 I2C 控制器。

I2C 控制器的驱动一般由芯片厂家提供，从设备树中找到 I2C 控制器的节点，可以发现节点中的 compatible 属性为 cdns,i2c-r1p10，通过这个 compatible 可以追寻到，使用的 I2C 控制器驱动为 drivers\i2c\busses\i2c-cadence.c。I2C 控制器的驱动相关暂且了解这么多，把重点放在 I2C 的设备驱动中去。

11.1.2 I2C 设备驱动

设备驱动又分为两个部分，设备 i2c_client 和驱动 i2c_driver。

1) i2c_client

i2c_client 用于描述设备信息，定义如下：

```
228.struct i2c_client {
229.    unsigned short flags;       /* div., see below      */
230.    unsigned short addr;        /* chip address - NOTE: 7bit    */
231.                    /* addresses are stored in the  */
232.                    /* _LOWER_ 7 bits       */
233.    char name[I2C_NAME_SIZE];
234.    struct i2c_adapter *adapter;    /* the adapter we sit on    */
235.    struct device dev;      /* the device structure     */
236.    int irq;            /* irq issued by device     */
237.    struct list_head detected;
238.#if IS_ENABLED(CONFIG_I2C_SLAVE)
239.    i2c_slave_cb_t slave_cb;    /* callback for slave mode  */
240.#endif
241.};
```

230 行的 addr 表示芯片地址，储存在低 7 位。

233 行的 name 表示设备名称。

234 行的 adapter 位设备对应 I2C 控制器。

每一个 I2C 设备对应一个 i2c_client。

2) i2c_driver

i2c_driver 是 Linux 框架中 I2C 处理的重点，定义如下：

```
169.struct i2c_driver {
170.    unsigned int class;
171.
172.    /* Notifies the driver that a new bus has appeared. You should avoid
173.    * using this, it will be removed in a near future.
174.    */
175.    int (*attach_adapter)(struct i2c_adapter *) __deprecated;
176.
177.    /* Standard driver model interfaces */
178.    int (*probe)(struct i2c_client *, const struct i2c_device_id *);
179.    int (*remove)(struct i2c_client *);
180.
181.    /* driver model interfaces that don't relate to enumeration  */
182.    void (*shutdown)(struct i2c_client *);
183.
184.    /* Alert callback, for example for the SMBus alert protocol.
185.    * The format and meaning of the data value depends on the protocol.
186.    * For the SMBus alert protocol, there is a single bit of data passed
187.    * as the alert response's low bit ("event flag").
188.    * For the SMBus Host Notify protocol, the data corresponds to the
189.    * 16-bit payload data reported by the slave device acting as master.
190.    */
191.    void (*alert)(struct i2c_client *, enum i2c_alert_protocol protocol,
192.            unsigned int data);
193.
194.    /* a ioctl like command that can be used to perform specific functions
195.    * with the device.
196.    */
197.    int (*command)(struct i2c_client *client, unsigned int cmd, void *arg);
198.
199.    struct device_driver driver;
200.    const struct i2c_device_id *id_table;
201.
202.    /* Device detection callback for automatic device creation */
203.    int (*detect)(struct i2c_client *, struct i2c_board_info *);
204.    const unsigned short *address_list;
205.    struct list_head clients;
206.};
```

178 行的 probe 函数和 platform 框架中的类似，I2C 设备和驱动匹配成功后就会执行。

199 行的 device_driver 结构体变量 driver 就是用于和设备匹配，和 platform 框架类似，使用设备树的话，需要设置 driver.of_match_table 中的 compatible 属性。

200 行的 id_table 和 platform 框架类似，未使用设备树时使用这个匹配表。

定义并初始化完成 i2c_driver 之后使用下面的函数来向内核注册：

```
int i2c_register_driver(struct module *owner, struct i2c_driver *driver)
```

owner 一般位 THIS_MODULE。

driver 就是需要注册的 i2c_driver。

返回 0 成功，负值失败。

相对的注销函数为：

```
void i2c_del_driver(struct i2c_driver *driver)
```

I2C 设备驱动编写示例：

```
1.  static int ax_probe(struct i2c_client *client, const struct i2c_device_id *id)
2.  {
3.      return 0;
4.  }
5.
```

```
6.  static int ax_remove(struct i2c_client *client)
7.  {
8.      return 0;
9.  }
10.
11. static const struct of_device_id ax_of_match[] =
12. {
13.     { .compatible = "alinx-xxx"},
14.     {/* sentinel */}
15. };
16.
17. static struct i2c_driver ax_driver = {
18.     .driver = {
19.         .owner = THIS_MODULE,
20.         .name   = "alinx-xxx",
21.         .of_match_table = ax_of_match,
22.     },
23.     .probe = ax_probe,
24.     .remove = ax_remove,
25. };
26.
27.
28. static int ax_init(void)
29. {
30.     i2c_add_driver(&ax_driver);
31.     return 0;
32. }
33.
34. static void ax_exit(void)
35. {
36.     i2c_del_driver(&ax_driver);
37. }
38.
39. module_init(ax_init);
40. module_exit(ax_exit);
```

11.1.3 I2C 设备驱动的实现流程

1) 设备树

首先要在对应的 I2C 节点中添加设备节点，如下：

```
1.  i2c0: i2c@e0004000 {
2.      compatible = "cdns,i2c-r1p10";
3.      clocks = <&clkc 38>;
4.      interrupt-parent = <&intc>;
5.      interrupts = <0 25 4>;
6.      reg = <0xe0004000 0x1000>;
7.      #address-cells = <1>;
8.      #size-cells = <0>;
9.  };
10. ……
11. &i2c0 {
12.     axrtc@68 {
13.         compatible = "alinx-rtc";
14.         status = "okay";
15.         reg = <0x68>;
16.     };
17. };
```

第一行的 i2c0 是控制器的节点。第 11 行引用控制器节点&i2c，并在里面添加设备节点 axrtc。

设备节点名称后面@接的是设备的地址 68。设备节点的关键属性有两个。compatible 兼容性用于和设备驱动相匹配。reg 和节点名@后面的值相同，都是设备地址。

设备树配置好后就可以仔 probe 函数中获取设备地址。

2) 数据收发

I2C 的手法通过内核中的 i2c_transfer 函数实现，这个函数最终会调用 i2c 控制器驱动中的 master_xfer 函数。

i2c_transfer 函数定义在 include/linux/i2c.h 中，原型如下：

```
int i2c_transfer(struct i2c_adapter *adap, struct i2c_msg *msgs, int num)
```

参数 adap 可以在 probe 函数中获取，probe 函数被调用时，第一个输入参数为设备树中对应节点的 struct i2c_client *client，client 中的 adap 也就是对应的控制器。

参数 msgs 是需要发送的数据。

参数 num 为需要发送 msgs 数量。

返回负值失败，返回非负值为 msgs 发送的数量。

msgs 的为 struct i2c_msg 类型的指针，struct i2c_msg 定义在 include/uapi/linux/i2c.h 中如下：

```
1.  struct i2c_msg {
2.      __u16 addr; /* slave address            */
3.      __u16 flags;
4.  #define I2C_M_RD        0x0001  /* read data, from slave to master */
5.                      /* I2C_M_RD is guaranteed to be 0x0001! */
6.  #define I2C_M_TEN       0x0010  /* this is a ten bit chip address */
7.  #define I2C_M_RECV_LEN      0x0400  /* length will be first received byte */
8.  #define I2C_M_NO_RD_ACK     0x0800  /* if I2C_FUNC_PROTOCOL_MANGLING */
9.  #define I2C_M_IGNORE_NAK    0x1000  /* if I2C_FUNC_PROTOCOL_MANGLING */
10. #define I2C_M_REV_DIR_ADDR  0x2000  /* if I2C_FUNC_PROTOCOL_MANGLING */
11. #define I2C_M_NOSTART       0x4000  /* if I2C_FUNC_NOSTART */
12. #define I2C_M_STOP      0x8000  /* if I2C_FUNC_PROTOCOL_MANGLING */
13.     __u16 len;      /* msg length                */
14.     __u8 *buf;      /* pointer to msg data           */
15. };
```

根据 flags 的值，i2c_transfer 函数执行不同的工作，包括读写。

调用 i2c_transfer 函数之前，需要先构建 struct i2c_msg 变量，如下：

```
1.  static int ax_read_regs(struct axrtc_dev *dev, u8 reg, void *val, int len)
2.  {
3.      int ret;
4.      struct i2c_msg msg[2];
5.      struct i2c_client *client = (struct i2c_client *)dev->private_data;
6.
7.      msg[0].addr = client->addr;
8.      msg[0].flags = 0;
9.      msg[0].buf = &reg
10.     msg[0].len = 1;
11.
12.     msg[1].addr = client->addr;
13.     msg[1].flags = I2C_M_RD;
14.     msg[1].buf = val;
15.     msg[1].len = len;
16.
17.     ret = i2c_transfer(client->adapter, msg, 2);
18.     if(2 == ret)
19.     {
20.         ret = 0;
21.     }
22.     else
23.     {
24.         printk("i2c read failed %d", ret);
25.         ret = -EREMOTEIO;
26.     }
27.
28.     return ret;
29. }
```

第 4 行定义了 struct i2c_msg 类型的数组 msg。

这里的 client 获取了 dev->private_data 中的值，dev->private_data 是仔 probe 中设置的私有变量，具体到后面的实验中再去分析，先看 msg 的构造。

7~10 行构建 msg[0]，addr 为设备地址值，使用 client 中的 addr 即可。flags 赋为 0 时为写。flags 为写时 buf 的值为写入的数据的首地址。len 为写入数据的长度。

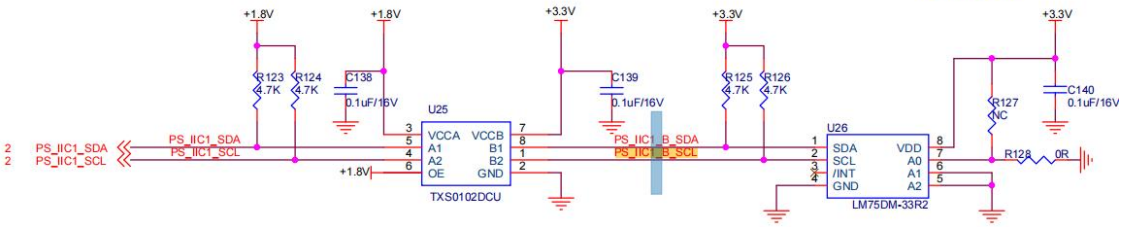
12~15 行构建 msg[1]，flag 等于 I2C_M_RD 为读数据，此时 buf 为储存读出数据 buffer 的首地址。len 为读出数据的长度。

11.3 实验

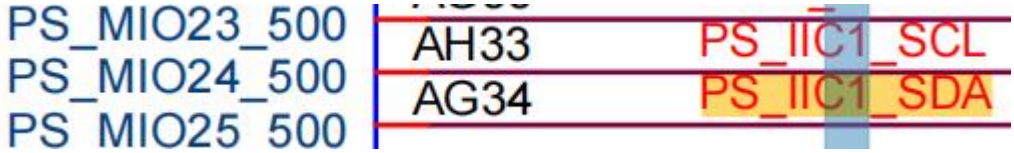
本章的实验使用 I2C 来做一个开发板上的 eeprom 数据读写实验。主要目标是验证 i2c 框架的使用是否正确，先往 eeprom 中写入数据再读出，两次数据一致则成功。

11.3.1 原理图

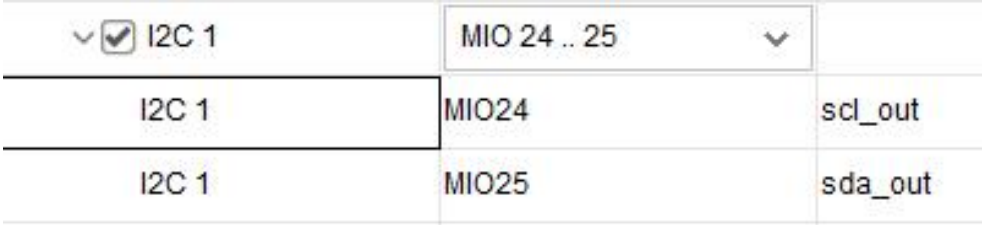
找到 eeprom 连接的 iic，和传感器共用一路。通过一个转换芯片，最终到 PS_IIC1_SDA。



通过 PS_IIC_SDA_1V8 找到连接到芯片的引脚，在 ps 端的 PS_MIO24、PS_MIO25。



查看 vivado 中 ultra scale+的配置，这里已经配置好了，把 I2C1 通道配置到 PS_MIO24、PS_MIO25 两个引脚上。



11.3.2 设备树

打开 system-user.dtsi 文件，在根目录外添加下面的节点：

```
/include/  "system-conf.dtsi"
/ {
};

/*  SD  */
&sdhci1  {
      disable-wp;
      no-1-8-v;
};
```

```
/* USB   */
&dwc3_0 {
     status = "okay";
     dr_mode = "host";
};

&i2c1 {
     status = "okay";

     i2c_eeprom@50 {
          compatible = "ax_eeprom";
          reg = <0x50>;
     };
}
```

最后引用 i2c1 节点，在里面添加 i2c_eeprom 节点。

11.3.3 驱动程序

```
#include <linux/types.h>
#include <linux/kernel.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/uaccess.h>
#include <linux/module.h>
#include <linux/delay.h>
#include <linux/i2c.h>

struct ax_eeprom_dev
{
    dev_t devid;
    struct cdev cdev;
    struct class *class;
    struct device *device;
    struct i2c_client *client;
};


static int char_dev_open(struct inode *inode_p, struct file *file_p)
{
    int ret = 0;
    struct ax_eeprom_dev *ax_eeprom;

    ax_eeprom = container_of(inode_p->i_cdev, struct ax_eeprom_dev, cdev);
    file_p->private_data = ax_eeprom;

    return ret;
}

static int char_dev_release(struct inode *inode_p, struct file *file_p)
{
    return 0;
}

static ssize_t char_dev_read(struct file *file_p, char __user *buf, size_t len, loff_t *loff_t_p)

{
    int ret = 0;
    struct ax_eeprom_dev *ax_eeprom = file_p->private_data;
    struct i2c_msg msgs[2];
    unsigned char lo_buf[256] = {0};
    unsigned char data_addr_slave;
    ret = copy_from_user(&data_addr_slave, (unsigned char *)buf, 1);
    if(ret < 0)
    {
        ret = -1;
        printk("i2c read copy_from_user err\n");
        goto err;
```

```
    }

    msgs[0].addr = ax_eeprom->client->addr;
    msgs[0].flags = 0;
    msgs[0].buf = &data_addr_slave;
    msgs[0].len = 1;

    msgs[1].addr = ax_eeprom->client->addr;
    msgs[1].flags = I2C_M_RD;
    msgs[1].buf = lo_buf;
    msgs[1].len = len > (sizeof(lo_buf) - data_addr_slave) ? (sizeof(lo_buf) - data_addr_slave) : len;

    ret = i2c_transfer(ax_eeprom->client->adapter, msgs, 2);
    if(ret == 2)
    {
        ret = copy_to_user(buf, (char *)lo_buf, len > (sizeof(lo_buf) - data_addr_slave) ? (sizeof(lo_buf) - data_addr_slave) : len);
        if(ret < 0)
        {
            ret = -1;
            printk("i2c read copy_to_user err\n");
            goto err;
        }
    }
    else
    {
        printk("i2c read i2c_transfer err\n");
        ret = -1;
    }

    mdelay(10);
err:
    return ret;
}

static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t *loff_t_p)
{
    int ret = 0;
    struct ax_eeprom_dev *ax_eeprom = file_p->private_data;
    struct i2c_msg msgs[1];
    unsigned char lo_buf[17] = {0};
    ret = copy_from_user(lo_buf, (unsigned char *)buf, len > sizeof(lo_buf) ? sizeof(lo_buf) : len);
    if(ret < 0)
    {
        ret = -1;
        printk("i2c write copy_from_user err\n");
        goto err;
    }

    msgs[0].addr = ax_eeprom->client->addr;
    msgs[0].flags = 0;
    msgs[0].buf = lo_buf;
    msgs[0].len = (len > sizeof(lo_buf) ? sizeof(lo_buf) : len) > (256 - lo_buf[0] + 1) ? (256 - lo_buf[0] + 1) : (len > sizeof(lo_buf) ? sizeof(lo_buf) : len);

    ret = i2c_transfer(ax_eeprom->client->adapter, msgs, 1);
    if(ret == 1)
    {
        ret = 0;
    }
    else
    {
        printk("i2c write i2c_transfer err\n");
        ret = -1;
    }

    mdelay(10);
err:
    return ret;
}

static struct file_operations char_dev_opt = {
    .owner = THIS_MODULE,
    .open = char_dev_open,
    .read = char_dev_read,
    .write = char_dev_write,
```

```
125.     .release = char_dev_release,
126. };
127. 
128. 
129. static int axi2c_probe(struct i2c_client *client, const struct i2c_device_id *id)
130. {
131.     int ret = 0;
132.     struct ax_eeprom_dev *ax_eeprom;
133. 
134.     ax_eeprom = devm_kzalloc(&client->dev, sizeof(struct ax_eeprom_dev), GFP_KERNEL);
135.     if(ax_eeprom == NULL)
136.     {
137.         ret = -1;
138.         goto err;
139.     }
140.     ax_eeprom->client = client;
141. 
142.     i2c_set_clientdata(client, ax_eeprom);
143. 
144.     alloc_chrdev_region(&ax_eeprom->devid, 0, 1, "ax_eeprom");
145. 
146.     cdev_init(&ax_eeprom->cdev, &char_dev_opt);
147.     cdev_add(&ax_eeprom->cdev, ax_eeprom->devid, 1);
148. 
149.     ax_eeprom->class = class_create(THIS_MODULE, "ax_eeprom");
150.     if(IS_ERR(ax_eeprom->class))
151.     {
152.         ret = PTR_ERR(ax_eeprom->class);
153.         goto err;
154.     }
155. 
156.     ax_eeprom->device = device_create(ax_eeprom->class, NULL, ax_eeprom->devid, NULL, "ax_eeprom
    ");
157.     if(IS_ERR(ax_eeprom->device))
158.     {
159.         ret = PTR_ERR(ax_eeprom->device);
160.         goto err;
161.     }
162. 
163. err:
164.     return ret;
165. }
166. 
167. static int axi2c_remove(struct i2c_client *client)
168. {
169.     struct ax_eeprom_dev *ax_eeprom = i2c_get_clientdata(client);
170.     device_destroy(ax_eeprom->class, ax_eeprom->devid);
171.     class_destroy(ax_eeprom->class);
172.     cdev_del(&ax_eeprom->cdev);
173.     unregister_chrdev_region(ax_eeprom->devid, 1);
174. 
175.     return 0;
176. }
177. 
178. static const struct of_device_id axi2c_of_match[] =
179. {
180.     { .compatible = "ax_eeprom" },
181.     {}
182. };
183. 
184. static const struct i2c_device_id axi2c_id[] = {
185.     {"ax_eeprom"},
186.     {}
187. };
188. 
189. static struct i2c_driver axi2c_drv =
190. {
191.     .driver =
192.     {
193.         .owner = THIS_MODULE,
194.         .name = "ax_eeprom",
195.         .of_match_table = axi2c_of_match,
196.     },
197.     .id_table = axi2c_id,
198.     .probe = axi2c_probe,
199.     .remove = axi2c_remove,
200. };
201. 
202. static int __init ax_i2c_init(void)
```

```
203. {
204.     return i2c_add_driver(&axi2c_drv);
205. }
206.
207. static void __exit ax_i2c_exit(void)
208. {
209.     i2c_del_driver(&axi2c_drv);
210. }
211.
212. module_init(ax_i2c_init);
213. module_exit(ax_i2c_exit);
214.
215. MODULE_LICENSE("GPL v2");
```

在 open 函数中设置私有数据。

i2c 读取中构建 msg 一般构建一个两个元素的数组，第一个 msg 标记为写用于发送目标数据地址，第二个 msg 标记为读。

probe 函数，内容就是字符设备的注册。

of 匹配表，compatible 字段和设备树中的保持一致。

id_table 最好也实现以下，即使用不到。

驱动入口函数中使用 i2c_add_driver 函数注册 i2c 驱动。i2c_add_driver 函数是对 i2c_register_driver 的封装，省去了输入 THIS_MODULE。

在驱动出口函数中使用 i2c_del_driver 删除驱动。

11.3.4 测试程序

```
1.  #include <stdio.h>
2.  #include <string.h>
3.  #include <unistd.h>
4.  #include <fcntl.h>
5.
6.  int main(int argc, char *argv[])
7.  {
8.      int fd, ret = 0;
9.      char *filename;
10.
11.     unsigned char w_buf[256] = {0};
12.     unsigned char r_buf[256] = {0};
13.     unsigned char s_buf[17] = {0};
14.
15.     for(int i = 0; i < 256; i ++)
16.     {
17.         w_buf[i] = i;
18.     }
19.
20.     if(argc != 2)
21.     {
22.         printf("Error Usage\r\n");
23.         return -1;
24.     }
25.
26.     filename = argv[1];
27.     fd = open(filename, O_RDWR);
28.     if(fd < 0)
29.     {
30.         printf("file %s open failed\r\n", argv[1]);
31.         return -1;
32.     }
33.
34.     for(int i = 0; i < 16; i ++)
35.     {
36.         s_buf[0] = i * 16;
37.         memcpy(&s_buf[1], &w_buf[s_buf[0]], 16);
38.         ret = write(fd, s_buf, 17);
39.         if(ret < 0)
40.         {
41.             printf("eeprom write err %d\n", i);
42.             return 0;
```

```
43.         }
44.     }
45.
46.     r_buf[0] = 0;
47.     ret = read(fd, r_buf, 256);
48.     if(ret < 0)
49.     {
50.         printf("eeprom read err\n");
51.         return 0;
52.     }
53.
54.     if(!memcmp(r_buf, w_buf, 256))
55.     {
56.         printf("eeprom test ok\n");
57.     }
58.     else
59.     {
60.         printf("eeprom test ng\n");
61.     }
62. }
```

应用程序就是调用 write 先写数据，在 read 都出来对比数据，看两次数据是否一致。

11.3.5 运行测试

加载驱动，运行应用程序加上设备描述文件作为参数。查看打印信息。

```
root@petalinux:~# cd /mnt/
root@petalinux:/mnt# ls
ax_eeprom              char_drv_test_conc    linux
ax_i2c.ko              gpio-in               linux
ax_pwm.ko              input_drv.ko          linux
char_drv_test          input_test            pl-ke
root@petalinux:/mnt# insmod ax_i2c.ko
root@petalinux:/mnt# ./ax_eeprom /dev/ax_eeprom
eeprom test ok
root@petalinux:/mnt#
```

第十二章 SPI 驱动

和 usb、i2c 一样，SPI 驱动的框架也分为总线(控制器)驱动和设备驱动。控制器驱动由内核提供也是不需要我们操心，重点还是放在设备驱动的实现上。

12.1 SPI 控制器驱动

大概了解一下 SPI 控制器总线。内核中使用 spi_master 结构体来表示一个 SPI 主机控制器，定义在文件 include/linux/spi/spi.h 中的 402 行。用 i2c 总线框来类比，spi_master 中也有数据传输的函数 transfer、transfer_one_message 等。transfer_one_message 的区别在于会把数据包保存 spi_massage，以队列形式发送。这些功能函数都由芯片厂家提供，我们稍做了解就行了。

1) spi_master 申请与释放

定义好 spi_master 结构体变量之后，使用下面的方法向内核申请：

```
struct spi_master *spi_alloc_master(struct device *dev, unsigned size)
```

dev 一般是 platform_device 中的 dev 成员变量。

size 是私有数据大小，可以通过函数 spi_master_get_devdata 获取私有数据。

申请成功返回 spi_master。

释放 spi_master 使用下面的函数：

```
void spi_master_put(struct spi_master *master)
```

2) spi_master 注册于注销

spi_master 初始化完成后使用下面的函数注册：

```
int spi_register_master(struct spi_master *master)
```

使用下面的函数注销：

```
void spi_unregister_master(struct spi_master *master)
```

12.2 SPI 设备驱动

1) spi_driver

Linux 内核中使用 spi_driver 结构体来表示 spi 设备驱动。spi_driver 结构体定义在文件 include/linux/spi/spi.h 中如下：

```
1.  struct spi_driver {
2.      const struct spi_device_id *id_table;
3.      int         (*probe)(struct spi_device *spi);
4.      int         (*remove)(struct spi_device *spi);
5.      void            (*shutdown)(struct spi_device *spi);
6.      struct device_driver    driver;
7.  };
```

id_table 用于匹配 spi 设备，匹配成功就会执行 probe 函数。设备移除时则执行 remove 函数。

spi_driver 初始化完成后，使用下面的函数向内核注册：

```
int spi_register_driver(struct spi_driver *sdrv)
```

使用下面的函数注销 spi_driver：

```
void spi_unregister_driver(struct spi_driver *sdrv)
```

spi_driver 框架示例如下：

```
static int ax_probe(struct spi_device *spi)
{
    return 0;
}

static int ax_remove(struct spi_device *spi)
{
    return 0;
}

static const struct spi_device_id ax_id[] = {
    {"xxx", 0},
    { }
};

static const struct of_device_id ax_of_match[] = {
    { .compatible = "xxx" },
    { }
};

static struct spi_driver ax_driver = {
    .probe = ax_probe,
    .remove = ax_remove,
    .driver = {
        .owner = THIS_MODULE,
        .name = "xxx",
        .of_match_table = ax_of_match,
    },
    .id_table = ax_id,
};

static int __init ax_init(void)
{
    return spi_register_driver(&ax_driver);
}

static void __exit ax_exit(void)
{
    spi_unregister_driver(&ax_driver);
}

module_init(ax_init);
module_exit(ax_exit);
```

编写 spi 设备驱动，至少需要我们去实现其中 probe 和 remove 两个函数。

2) spi 设备

内核中用 spi_device 结构体表示 spi 设备，引入设备树后，spi_device 几乎很少用了，重点看一下设备树中的描述 spi 设备方法。spi 设备的设备树节点格式可参考文件 Documentation\devicetree\bindings\mtd 中的说明。示例如下：

```
qspi: spi@e000d000 {
    clock-names = "ref_clk", "pclk";
    clocks = <&clkc 10>, <&clkc 43>;
    compatible = "xlnx,zynq-qspi-1.0";
    status = "disabled";
    interrupt-parent = <&intc>;
    interrupts = <0 19 4>;
    reg = <0xe000d000 0x1000>;
    #address-cells = <1>;
    #size-cells = <0>;

```

```
12.     flash: w25q256@0
13.     {
14.         #address-cells = <1>;
15.         #size-cells = <1>;
16.         compatible = "w25q256";
17.         reg = <0>;
18.         spi-max-frequency = <40000000>;
19.         m25p,fast-read;
20.     };
21. };
```

第 1 行 qspi 时 zynqmp 上的其中一路 spi 总线，也就是总线节点。

2~10 行是 spi 总线节点的属性，这部分是 xilinx 提供的，硬件相关的信息都可以和寄存器手册中的对应，没什么特别要注意的。

12 行是设备节点，他是挂在于 qspi 这路 spi 总线下的设备。flash 是别称，这个节点就是个 flash 芯片。w25q256 是节点名称，后面的@0 是指这个设备接在这个 spi 总线的通道 0 上。

14、15 行的属性是老朋友了。

16 行的 compatible 是驱动和设备匹配时会用到的重要属性，他的值需要和 spi_driver 中的 conpatible 字段一致。这与前面的总线框架中遇到的也是一样的。

17 行的 reg 属性和@后一样，表示 spi 通道。

18 行的 spi-max-frequency 属性设置比 spi 的最高频率，这里的频率就是 20Mhz。

19 行的 fast-read 表示这个设备支持快速读取，根据实际情况，如果不支持就去掉。

3) 驱动和设备的匹配

这部分和 i2c 也很相似，驱动和设备的匹配在总线(控制器)驱动中完成。

spi 总线定义为结构体 spi_bus_type，在文件 drivers/spi/spi.c 文件中，如下：

```
1.  struct bus_type spi_bus_type = {
2.      .name       = "spi",
3.      .dev_groups = spi_dev_groups,
4.      .match      = spi_match_device,
5.      .uevent     = spi_uevent,
6.  };
```

match 函数就是匹配函数，内核中 match 函数的实现为函数 spi_match_device()。如下：

```
1.  static int spi_match_device(struct device *dev, struct device_driver *drv)
2.  {
3.      const struct spi_device *spi = to_spi_device(dev);
4.      const struct spi_driver *sdrv = to_spi_driver(drv);
5.
6.      /* Attempt an OF style match */
7.      if (of_driver_match_device(dev, drv))
8.      return 1;
9.
10.     /* Then try ACPI */
11.     if (acpi_driver_match_device(dev, drv))
12.     return 1;
13.
14.     if (sdrv->id_table)
15.
16.     return !!spi_match_id(sdrv->id_table, spi);
17.
18.     return strcmp(spi->modalias, drv->name) == 0;
19. }
```

有设备树、ACPI、id_tabel 和 name 四种匹配方式，和前面总线驱动框架的类似。

12.3 SPI 设备驱动中数据收发处理

spi 设备驱动中收发数据需要用到一些结构和函数，我们一一来看。

1) spi_transfer

spi_transfer 用于描述 spi 的传输信息，定义如下：

```
1.  struct spi_transfer {
2.      /* it's ok if tx_buf == rx_buf (right?)
3.       * for MicroWire, one buffer must be null
4.       * buffers must work with dma_*map_single() calls, unless
5.       *   spi_message.is_dma_mapped reports a pre-existing mapping
6.       */
7.      const void  *tx_buf;
8.      void        *rx_buf;
9.      unsigned    len;
10.
11.     dma_addr_t  tx_dma;
12.     dma_addr_t  rx_dma;
13.     struct sg_table tx_sg;
14.     struct sg_table rx_sg;
15.
16.     unsigned    cs_change:1;
17.     unsigned    tx_nbits:3;
18.     unsigned    rx_nbits:3;
19. #define SPI_NBITS_SINGLE    0x01 /* 1bit transfer */
20. #define SPI_NBITS_DUAL      0x02 /* 2bits transfer */
21. #define SPI_NBITS_QUAD      0x04 /* 4bits transfer */
22.     u8      bits_per_word;
23.     u16     delay_usecs;
24.     u32     speed_hz;
25.     u32     dummy;
26.     struct list_head transfer_list;
27. };
```

tx_buf、rx_buf 分别是保存发送和接收数据。

len 是数据长度，spi 是全双工通讯，在单词通讯中收发数据长度是一样的，所以只要一个 len 就行了。

2) spi_message

spi_message 相当于是 spi_transfer 的发送队列，spi_transfer 需要添加到 spi_message 中去发送。

3) spi_message_init()

spi_message 需要使用函数 spi_message_init()来初始化。

4) spi_message_add_tail()

spi_message 初始化完成后使用 spi_message_add_tail()函数把 spi_transfer 添加到 spi_message 中，函数原型为：

```
void spi_message_add_tail(struct spi_transfer *t, struct spi_message *m)
```

5) spi_sync()

spi_sync()函数使用同步阻塞的方式传输 spi_message，原型如下：

```
int spi_sync(struct spi_device *spi, struct spi_message *message)
```

6) spi_async()

spi_async()函数使用异步非阻塞的方式传输 spi_message，原型如下：

```
int spi_async(struct spi_device *spi, struct spi_message *message)
```

整体步骤示例如下，具体的写法到实验中再去看：

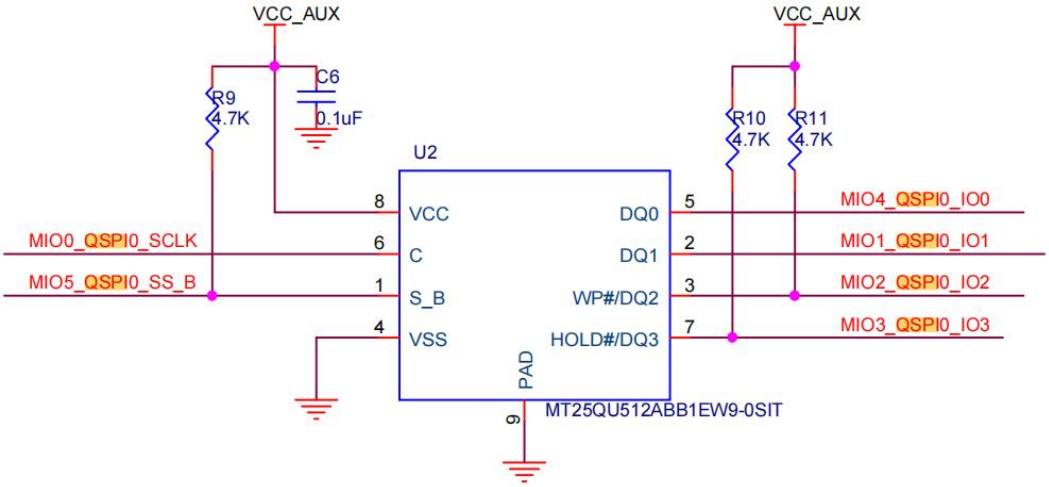
```
1.  static int ax_spi_send(struct spi_device *spi, u8 *buf, int len)
2.  {
3.      int ret;
4.      struct spi_message msg;
5.      struct spi_transfer trans =
6.      {
7.          .tx_buf = buf,
8.          .len = len,
9.      };
10.     spi_message_init(&msg);
11.     spi_message_add_tail(trans, &msg);
12.     ret = spi_sync(spi, &msg);
13.     return ret;
14. }
```

12.4 实验

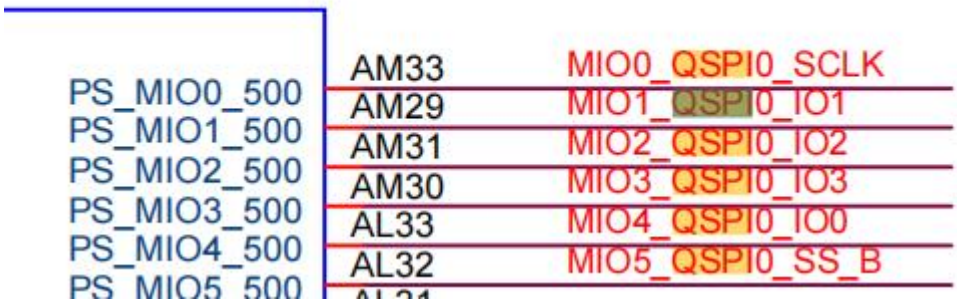
本章使用 zynqmp 上的 QSPI 读写 qflash。思路和 17 章的 i2c 差不多。

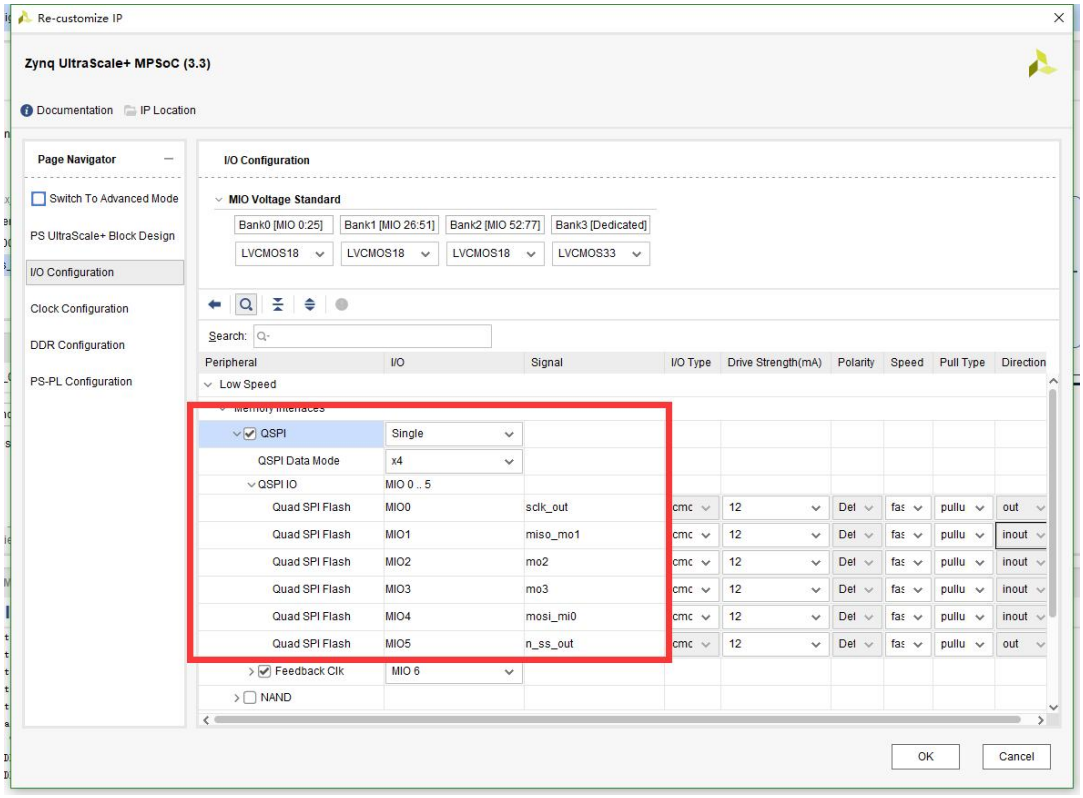
12.4.1 原理图

原理图中找到 flash 芯片如下。数据线时钟线这些都由总线驱动完成，不用我们操心。当然，如果需要自己实现总线驱动的话，就需要去了解这些线接到芯片的哪个引脚，这个引脚又需要做什么配置。



这里就不去看引脚的具体说明了。在 vivado 中，也能找到相应的配置。





12.4.2 设备树

打开 system-user.dtsi 文件，修改如下

```
/include/ "system-conf.dtsi"
/ {
};

/* SD */
&sdhci1 {
	disable-wp;
	no-1-8-v;
};

/* USB  */
&dwc3_0 {
	status = "okay";
	dr_mode = "host";
};

&qspi {
	status = "okay";

	qflash: mt25qu256@0 {
		compatible = "mt25qu256";
		reg = <0>;
		spi-max-frequency = <40000000>;
		m25p,fast-read;
	};
};
```

12.4.3 驱动程序

```
1.  #include <linux/types.h>
2.  #include <linux/kernel.h>
3.  #include <linux/cdev.h>
4.  #include <linux/device.h>
5.  #include <linux/fs.h>
6.  #include <linux/init.h>
7.  #include <linux/uaccess.h>
8.  #include <linux/module.h>
9.  #include <linux/delay.h>
```

```
#include <linux/spi/spi.h>
#include <linux/spi/flash.h>

#define CMD_SECTOR_ERASE    0xD8
#define CMD_BULK_ERASE      0xc7
#define CMD_WRITE_ENABLE    0x06
#define CMD_WAIT_READY      0x05
#define CMD_READ_BYTES      0x03
#define CMD_PAGE_PROGRAM    0x02
#define CMD_MAX             4

struct ax_qflash_dev
{
    dev_t devid;
    u8     cmd[CMD_MAX];
    struct cdev cdev;
    struct class *class;
    struct device *device;
    struct spi_device *client;
};

static int read_sr(struct ax_qflash_dev *dev)
{
    ssize_t retval;
    u8 code = 0x05;
    u8 val;
    struct spi_device *spi = dev->client;

    retval = spi_write_then_read(spi, &code, 1, &val, 1);

    if (retval < 0)
    {
        return retval;
    }

    return val;
}

static int wait_till_ready(struct ax_qflash_dev *dev)
{
    int count;
    int sr;

    for (count = 0; count < 100000; count++)
    {
        sr = read_sr(dev);
        if(!(sr & 0x00000001))
        {
            break;
        }
    }

    return 1;
}

static int ax_spi_erase(struct ax_qflash_dev *dev, loff_t addr)
{
    int ret;
    char cmd_buf[1] = {0};
    struct spi_device *spi = dev->client;
    struct spi_transfer trans[2] = {0};
    struct spi_message msg;
    spi_message_init(&msg);

    wait_till_ready(dev);

    cmd_buf[0] = CMD_WRITE_ENABLE;
    spi_write(spi, cmd_buf, 1);

    dev->cmd[0] = CMD_SECTOR_ERASE;
    dev->cmd[1] = addr >> 16;
    dev->cmd[2] = addr >> 8;
    dev->cmd[3] = addr;

    trans[0].tx_buf = dev->cmd;
    trans[0].len = CMD_MAX;
    spi_message_add_tail(&trans[0], &msg);

    ret = spi_sync(spi, &msg);
```

```
89.
90.     return ret;
91. }
92.
93. static int ax_spi_write(struct ax_qflash_dev *dev, loff_t addr, const char *buf, size_t len)
94. {
95.     int ret;
96.     char cmd_buf[1] = {0};
97.     struct spi_device *spi = dev->client;
98.     struct spi_transfer trans[2] = {0};
99.     struct spi_message msg;
100.    spi_message_init(&msg);
101.
102.    wait_till_ready(dev);
103.
104.    cmd_buf[0] = CMD_WRITE_ENABLE;
105.    spi_write(spi, cmd_buf, 1);
106.
107.    dev->cmd[0] = CMD_PAGE_PROGRAM;
108.    dev->cmd[1] = addr >> 16;
109.    dev->cmd[2] = addr >> 8;
110.    dev->cmd[3] = addr;
111.
112.    trans[0].tx_buf = dev->cmd;
113.    trans[0].len = CMD_MAX;
114.    spi_message_add_tail(&trans[0], &msg);
115.
116.    trans[1].tx_buf = buf;
117.    trans[1].len = len;
118.    spi_message_add_tail(&trans[1], &msg);
119.
120.    ret = spi_sync(spi, &msg);
121.
122.    return ret;
123. }
124.
125. static int ax_spi_read(struct ax_qflash_dev *dev, loff_t addr, char *buf, size_t len)
126. {
127.    int ret;
128.    struct spi_device *spi = dev->client;
129.    struct spi_transfer trans[2] = {0};
130.    struct spi_message msg;
131.    spi_message_init(&msg);
132.
133.    wait_till_ready(dev);
134.
135.    dev->cmd[0] = CMD_READ_BYTES;
136.    dev->cmd[1] = addr >> 16;
137.    dev->cmd[2] = addr >> 8;
138.    dev->cmd[3] = addr;
139.
140.    trans[0].tx_buf = dev->cmd;
141.    trans[0].len = CMD_MAX;
142.    spi_message_add_tail(&trans[0], &msg);
143.
144.    trans[1].rx_buf = buf;
145.    trans[1].len = len;
146.    spi_message_add_tail(&trans[1], &msg);
147.
148.    ret = spi_sync(spi, &msg);
149.
150.    return ret;
151. }
152.
153. static int char_dev_open(struct inode *inode_p, struct file *file_p)
154. {
155.    int ret = 0;
156.
157.    return ret;
158. }
159.
160. static int char_dev_release(struct inode *inode_p, struct file *file_p)
161. {
162.    return 0;
163. }
164.
165. static ssize_t char_dev_read(struct file *file_p, char __user *buf, size_t len, loff_t *loff_t_p)

166. {
```

```
167.     int ret = 0;
168.
169.     return ret;
170. }
171.
172. static ssize_t char_dev_write(struct file *file_p, const char __user *buf, size_t len, loff_t *l
     off_t_p)
173. {
174.     int ret = 0;
175.
176.     return ret;
177. }
178.
179. static struct file_operations char_dev_opt = {
180.     .owner = THIS_MODULE,
181.     .open = char_dev_open,
182.     .read = char_dev_read,
183.     .write = char_dev_write,
184.     .release = char_dev_release,
185. };
186.
187.
188. static int ax_spi_probe(struct spi_device *spi)
189. {
190.     int ret = 0, i = 0;
191.     struct ax_qflash_dev *ax_qflash;
192.
193.     u8 tx_data[200] = {0};
194.     u8 rx_data[200] = {0};
195.
196.     for(i = 0; i < sizeof(tx_data); i ++)
197.     {
198.         tx_data[i] = i;
199.     }
200.
201.     ax_qflash = devm_kzalloc(&spi->dev, sizeof(struct ax_qflash_dev), GFP_KERNEL);
202.     if(ax_qflash == NULL)
203.     {
204.         ret = -1;
205.         goto err;
206.     }
207.     ax_qflash->client = spi;
208.
209.     spi_set_drvdata(spi, ax_qflash);
210.
211.     alloc_chrdev_region(&ax_qflash->devid, 0, 1, "ax_qflash");
212.
213.     cdev_init(&ax_qflash->cdev, &char_dev_opt);
214.     cdev_add(&ax_qflash->cdev, ax_qflash->devid, 1);
215.
216.     ax_qflash->class = class_create(THIS_MODULE, "ax_qflash");
217.     if(IS_ERR(ax_qflash->class))
218.     {
219.         ret = PTR_ERR(ax_qflash->class);
220.         goto err;
221.     }
222.
223.     ax_qflash->device = device_create(ax_qflash->class, NULL, ax_qflash->devid, NULL, "ax_qflash
     ");
224.     if(IS_ERR(ax_qflash->device))
225.     {
226.         ret = PTR_ERR(ax_qflash->device);
227.         goto err;
228.     }
229.
230.     ax_spi_erase(ax_qflash, 0x11);
231.     mdelay(10);
232.     ax_spi_write(ax_qflash, 0x11, tx_data, sizeof(tx_data));
233.     mdelay(10);
234.     ax_spi_read(ax_qflash, 0x11, rx_data, sizeof(rx_data));
235.
236.     ret = memcmp(tx_data, rx_data, sizeof(tx_data));
237.     if(!ret)
238.     {
239.         printk("spi flash test ok\n");
240.     }
241.     else
242.     {
243.         printk("spi flash test ng %d\n", ret);
```

```
244.     }
245.
246. err:
247.     return 0;
248. }
249.
250. static int ax_spi_remove(struct spi_device *spi)
251. {
252.     struct ax_qflash_dev *ax_qflash = spi_get_drvdata(spi);
253.     device_destroy(ax_qflash->class, ax_qflash->devid);
254.     class_destroy(ax_qflash->class);
255.     cdev_del(&ax_qflash->cdev);
256.     unregister_chrdev_region(ax_qflash->devid, 1);
257.
258.     return 0;
259. }
260.
261. static const struct of_device_id ax_spi_of_match[] =
262. {
263.     { .compatible = "mt25qu256" },
264.     {}
265. };
266.
267. static const struct spi_device_id ax_spi_id[] = {
268.     {"mt25qu256"},
269.     {}
270. };
271.
272. static struct spi_driver ax_spi_drv =
273. {
274.     .driver =
275.     {
276.         .owner = THIS_MODULE,
277.         .name = "mt25qu256",
278.         .of_match_table = ax_spi_of_match,
279.     },
280.     .id_table = ax_spi_id,
281.     .probe = ax_spi_probe,
282.     .remove = ax_spi_remove,
283. };
284.
285. static int __init ax_spi_init(void)
286. {
287.     return spi_register_driver(&ax_spi_drv);
288. }
289.
290. static void __exit ax_spi_exit(void)
291. {
292.     spi_unregister_driver(&ax_spi_drv);
293. }
294.
295. module_init(ax_spi_init);
296. module_exit(ax_spi_exit);
297.
298. MODULE_LICENSE("GPL v2");
```

这节在驱动程序中完成测试。

spi 的读写和 i2c 比较相似，都是先打包再提交。

在 230 行即后面出现的 0x11 是写入的地址，可以改其他值，自定义即可，但需要注意地址和写入的数据的大小不能超过总长。

12.4.4 运行测试

加载驱动，查看打印信息即可。

```
root@petalinux:/mnt# insmod ax_spi.ko
[  108.268747] spi flash test ok
root@petalinux:/mnt#
```

第十三章 串口驱动

串口是常用的外设，和其他 mcu 通讯、和传感器 2G、 GPS 等模块通讯，都经常会选择串口。在 Linux 系统下也不例外，我们通过串口工具来和开发板进行交互，无疑就是用到了串口。串口有很多电平标准，TTL、232、485 等等，但他们的驱动程序都是一样的。在嵌入式 Linux 系统中，串口被看成终端设备（tty），包括 3 个结构体：uart_driver、uart_port、uart_ops 定义在文件 include/serial_core.h 中。实现一个 uart 驱动程序只要实现这 3 个结构体即可。

uart 的驱动一般也是由芯片厂家提供。xilinx 提供的 uart 驱动在 drivers/tty/serial/xilinx_uartps.c 中。我们只要在设备树中添加相应的串口节点，就可以使用串口外设了。这章就来学习 Linux 中串口驱动框架，并对 xilinx 的串口驱动实现稍做了解。

13.1 uart 驱动框架

1) usb_driver

Linux 内核中使用 usb_driver 来表示，uart_driver 包含了串口设备名、串口驱动名、主次设备号等信息，并且包含了 tty_driver 使得底层串口驱动无需关心 tty_driver。

uart_driver 定义在文件 include/linux/serial_core.h 中，如下：

```
1.  struct uart_driver {
2.      struct module       *owner;
3.      const char      *driver_name;
4.      const char      *dev_name;
5.      int          major;
6.      int          minor;
7.      int          nr;
8.      struct console      *cons;
9.
10.     /*
11.      * these are private; the low level driver should not
12.      * touch these; they should be initialised to NULL
13.      */
14.     struct uart_state   *state;
15.     struct tty_driver   *tty_driver;
16. };
```

onwer 一般为 THIS_MODULE。

driver_name 为驱动名。

dev_name 为设备名。

cons 为控制台。

定义好 uart_driver 后使用下面你的函数来向内核注册：

```
int uart_register_driver(struct uart_driver *drv)
```

drv：要注册的 uart_driver。

返回值：0 成功，负值失败。

相对的注销 uart_driver 使用下面的函数：

```
void uart_unregister_driver(struct uart_driver *drv)
```

drv 为要注销的 uart_driver。

2) uart_prot

uart_port 表示一个具体的 uart 端口，用于描述一个 uart 端口的 I/O 端口或 I/O 内存地址、FIFO 大小、端口类型等信息。定义在文件 include/linux/serial_core.h 中，如下：

```
117.struct uart_port {
118.    spinlock_t      lock;           /* port lock */
119.    unsigned long       iobase;         /* in/out[bwl] */
120.    unsigned char __iomem   *membase;       /* read/write[bwl] */
121.    unsigned int        (*serial_in)(struct uart_port *, int);
122.    void            (*serial_out)(struct uart_port *, int, int);
123.    void            (*set_termios)(struct uart_port *,
124.                                   struct ktermios *new,
125.                                   struct ktermios *old);
126.    unsigned int        (*get_mctrl)(struct uart_port *);
127.    void            (*set_mctrl)(struct uart_port *, unsigned int);
128.    int         (*startup)(struct uart_port *port);
129.    void            (*shutdown)(struct uart_port *port);
130.  ···········


237.    const struct uart_ops   *ops;
238.    unsigned int        custom_divisor;
239.    unsigned int        line;           /* port index */
240.    unsigned int        minor;
241.    resource_size_t     mapbase;        /* for ioremap */
242.    resource_size_t     mapsize;
243.    struct device       *dev;           /* parent device */
244.    unsigned char       hub6;           /* this should be in the 8250 driver */
245.    unsigned char       suspended;
246.    unsigned char       irq_wake;
247.    unsigned char       unused[2];
248.    struct attribute_group  *attr_group;        /* port specific attributes */
249.    const struct attribute_group **tty_groups;  /* all attributes (serial core use only) *
    /
250.    struct serial_rs485     rs485;
251.    void            *private_data;      /* generic platform data pointer */
252.};
```

其中最主要的就是 ops 串口驱动操作函数集。urat_port 定义后需要和 uart_driver 关联。使用下面的函数把 uart_port 添加到 uart_driver 中：

```
int uart_add_one_port(struct uart_driver *drv, struct uart_port *uport)
```

drv：注册目标 uart_port 对应的 uart_driver。

uport：要添加到 uart_driver 的 uart_port。

返回值： 0 成功；负值失败。

相对的使用下面的函数把 uart_port 从 uart_driver 中删除：

```
int uart_remove_one_port(struct uart_driver *drv, struct uart_port *uport)
```

drv：卸载目标 uart_port 对应的 uart_driver。

uport：要卸载的 uart_port。

返回值： 0 成功；负值失败。

3) uart_ops

uart_ops 是 uart_port 中重要的成员，是 uart 具体驱动函数的集合。内核使用串口收发数据最终都是调用 ops 中的函数。uart_ops 定义在文件 include/linux/serial_core.h 中，如下：

```
1.  struct uart_ops {
2.      unsigned int    (*tx_empty)(struct uart_port *);
3.      void        (*set_mctrl)(struct uart_port *, unsigned int mctrl);
4.      unsigned int    (*get_mctrl)(struct uart_port *);
5.      void        (*stop_tx)(struct uart_port *);
6.      void        (*start_tx)(struct uart_port *);
7.      void        (*throttle)(struct uart_port *);
```

```
    void        (*unthrottle)(struct uart_port *);
    void        (*send_xchar)(struct uart_port *, char ch);
    void        (*stop_rx)(struct uart_port *);
    void        (*enable_ms)(struct uart_port *);
    void        (*break_ctl)(struct uart_port *, int ctl);
    int     (*startup)(struct uart_port *);
    void        (*shutdown)(struct uart_port *);
    void        (*flush_buffer)(struct uart_port *);
    void        (*set_termios)(struct uart_port *, struct ktermios *new,
                       struct ktermios *old);
    void        (*set_ldisc)(struct uart_port *, struct ktermios *);
    void        (*pm)(struct uart_port *, unsigned int state,
                  unsigned int oldstate);

    /*
     * Return a string describing the type of the port
     */
    const char  *(*type)(struct uart_port *);

    /*
     * Release IO and memory resources used by the port.
     * This includes iounmap if necessary.
     */
    void        (*release_port)(struct uart_port *);

    /*
     * Request IO and memory resources used by the port.
     * This includes iomapping the port if necessary.
     */
    int     (*request_port)(struct uart_port *);
    void        (*config_port)(struct uart_port *, int);
    int     (*verify_port)(struct uart_port *, struct serial_struct *);
    int     (*ioctl)(struct uart_port *, unsigned int, unsigned long);
#ifdef CONFIG_CONSOLE_POLL
    int     (*poll_init)(struct uart_port *);
    void        (*poll_put_char)(struct uart_port *, unsigned char);
    int     (*poll_get_char)(struct uart_port *);
#endif
};
```

uart_ops 中的函数需要底层开发人员去实现，是直接和寄存器打交道的部分。uart_ops 结构体中的这些函数的具体含义可以参考文档 Documentation/serial/driver。

总结一下，一个串口驱动要完成的主要工作为：

1. 定义 uart_driver、uart_ops、uart_port 等结构体变量并初始化。
2. 驱动模块加载时使用 uart_register_driver()和 uart_add_one_port()函数注册 uart_driver 并添加端口。驱动模块卸载时使用函数 uart_unregister_driver()和 uart_remove_one_port()来注销 uart_driver 并移除端口。
3. 根据具体硬件的 datasheet 实现 uart_ops 中的成员函数。

然后我们再对应到具体的驱动代码来看。看看 xilinx 的 uart 实现。

13.2 xilinx 的 uart 驱动

xilinx 的 uartip 驱动在内核中默认都是配置好的，vivado 工程中添加了就可以直接用。

简单看一下源码，先看看设备树中 uart 的节点描述，打开文件 zynqmp.dtsi。找到 uart 相关节点，如下：

```
 uart0: serial@ff000000 {
     u-boot,dm-pre-reloc;
     compatible = "cdns,uart-r1p12", "xlnx,xuartps";
```

```
        status = "disabled";
        interrupt-parent = <&gic>;
        interrupts = <0 21 4>;
        reg = <0x0 0xff000000 0x0 0x1000>;
        clock-names = "uart_clk", "pclk";
        power-domains = <&zynqmp_firmware PD_UART_0>;
};

uart1: serial@ff010000 {
        u-boot,dm-pre-reloc;
        compatible = "cdns,uart-r1p12", "xlnx,xuartps";
        status = "disabled";
        interrupt-parent = <&gic>;
        interrupts = <0 22 4>;
        reg = <0x0 0xff010000 0x0 0x1000>;
        clock-names = "uart_clk", "pclk";
        power-domains = <&zynqmp_firmware PD_UART_1>;
};
```

节点中两个 uart 节点都是 ps 端的串口，zynqmp 的 ps 端就只有两路串口，如果需要更多，就需要借助 pl 端的资源。

首先根据 compatible 属性找到对应的驱动代码，为文件 drivers/tty/serial/xilinx_uartps.c。其中 of_device_id 如下：

```
/* Match table for of_platform binding */
static const struct of_device_id cdns_uart_of_match[] = {
        { .compatible = "xlnx,xuartps", },
        { .compatible = "cdns,uart-r1p8", },
        { .compatible = "cdns,uart-r1p12", .data = &zynqmp_uart_def },
        { .compatible = "xlnx,zynqmp-uart", .data = &zynqmp_uart_def },
        {}
};
```

在驱动代码 xilinx_uartps.c 中 1614 行，会发现如下代码：

```
static struct platform_driver cdns_uart_platform_driver = {
        .probe   = cdns_uart_probe,
        .remove  = cdns_uart_remove,
        .driver  = {
                .name = CDNS_UART_NAME,
                .of_match_table = cdns_uart_of_match,
                .pm = &cdns_uart_dev_pm_ops,
        },
};
```

可见，uart 本质上是一个 platform 驱动。

然后根据前面说的 uart 框架，来对应到这个驱动代码中去。

1) uart_driver

```
static struct uart_driver cdns_uart_uart_driver = {
        .owner          = THIS_MODULE,
        .driver_name    = CDNS_UART_NAME,
        .dev_name       = CDNS_UART_TTY_NAME,
        .major          = CDNS_UART_MAJOR,
        .minor          = CDNS_UART_MINOR,
        .nr     = CDNS_UART_NR_PORTS,
#ifdef CONFIG_SERIAL_XILINX_PS_UART_CONSOLE
        .cons           = &cdns_uart_console,
#endif
};
……
static int __init cdns_uart_init(void)
{
```

```
    int retval = 0;

    /* Register the cdns_uart driver with the serial core */
    retval = uart_register_driver(&cdns_uart_uart_driver);
    if (retval)
        return retval;
        ……
    return retval;
}

static void __exit cdns_uart_exit(void)
{
    /* Unregister the platform driver */
    platform_driver_unregister(&cdns_uart_platform_driver);
        ……
}
```

可以找到 uart_driver 类型的变量 cdns_uart_uart_driver，并且初始化了。

之后在驱动入口函数和出口函数中，分别有对应的注册和注销方法。

2) uart_port

```
static struct uart_port cdns_uart_port[CDNS_UART_NR_PORTS];

/**
 * cdns_uart_get_port - Configure the port from platform device resource info
 * @id: Port id
 *
 * Return: a pointer to a uart_port or NULL for failure
 */
static struct uart_port *cdns_uart_get_port(int id)
{
    struct uart_port *port;

    /* Try the given port id if failed use default method */
    if (cdns_uart_port[id].mapbase != 0) {
        /* Find the next unused port */
        for (id = 0; id < CDNS_UART_NR_PORTS; id++)
            if (cdns_uart_port[id].mapbase == 0)
                break;
    }

    if (id >= CDNS_UART_NR_PORTS)
        return NULL;

    port = &cdns_uart_port[id];

    /* At this point, we've got an empty uart_port struct, initialize it */
    spin_lock_init(&port->lock);
    port->membase    = NULL;
    port->irq    = 0;
    port->type    = PORT_UNKNOWN;
    port->iotype = UPIO_MEM32;
    port->flags  = UPF_BOOT_AUTOCONF;
    port->ops    = &cdns_uart_ops;
    port->fifosize   = CDNS_UART_FIFO_SIZE;
    port->line    = id;
    port->dev    = NULL;
    return port;
}

static int cdns_uart_probe(struct platform_device *pdev)
{
    int rc, id, irq;
    struct uart_port *port;
    struct resource *res;
```

```
45.     struct cdns_uart *cdns_uart_data;
46.     const struct of_device_id *match;
47.         ……
48.     /* Look for a serialN alias */
49.     id = of_alias_get_id(pdev->dev.of_node, "serial");
50.     if (id < 0)
51.         id = 0;
52.
53.     /* Initialize the port structure */
54.     port = cdns_uart_get_port(id);
55.
56.     if (!port) {
57.         dev_err(&pdev->dev, "Cannot get uart_port structure\n");
58.         rc = -ENODEV;
59.         goto err_out_notif_unreg;
60.     }
61.
62.     /*
63.      * Register the port.
64.      * This function also registers this device with the tty layer
65.      * and triggers invocation of the config_port() entry point.
66.      */
67.     port->mapbase = res->start;
68.     port->irq = irq;
69.     port->dev = &pdev->dev;
70.     port->uartclk = clk_get_rate(cdns_uart_data->uartclk);
71.     port->private_data = cdns_uart_data;
72.     cdns_uart_data->port = port;
73.     platform_set_drvdata(pdev, port);
74.
75.         ……
76.     rc = uart_add_one_port(&cdns_uart_uart_driver, port);
77.     if (rc) {
78.         dev_err(&pdev->dev,
79.             "uart_add_one_port() failed; err=%i\n", rc);
80.         goto err_out_pm_disable;
81.     }
82.
83.         ……
84. }
85.
```

在程序中，可以找到如上的代码片段，首先定义了 uart_port 型的数组 cdns_uart_port[CDNS_UART_NR_PORTS]。

又实现了函数 cdns_uart_get_port 来对 uart_port 实现初始化。

在 probe 函数中 73 行(实际源码中 1554 行)，把 uart_port 添加到设备驱动结构体的私有数据中，以便于在之后 ops 函数实现时调用。

同样在 probe 函数 76 行(实际源码 1561 行)，调用 uart_add_one_port 把 uart_port 添加到 uart_driver 中。

3) uart_ops

uart_ops 变量定义在 1081 行，名为 cdns_uart_ops，如下：

```
1.  static const struct uart_ops cdns_uart_ops = {
2.      .set_mctrl  = cdns_uart_set_mctrl,
3.      .get_mctrl  = cdns_uart_get_mctrl,
4.      .start_tx   = cdns_uart_start_tx,
5.      .stop_tx    = cdns_uart_stop_tx,
6.      .stop_rx    = cdns_uart_stop_rx,
7.      .tx_empty   = cdns_uart_tx_empty,
```

```
8.      .break_ctl  = cdns_uart_break_ctl,
9.      .set_termios    = cdns_uart_set_termios,
10.     .startup    = cdns_uart_startup,
11.     .shutdown   = cdns_uart_shutdown,
12.     .pm     = cdns_uart_pm,
13.     .type       = cdns_uart_type,
14.     .verify_port    = cdns_uart_verify_port,
15.     .request_port   = cdns_uart_request_port,
16.     .release_port   = cdns_uart_release_port,
17.     .config_port    = cdns_uart_config_port,
18. #ifdef CONFIG_CONSOLE_POLL
19.     .poll_get_char  = cdns_uart_poll_get_char,
20.     .poll_put_char  = cdns_uart_poll_put_char,
21. #endif
22. };
```

在 1112 行的 uart_port 初始化函数中，其中的 ops 就是赋值为 cdns_uart_ops。cdns_uart_ops 中的函数，也就是 uart 具体的驱动函数了。

第十四章 块设备驱动

Linux 设备被分为字符设备、块设备、网络设备。前面介绍的框架都是和字符设备结合起来实验的，这章换换口味来说说块设备。

14.1 块设备简介

块设备被区别于字符设备肯定是有他的特殊性。

先回顾一下字符设备，在之前的字符设备实验中，字符设备框架部分几乎可以保证通用，少有区别的部分只有 file_operations 中操作函数的实现。

既然能保证通用性，又为何要引入块设备的概念呢?

块设备较于字符设备最突出的特征就是，他需要支持批量的数据读写操作。当然多次调用字符设备的读写也可以达成最终目的，但具体到硬件操作就会遇到几个问题:

1) 一些块设备如 flash 在写之前需要先擦除当前扇区
2) 一些块设备的机械结构需要按地址顺序读写才能达到最高效率

因此块设备的驱动中应该要有这些功能:

1) 有读出扇区数据的 buffer
2) 有读写操作队列，并能按地址优化排列读写顺序

而块设备框架的引入正是应对了这些问题。

14.2 内核中的块设备访问

从应用层到硬件，块设备的访问过程如下:

1) 应用层的 app 调用 read 等方法操作文件如 txt 等
2) 经由文件系统 ext2、ext3 等，把对文件的操作转化为对扇区的读写
3) 调用块设备驱动中对应的操作函数
4) 操作硬件

与字符设备的差异在于中间多了一个文件系统。这一步中，有一个关键的函数 ll_rw_block()，从这个函数开始，就进入设备层面了。

ll_rw_block()函数，定义在文件 fs/buffer.c 中。如下:

```
1.  void ll_rw_block(int op, int op_flags,  int nr, struct buffer_head *bhs[]
    )
2.  {
3.      int i;
4.
5.      for (i = 0; i < nr; i++) {
6.          struct buffer_head *bh = bhs[i];
7.
8.          if (!trylock_buffer(bh))
9.              continue;
10.         if (op == WRITE) {
11.             if (test_clear_buffer_dirty(bh)) {
```

```
12.                 bh->b_end_io = end_buffer_write_sync;
13.                 get_bh(bh);
14.                 submit_bh(op, op_flags, bh);
15.                 continue;
16.             }
17.         } else {
18.             if (!buffer_uptodate(bh)) {
19.                 bh->b_end_io = end_buffer_read_sync;
20.                 get_bh(bh);
21.                 submit_bh(op, op_flags, bh);
22.                 continue;
23.             }
24.         }
25.         unlock_buffer(bh);
26.     }
27. }
```

输入参数中的 buffer_head 结构体是缓冲区的描述符，存放缓冲区信息，定义如下：

```
1.  struct buffer_head {
2.      unsigned long b_state;              //缓冲区状态
3.      struct buffer_head *b_this_page;    //页缓冲区头
4.      struct page *b_page;                //存储缓冲区所在页
5.      sector_t b_blocknr;                 //块号
6.      size_t b_size;                      //块大小
7.      char *b_data;                       //页数据缓冲区
8.
9.      struct block_device *b_bdev;        //块设备
10.
11.     bh_end_io_t *b_end_io;              //I/O 完成后执行函数指针
12.
13.     void *b_private;                    //b_end_io 的输入参数
14.
15.     struct list_head b_assoc_buffers;   //映射链表
16.
17.     /* mapping this buffer is associated with */
18.     struct address_space *b_assoc_map;
19.     atomic_t b_count;                   //缓冲区计数
20. };
```

再看 ll_rw_block()函数中 14 和 21 行最终做的操作 submit_bh()函数。submit_bh()是对函数 submit_bh_wbc()的简化封装，submit_bh_wbc()定义如下(省略原有注释)：

```
1.  int submit_bh(int rw, struct buffer_head * bh)
2.  {
3.      struct bio *bio;                                    //定义一个
    bio(block input output),也就是块设备 i/o
4.      BUG_ON(!buffer_locked(bh));
5.      BUG_ON(!buffer_mapped(bh));
6.      BUG_ON(!bh->b_end_io);
7.      BUG_ON(buffer_delay(bh));
8.      BUG_ON(buffer_unwritten(bh));
9.
10.     if (test_set_buffer_req(bh) && (op == REQ_OP_WRITE))
11.         clear_buffer_write_io_error(bh);
```

```
12.
13.     bio = bio_alloc(GFP_NOIO, 1);                          //分配 bio
14.
15.     if (wbc) {
16.         wbc_init_bio(wbc, bio);
17.         wbc_account_io(wbc, bh->b_page, bh->b_size);
18.     }
19.
20.     /* 构造 bio */
21.     bio->bi_sector = bh->b_blocknr * (bh->b_size >> 9); //存放逻辑块号
22.     bio->bi_bdev = bh->b_bdev;                             //存放对应的块设
    备
23.
24.     bio_add_page(bio, bh->b_page, bh->b_size, bh_offset(bh));
25.     BUG_ON(bio->bi_iter.bi_size != bh->b_size);
26.
27.     bio->bi_end_io = end_bio_bh_io_sync;                   //设置 i/o 回调函
    数
28.     bio->bi_private = bh;                                  //回调函数参
    数，指向哪个缓冲区
29.     bio->bi_flags |= bio_flags;
30.
31.     guard_bio_eod(op, bio);
32.
33.     if (buffer_meta(bh))
34.         op_flags |= REQ_META;
35.     if (buffer_prio(bh))
36.         op_flags |= REQ_PRIO;
37.     bio_set_op_attrs(bio, op, op_flags);
38.
39.     submit_bio(bio);                                       //提交 bio
40.     return 0;
41. }
```

这里的工作主要就是使用 buffer_head 来构造一个 bio，bio 即 block input/output，块设备输入输出。最终调用 submit_bio()函数提交 bio 到内核队列。

再往下看，submit_bio()函数最终调用了 generic_make_request()，再往下调用__generic_make_request()，再到__make_request()，在__make_request()函数中的 elv_merge()函数是内核中的电梯算法(elevator merge)，所谓的电梯算法，也就是实现了我们上面说的按地址顺序优化读写顺序。

我们担心的事情内核已经帮我们完成了，而块设备框架，则在此基础上展开。

14.3 块设备框架

字符设备框架可以参考内核中先用的字符设备驱动代码 drivers\block\z2ram.c 或其他。

大致步骤如下：

1) 分配一个 gendisk 结构体
2) 分配一个队列 request_queue_t
3) 设置 gendisk 结构体的成员

4) 注册 gendisk 结构体

分别来看。

14.3.1 gendisk 结构体

gendisk 结构体定义在 genhd.h 中，如下：

```
struct gendisk {
    /* major, first_minor and minors are input parameters only,
     * don't use directly.  Use disk_devt() and disk_max_parts().
     */
    int major;          /* major number of driver */
    int first_minor;
    int minors;                     /* maximum number of minors, =1 for
                                     * disks that can't be partitioned. */

    char disk_name[DISK_NAME_LEN];  /* name of major driver */
    char *(*devnode)(struct gendisk *gd, umode_t *mode);

    unsigned int events;        /* supported events */
    unsigned int async_events;  /* async events, subset of all */

    /* Array of pointers to partitions indexed by partno.
     * Protected with matching bdev lock but stat and other
     * non-critical accesses use RCU.  Always access through
     * helpers.
     */
    struct disk_part_tbl __rcu *part_tbl;
    struct hd_struct part0;

    const struct block_device_operations *fops;
    struct request_queue *queue;
    void *private_data;

    int flags;
    struct kobject *slave_dir;

    struct timer_rand_state *random;
    atomic_t sync_io;       /* RAID */
    struct disk_events *ev;
#ifdef  CONFIG_BLK_DEV_INTEGRITY
    struct kobject integrity_kobj;
#endif  /* CONFIG_BLK_DEV_INTEGRITY */
    int node_id;
    struct badblocks *bb;
};
```

部分成员介绍：

1. major 为设备主设备号。
2. first_minor 起始次设备号。
3. minors 次设备号的数量，也称为分区数量，值为 1 表示无法分区。
4. disk_name 设备名称。
5. part 分区表的信息。
6. fops 块设备操作集合，块设备的操作函数与字符设备差别较大，碰到了再去研究。

7. queue 申请队列，用于管理该设备 IO 申请队列的指针。
8. private_data 私有数据。
9. capacity 扇区数。

gendisk 的操作大概如下：

1) 在块设备驱动程序中，首先定义一个 gendisk 结构体变量，在入口函数中使用函数 struct gendisk *alloc_disk(int minors)分配空间，minors 是分区的数量。
2) 分配队列 queue。
3) 使用 register_blkdev 函数向内核申请块设备的主设备号。
4) 使用函数 set_capacity()设置扇区数。
5) 设置其他参数。
6) 使用函数 add_disk()向内核注册 gendisk。
7) 在出口函数中实现相反的操作：

使用 put_disk()和 del_gendisk()来注销,释放 gendisk 结构体。

使用 kfree()释放磁盘扇区缓存。

使用 blk_cleanup_queue()清除内存中的申请队列。

使用 unregister_blkdev()卸载块设备。

14.3.2 request_queue

首先定义一个 request_queue 结构体变量。使用下面函数来分配：

```
struct request_queue *blk_init_queue(request_fn_proc *rfn, spinlock_t *lock)
```

rfn 是队列处理函数。

lock 是自旋锁。

使用 blk_init_queue 函数获取到 request_queue 之后复制给 ramblock_disk.queue 即可。

这里 rfn 函数的实现，也就是块设备中的重点了。

块设备框架的使用，到实验例程中去讲解。

14.4 实验

本章实验把内存模拟成块设备来操作，重在熟悉块设备的框架和块设备驱动编写流程。

14.1.1 设备树

```
/include/ "system-conf.dtsi"
/ {
};

/* SD */
&sdhci1 {
    disable-wp;
    no-1-8-v;
};

/* USB  */
&dwc3_0 {
    status = "okay";
    dr_mode = "host";
};
```

14.1.2 驱动程序

```
#include <linux/major.h>
#include <linux/vmalloc.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/blk-mq.h>
#include <linux/bitops.h>
#include <linux/mutex.h>
#include <linux/slab.h>

#define AX_BLOCK_SIZE         (1024*64)
#define SECTORS_NUM           AX_BLOCK_SIZE / 512

#define AX_BLOCK_MAJOR        (40)
#define Z2MINOR_COUNT         (8)

#define DEVICE_NAME           "ax_block"

static DEFINE_SPINLOCK(ax_block_lock);

struct ax_block{
    struct gendisk        *z2ram_gendisk;
    struct request_queue  *z2_queue;
    struct blk_mq_tag_set tag_set;
    unsigned char         *block_buf;
};

static struct ax_block ax_block_drv;


static blk_status_t z2_queue_rq(struct blk_mq_hw_ctx *hctx, const struct blk_mq_queue_data *bd)

{
    struct request *req = bd->rq;
    unsigned long start = (unsigned long)((blk_rq_pos(req) << 9) + ax_block_drv.block_buf);
    unsigned long len = blk_rq_cur_bytes(req);
    void *buffer = bio_data(req->bio);

    blk_mq_start_request(req);
    spin_lock_irq(&ax_block_lock);

    if(rq_data_dir(req) == READ)
    {
        printk("ax_request read\n");
        memcpy(buffer, (char *)start, len);
    }
    else
    {
        printk("ax_request write\n");
        memcpy((char *)start, buffer, len);
    }

    spin_unlock_irq(&ax_block_lock);
    blk_mq_end_request(req, BLK_STS_OK);

    return BLK_STS_OK;
}


static const struct blk_mq_ops z2_mq_ops = {
    .queue_rq    = z2_queue_rq,
};

static const struct block_device_operations z2_fops =
{
    .owner      = THIS_MODULE,
};

static struct kobject *z2_find(dev_t dev, int *part, void *data)
{
    *part = 0;
    return get_disk_and_module(ax_block_drv.z2ram_gendisk);
}

static int __init ax_block_init(void)
{
```

```
75.     register_blkdev(AX_BLOCK_MAJOR, DEVICE_NAME);
76.
77.     ax_block_drv.z2ram_gendisk = alloc_disk(1);
78.     ax_block_drv.z2_queue = blk_mq_init_sq_queue(&ax_block_drv.tag_set, &z2_mq_ops, 16, BLK_MQ_F
    _SHOULD_MERGE);
79.
80.     ax_block_drv.z2ram_gendisk->major = AX_BLOCK_MAJOR;
81.     ax_block_drv.z2ram_gendisk->first_minor = 0;
82.     ax_block_drv.z2ram_gendisk->fops = &z2_fops;
83.     sprintf(ax_block_drv.z2ram_gendisk->disk_name, "ax_block");
84.
85.     ax_block_drv.z2ram_gendisk->queue = ax_block_drv.z2_queue;
86.
87.     set_capacity(ax_block_drv.z2ram_gendisk, SECTORS_NUM);
88.     ax_block_drv.block_buf = kzalloc(AX_BLOCK_SIZE, GFP_KERNEL);
89.
90.     add_disk(ax_block_drv.z2ram_gendisk);
91.     blk_register_region(MKDEV(AX_BLOCK_MAJOR, 0), Z2MINOR_COUNT, THIS_MODULE, z2_find, NULL, NUL
    L);
92.
93.     return 0;
94. }
95.
96. static void __exit ax_block_exit(void)
97. {
98.     blk_unregister_region(MKDEV(AX_BLOCK_MAJOR, 0), Z2MINOR_COUNT);
99.     unregister_blkdev(AX_BLOCK_MAJOR, DEVICE_NAME);
100.    del_gendisk(ax_block_drv.z2ram_gendisk);
101.    put_disk(ax_block_drv.z2ram_gendisk);
102.    blk_cleanup_queue(ax_block_drv.z2_queue);
103.    blk_mq_free_tag_set(&ax_block_drv.tag_set);
104.
105.    return;
106. }
107.
108. module_init(ax_block_init);
109. module_exit(ax_block_exit);
110. MODULE_LICENSE("GPL");
```

先看块设备框架，入口函数出口函数以及操作函数三个部分的结构和字符设备框架无异。

77 行先使用函数 alloc_disk 分配块设备。

78 行使用 blk_init_queue 函数分配申请队列，并提供队列处理函数 z2_queue_rq。

85 行把分配的队列设置到块设备结构体成员中。

87 行设置扇区数，95 行使用 kalloc 函数获取缓存来模拟块设备。

90 行使用 add_disk 函数向内核注册 gendisk 结构体。

驱动出口函数中做相反的注销操作。

然后回头看 30 行队列处理函数 z2_queue_rq 中的内容。

可以看到最终的处理就是 memcpy，把读的或者写的内容拷贝到 buffer 或者内存地址中。当然这里是因为使用内存所以可以直接使用 memcpy 方法，实际使用外设块设备，读写方法会复杂一些。

在 memcpy 的前一步，就是判断本次操作是读还是写，调用函数 rq_data_dir 函数，如果申请是读，就会返回 READ，函数参数是 struct request 类型的指针。注意我们的队列处理函数，输入参数是 struct request_queue 类型的指针，是申请的队列，使用函数 blk_fetch_request 函数可以从申请队列中获取队列最前的申请。获取到的申请，就是我们后面处理的对象。memcpy 的关键参数有三，数据来源，数据目标，和数据长度。这些都要通过申请结构体变量指针 struct request *req 来获取。通过 bio_data(req->bio)获取数据目标地址。

使用 blk_rq_pos(req)获取当前扇区，然后乘上扇区大小，再加上缓存的首地址，则是当前申请数据所在的首地址了。

使用方法 blk_rq_cur_bytes(req)获取当前申请的数据长度。在队列处理函数中，需要轮询队列中的每一个申请，方法就是使用函数 blk_fetch_request(q)，如果有申请，则返回正值。最后处理完了使用 blk_end_request_cur(req, 0)函数把处理完成的申请移出队列。

另外在读和写之前加上 printk 来查看看效果。

14.1.3 运行测试

加载驱动之后，在/dev 路径中可以找到名为“ax_block”的设备描述文件。

执行下面的命令

```
mkfs.vfat   /dev/ax_block
cd ~
mkdir   block_test
mount   /dev/ax_block   ./block_test
```

先给块设备操作文件设置分区，然后新建一个文件夹，把块设备文件挂载到文件夹上。然后就可以正常使用块设备了。

```
root@petalinux:/mnt# insmod ax-block.ko
[   43.979493] ax_block: loading out-of-tree module taints kernel.
root@petalinux:/mnt# mkfs.vfat /dev/ax_block
[   79.735398] ax_request read
[   79.738274] ax_request read
[   79.741239] ax_request write
root@petalinux:/mnt# cd
root@petalinux:~# mkdir ./block_test/
root@petalinux:~# mount /dev/ax_block ./block_test/
[  146.166786] ax_request read
[  146.169893] ax_request read
[  146.172843] ax_request read
[  146.175818] ax_request read
[  146.178644] ax_request read
[  146.181472] ax_request read
[  146.184273] ax_request read
[  146.187276] ax_request read
[  146.190128] ax_request read
[  146.192949] ax_request read
[  146.195776] ax_request read
[  146.198600] ax_request write
root@petalinux:~# cd ./block_test/
root@petalinux:~/block_test# touch test.txt
root@petalinux:~/block_test# vi test.txt [  176.917502] ax_request write

root@petalinux:~/block_test# cat test.txt
love fpga
root@petalinux:~/block_test#
```

第十五章 网卡驱动

15.1 Linux 网卡驱动

注意这里说的是网卡驱动不是网络驱动。网络驱动是相当复杂的。我们知道 OSI 七层网络模型从上往下为：应用层、表示层、会话层、传输层、网络层、数据链路层、物理层，而 Linux 中使用 TCP/IP 四层模型为：应用层、传输层、网际层、网络接口层。

OSI	Linux TCP/IP
应用层	应用层
表示层	
会话层	
传输层	传输层
网络层	网际层
数据链路层	网络接口层
物理层	

网卡驱动是被包含于网络驱动中的一环，属于网络接口层，包括数据链路层和物理层。也是网络驱动中唯一与硬件相关的部分，其他的都是纯软件的概念，对于底层驱动开发而言，只需要关心硬件相关的内容，因此这里我们来学习网卡驱动。

网卡驱动需要实现两个部分，一是和上层交互数据收发，二是结合具体硬件处理数据。网络接口层中又可以细分四层：网络协议接口层、网络设备接口层、设备驱动功能层、网络设备与媒介层。网络协议接口层负责对上层发送或接收数据，设备驱动层负责对硬件发送或接收数据，网络设备接口层则在中间起到数据承接的作用，网络设备层则负责实际的数据收发操作。

这里的关键就是网络设备接口层，Linux 使用 net_device 结构体来描述一个网络设备信息，并对不同得硬件做了概括和统一。

从功能得角度看，网卡设备需要实现的功能有三，初始化、发送和接受，我们以 net_device 结构体为切入点，以三个需要实现的功能为引来看 Linux 网卡驱动程序的实现方法。

15.1.1 初始化

net_device 结构体表示一个网卡，网卡的初始化，实际也就是定义一个 net_device 结构体变量，并初始化这个变量。先看一下 net_device 结构体的具体定义，在文件 include/linux/netdevice.h 中，如下(有省略和更改)：

```
1.  struct net_device {
2.      char            name[IFNAMSIZ]; //网卡设备名称
3.      struct hlist_node   name_hlist;
4.      char            *ifalias;
5.
6.      unsigned long       mem_end;    //该设备的内存结束地址
7.      unsigned long       mem_start;  //该设备的内存起始地址
8.      unsigned long       base_addr;  //该设备的内存 I/O 基地址
```

```
    int         irq;                //该设备的中断号

    atomic_t        carrier_changes;

    unsigned long       state;      //网络设备和网络适配器的状态信息

    ……

    struct net_device_stats stats;  //用来保存统计信息的 net_device_stats 结构体

    ……

    const struct net_device_ops *netdev_ops;
                                    //网络设备操作函数
    ……

    unsigned int        flags;      //flags 指网络接口标志,常见取值如下
                                    //IFF_UP:设备被激活并可以开始发送数据包时设置该
标志
                                    //IFF_AUTOMEDIA:设置设备可在多种媒介间切换
                                    //IFF_BROADCAST:允许广播
                                    //IFF_DEBUG:调试模式
                                    //IFF_LOOPBACK:回环
                                    //IFF_NOARP:无可执行 ARP,点对点接口就不需要运行
ARP
                                    //IFF_POINTOPOINT:点到点链路
    ……

    unsigned int        mtu;        //最大数据包
    unsigned short      type;       //接口的硬件类型
    unsigned short      hard_header_len;
                                    //硬件帧头长度,一般被赋为 ETH_HLEN 值为 14

    ……

    unsigned long       last_rx;    //接收数据包的时间戳,调用 netif_rx()后赋上
jiffies 即可
    unsigned char       *dev_addr;  //MAC 地址

    ……
};
```

除了一些基本信息名称、地址之外，最为关键的成员就是 struct net_device_ops 类型的指针变量 netdev_ops，他包含了网络设备的操作函数集我们需要实现的发包函数也包含在其中，看一下他的具体定义，也在这个文件中，如下(有省略)：

```
struct net_device_ops {
    ……
    int         (*ndo_open)(struct net_device *dev);
    int         (*ndo_stop)(struct net_device *dev);
    netdev_tx_t     (*ndo_start_xmit)(struct sk_buff *skb,
                          struct net_device *dev);
    ……
};
```

ndo_open 用于开启网络设备，ndo_stop 用于关闭网络设备。

ndo_start_xmit 函数就是关键的发包函数了。

再看一下 net_device 结构体中的 struct net_device_stats stats 成员，net_device_stats 结构体定义如下：

```
1.  struct net_device_stats {
2.      unsigned long   rx_packets;
3.      unsigned long   tx_packets;
4.      unsigned long   rx_bytes;
5.      unsigned long   tx_bytes;
6.      unsigned long   rx_errors;
7.      unsigned long   tx_errors;
8.      unsigned long   rx_dropped;
9.      unsigned long   tx_dropped;
10.     unsigned long   multicast;
11.     unsigned long   collisions;
12.     unsigned long   rx_length_errors;
13.     unsigned long   rx_over_errors;
14.     unsigned long   rx_crc_errors;
15.     unsigned long   rx_frame_errors;
16.     unsigned long   rx_fifo_errors;
17.     unsigned long   rx_missed_errors;
18.     unsigned long   tx_aborted_errors;
19.     unsigned long   tx_carrier_errors;
20.     unsigned long   tx_fifo_errors;
21.     unsigned long   tx_heartbeat_errors;
22.     unsigned long   tx_window_errors;
23.     unsigned long   rx_compressed;
24.     unsigned long   tx_compressed;
25. };
```

实际就是网络状态的统计信息合集，我们会在收发处理函数中去操作这个 stats 来记录状态。

可见 net_device 结构体变量是网卡设备的关键，网卡设备的初始化也就是 net_device 结构体变量的初始化，步骤如下：

1) 定义一个 net_device 结构体变量
2) 使用函数 alloc_netdev()分配一个 net_device 结构体变量
3) 初始化硬件寄存器
4) 设置 net_device 结构体变量成员变量
5) 使用 register_netdev()注册 net_device 结构体变量

alloc_netdev()是个宏定义，具体如下：

```
alloc_netdev(sizeof_priv, name, name_assign_type, setup)
```

第一个参数 sizeof_priv 为私有变量的大小。

第二个参数 name 为设备名。

第三个参数 name_assign_type 为设备来源名称。

第四个参数 setup 为 net_device 的 setup()函数指针，setup()函数接收的参数为 struct net_device 指针，用于预置 net_device 成员的值。

register_netdev()只有一个输入参数就是需要注册的 net_device 结构体变量指针。

15.1.2 发包

发包函数 ndo_start_xmit 需要我们自己构建，原型如下：

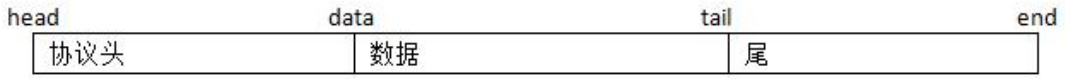
```
netdev_tx_t (*ndo_start_xmit)(struct sk_buff *skb, struct net_device *dev);
```

第一个输入参数 struct sk_buff 是指 socket(套接字)缓冲区，用于网络模型中各层之间的数据传递。struct sk_buff 定义在文件 include/linux/skbuff.h 中，是一个双向链表，如下(有省略)：

```
1.  struct sk_buff {
2.      union {
3.          struct {
```

```
4.              /* These two members must be first. */
5.              struct sk_buff      *next;  //指向下一个 sk_buff 结构体
6.              struct sk_buff      *prev;  //指向前一个 sk_buff 结构体
7.              ……
8.          };
9.          ……
10.     };
11.
12.     ……
13.
14.     unsigned int        len,    //数据包的总长度,包括线性数据和非线性数据
15.                 data_len;       //非线性的数据长度
16.     __u16           mac_len,    //mac 包头长度
17.                 hdr_len;
18.
19.     ……
20.
21.     __u32           priority;   //当前 sk_buff 结构体的优先级
22.
23.     ……
24.
25.     __be16          protocol;   //存放上层的协议类型,可以通过 eth_type_trans()来获
    取
26.     __u16           transport_header;   //传输层头部的偏移值
27.     __u16           network_header;     //网络层头部的偏移值
28.     __u16           mac_header;         //MAC 数据链路层头部的偏移值
29.
30.     ……
31.
32.     /* public: */
33.
34.     /* These elements must be at the end, see alloc_skb() for details.  */
35.     sk_buff_data_t      tail;           //指向缓冲区的数据包末尾
36.     sk_buff_data_t      end;            //指向缓冲区的末尾
37.     unsigned char       *head,          //指向缓冲区的协议头开始位置
38.                 *data;                  //指向缓冲区的数据包开始位置
39.     ……
40. };
```

sk_buff 中个数据段排布空间如下：



数据段中又分为几段：

MAC 头	IP 头	数据类型	数据

MAC 头用结构体 ethhdr 表示。

IP 头用结构体 iphdr 表示。

数据类型为一个字节。

发包函数 ndo_start_xmit 中具体要做的是如下：

1) 调用函数 netif_stop_queue()停止上层下传数据。
2) 通过硬件发送数据。
3) 调用函数 dev_kfree_skb()释放 sk_buff。
4) 发送成功进入中断时，更新统计信息，调用 netif_wake_queue()函数，使上层继续下发数据。

5) 发送超时则在 net_device_ops 中的 ndo_tx_timeout 函数中调用 netif_wake_queue()函数，使上层继续下发数据。

dev_kfree_skb()函数原型如下：

```
#define dev_kfree_skb(a)      consume_skb(a)
void consume_skb(struct sk_buff *skb)
```

netif_wake_queue()函数原型如下：

```
static inline void netif_wake_queue(struct net_device *dev)
```

netif_stop_queue()函数原型如下：

```
static inline void netif_stop_queue(struct net_device *dev)
```

15.1.3 收包

收包一般放在中断中处理。处理过程为：

1) 使用 alloc_skb()函数构造一个 sk_buff。
2) 使用 skb_reserve(sk_buff,2)把 sk_buff 里的数据包先后位移 2 字节以腾出 sk_buff 里的头部空间。
3) 读取网络设备硬件上接收到的数据，把数据复制到 sk_buff 的成员 data 指针。
4) 使用 eth_type_trans()函数来获取上层协议，将返回值赋给 sk_buff 的成员 protocol。
5) 更新统计信息，最后使用 netif_rx()将 sk_buff 上传到上层协议。

15.2 实验

15.1.1 驱动程序

```
1.  #include <linux/module.h>
2.  #include <linux/kernel.h>
3.  #include <linux/types.h>
4.  #include <linux/fcntl.h>
5.  #include <linux/ioport.h>
6.  #include <linux/in.h>
7.  #include <linux/skbuff.h>
8.  #include <linux/string.h>
9.  #include <linux/init.h>
10. #include <linux/bitops.h>
11. #include <linux/ip.h>
12. #include <linux/netdevice.h>
13. #include <linux/etherdevice.h>
14. #include <asm/io.h>
15. #include <asm/irq.h>
16.
17. /* 定义一个 net_device 结构体变量 */
18. static struct net_device *ax_net_dev;
19.
20. /* 模拟接收，构造一个虚拟的 sk_buff 上报，并更新统计信息 */
21. static void ax_net_rx(struct sk_buff *skb, struct net_device *dev)
22. {
23.     struct iphdr *ih;
24.     __be32 *saddr, *daddr, tmp;
25.     unsigned char   tmp_dev_addr[ETH_ALEN];
26.     struct ethhdr *ethhdr;
27.     struct sk_buff *rx_skb;
28.
29.     ethhdr = (struct ethhdr *)skb->data;
30.
31.     memcpy(tmp_dev_addr, ethhdr->h_dest, ETH_ALEN);
32.     memcpy(ethhdr->h_dest, ethhdr->h_source, ETH_ALEN);
33.     memcpy(ethhdr->h_source, tmp_dev_addr, ETH_ALEN);
34.
35.     ih = (struct iphdr *)(skb->data + sizeof(struct ethhdr));
36.     saddr = &ih->saddr;
```

```
    daddr = &ih->daddr;

    tmp = *saddr;
    *saddr = *daddr;
    *daddr = tmp;

    rx_skb = dev_alloc_skb(skb->len + 2);
    skb_reserve(rx_skb, 2);
    memcpy(skb_put(rx_skb, skb->len), skb->data, skb->len);

    rx_skb->dev = dev;
    rx_skb->protocol = eth_type_trans(rx_skb, dev);
    rx_skb->ip_summed = CHECKSUM_UNNECESSARY;
    dev->stats.rx_packets++;
    dev->stats.rx_bytes += skb->len;

    netif_rx(rx_skb);
}

static netdev_tx_t ax_net_tx(struct sk_buff *skb, struct net_device *dev)
{
    /* 停止上层数据下传队列 */
    netif_stop_queue(dev);
    /* 模拟接收，以达到一个完成的发送接收过程 */
    ax_net_rx(skb, dev);
    /* 释放 skb */
    dev_kfree_skb (skb);
    /* 发送完成，恢复上层数据下传队列 */
    netif_wake_queue(dev);
    /* 更新统计信息 */
    dev->stats.tx_packets++;
    dev->stats.tx_bytes += skb->len;

    return NETDEV_TX_OK;
}

/* 网卡设备操作函数集 */
static const struct net_device_ops ax_netdev_ops =
{
    .ndo_start_xmit = ax_net_tx,
};

/* 驱动入口函数 */
static int __init ax_net_init(void)
{
    /* 分配 net_device 结构体 */
    ax_net_dev = alloc_netdev(0, "ax_net%d", NET_NAME_UNKNOWN, ether_setup);
    /* 设置操作函数集 */
    ax_net_dev->netdev_ops = &ax_netdev_ops;

    /* 设置 MAC 地址 */
    ax_net_dev->dev_addr[0] = 0x0A;
    ax_net_dev->dev_addr[1] = 0x0B;
    ax_net_dev->dev_addr[2] = 0x0C;
    ax_net_dev->dev_addr[3] = 0x0D;
    ax_net_dev->dev_addr[4] = 0x0E;
    ax_net_dev->dev_addr[5] = 0x0F;
    /* 设置 ping 功能 */
    ax_net_dev->flags     |= IFF_NOARP;
    ax_net_dev->features  |= NETIF_F_CSUM_MASK;
    /* 注册网卡驱动 */
    register_netdev(ax_net_dev);

    return 0;
}

/* 驱动出口函数 */
static void __exit ax_net_exit(void)
{
    unregister_netdev(ax_net_dev);
    free_netdev(ax_net_dev);
}

module_init(ax_net_init);
module_exit(ax_net_exit);

/* 驱动描述信息 */
```

```
114. MODULE_AUTHOR("Alinx");
115. MODULE_ALIAS("net card test");
116. MODULE_DESCRIPTION("NET CARD driver");
117. MODULE_VERSION("v1.0");
118. MODULE_LICENSE("GPL");
```

驱动入口函数中，先使用 alloc_netdev()来分配一个 net_device 结构体，设置 net_device 结构体变量的成员，设置完成后使用 register_netdev()向内核注册 net_device 结构体。设置 net_device 结构体变量成员主要是操作函数和 109 行和 110 行设置 ping 功能。

发包函数中，先调用 netif_stop_queue(dev)函数停止上层发送数据包，之后如果是真实的网卡，应该要调用硬件接口来发送数据因为这里是模拟网卡，所以直接调用模拟接受的函数表示发送完成。接受函数 ax_net_rx()之后再看。发送完成后释放 skb_buff 并调用 netif_wake_queue()函数恢复上层发送数据包。最后更新统计信息。

ax_net_rx 函数用来模拟接收，本是应该放在接收中断中处理的。在接收函数中，修改 sk_buff 中的数据，使它变成一个接收 sk_buff，计提方法就是对调 erhhdr 和 iphdr 结构体中发送、接收 mac 地址。使用 ip_fast_csum 函数重新获取 iphdr 校验。之后设置 flag，0x08 表示发送 ping 包，接收需要修改为 0。重新构造 sk_buff，重新赋值。使用 eth_type_trans 函数获取上层协议。最后调用 netif_rx 提交 sk_buff。

15.1.2 运行测试

加载驱动后，使用 ls /sys/class/net/命令查看网卡驱动有没有存在。

再使用命令 ifconfig ax_net0 3.3.3.3 设置 ip，之后就可以 ping 测试了。

之后使用 ifconfig 命令查看网卡信息，核对 mac 地址信息和收发包数的统计信息。

```
root@petalinux:/mnt# insmod ax-net.ko
[   49.838465] ax_net0: mixed HW and IP checksum settings.
root@petalinux:/mnt# cd
root@petalinux:~# ifconfig ax_net0 3.3.3.3
root@petalinux:~# ping 3.3.3.4
PING 3.3.3.4 (3.3.3.4): 56 data bytes
64 bytes from 3.3.3.4: seq=0 ttl=64 time=0.090 ms
64 bytes from 3.3.3.4: seq=1 ttl=64 time=0.049 ms
64 bytes from 3.3.3.4: seq=2 ttl=64 time=0.042 ms
64 bytes from 3.3.3.4: seq=3 ttl=64 time=0.040 ms
^C
--- 3.3.3.4 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.040/0.055/0.090 ms
```

```
root@petalinux:~# ifconfig
ax_net0   Link encap:Ethernet  HWaddr 0A:0B:0C:0D:0E:0F
          inet addr:3.3.3.3  Bcast:3.255.255.255  Mask:255.0.0.0
          inet6 addr: fe80::80b:cff:fe0d:e0f/64 Scope:Link
          UP BROADCAST RUNNING NOARP MULTICAST  MTU:1500  Metric:1
          RX packets:18 errors:0 dropped:0 overruns:0 frame:0
          TX packets:18 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2090 (2.0 KiB)  TX bytes:2090 (2.0 KiB)
```

第十六章 DMA 驱动

16.1 DMA 简介

CPU 和内存是计算机组成中都是不可或缺的部分。和 CPU 相比，内存是非常慢速的拖油瓶，CPU 的速度被内存限制，同时又不得不等待内存处理而无法去处理其他事件。所以使用 CPU 搬运内存数据，是非常浪费资源的。因此专门用于搬运内存数据的器件 DMA 应运而生。

DMA 是 Direct Memory Access 的缩写，也就是直接内存读写，所谓的直接，也就是内存到内存，不通过 CPU。DMA 的可以支持内存到外设、外设到内存、内存到内存的数据交互，必要时节省很多 CPU 资源。当然，虽然 DMA 几乎不占用 CPU，但还是会占用系统总线的。

16.2 DMAC

这里的 DMAC 指的是 DMA controller 即 DMA 控制器。一个外设支持 DMA 不代表它能够主动发起 DMA 操作只能说至少他支持被动 DMA 数据读写。前面我们说 DMA 几乎不占用 CPU 资源，是因为 DMA 传输需要有 CPU 发起，之后就全部交由 DMAC 搬运数据了。DMAC 搬运过程中，涉及到以下关键概念。

16.2.1DMA channels

大部分 DMAC 支持多路 DMA，也就是 DMA channels。DMA channel 相当于是 DMAC 提供的数据传输通道也就是数据传输的方法，解决了如何传输的问题。实际上 DMA channel 是一个抽象的概念，不是物理通道。前面我们说过，DMA 传输还是会占用总线资源的，总线资源十分有限，因此同时进行多路 DMA 传输不太现实。

抽象出 DMA channel 概念的目的实际是为了方便管理 DMA 的 client，让每个 client 都独占一格通道，但实际上传输时 DMAC 会进行先后仲裁，串行传输。

16.2.2 DMA request line

request line 指代 DMA 设备和 DMAC 之间的物理连线。这条线用于 DMA 设备通知 DMAC 是否可以开始传输，即解决了何时传输的问题。通常每个 DMA 数据收发节点(endpoint)都有 DMA request line 连接到 DMAC。

从主从的角度看，DMA channel 由 DMAC 提供，相当于主 cotroller，DMA request line 和 DMA 设备绑定相当于从 client。在系统中 DMA request line 的数量通常比 DMA channel 多，因为 DMA request line 不是每一刻都处于活跃状态。

16.2.3 transfer size、transfer wide、burst size

transfer wide 可以理解为单次传输数据的大小，和串口来比较的话，串口一次只能传一个字节，而 DMA 则可以选择一次能传输的数据大小。在这基础上的 transfer size 则是传输的次数，而不是单纯的总大小，也就是说 DMA 传输总长度实际上是 transfer size 乘上 transfer wide。

burst size 是指 DMAC 内部缓存大小。当 DMA 传输的源或目的是内存 memory 时，DMAC 会先读取数据到缓存，再传入或传出。

16.2.4 scatter-gather

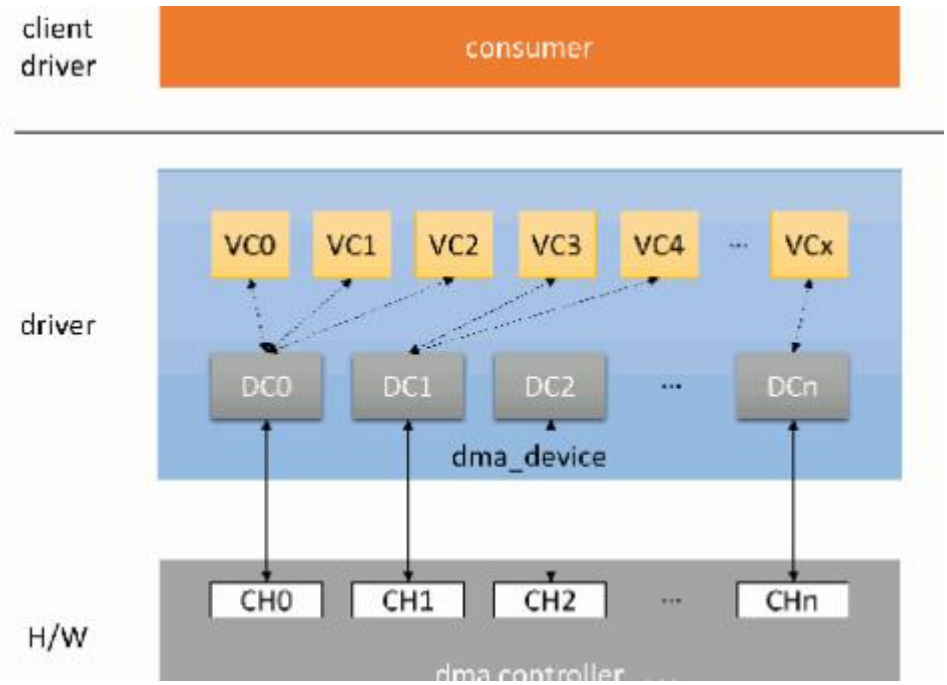
DMA 操作必须是连续的物理内存，在实际应用中，难免会遇到处理物理内存不连续的数据

的情况，scatter-gather 指的就是把不连续的数据拷贝到连续的 buffer 中的操作。这个操作过程可以用软件实现，也有直接的硬件支持。具体的实现暂且不用关心，这里主要是强调 DMA 操作必须是连续的物理内存这件事。

16.3 linux 中的 DMA 框架

Linux DMA engine 框架提供了 DMA controller 和 DMA client 两个框架。分别对应 DMA 提供者和 DMA 使用者两个角度。pl330 是个 DMA 控制器，实际上就是站 DMA 提供者的角度。使用 DMA 的对象实际可以具体到内存到内存，内存就是 DMA 的使用者。DMA 控制器相关的操作都可以抽出来，他们对于其他使用者来说，都是共通的。这也是 linux 系统一贯的设计思路。也是因为如此，原本并不复杂 DMA 在这样的设计下显得有些复杂。

DMA engine 框架示意图：



16.4.1 DMA controller 驱动框架

控制器的驱动也有现成的可用，这和前面的 I2C、SPI 之类的相似。我们主要还是去了解 client 的驱动，controller 驱动稍作了解。

DMA controller 框架抽象出 channel 对应 DMAC 的物理通道，又定义了虚拟的 channel，软件上可以实现多个虚拟 channel 对应一个物理通道。

介绍一下 DMA controller 框架中主要涉及到的数据结构：

1) struct dma_device

struct dma_device 定义在 include/linux/dmaengine.h 中。如下

```
1. struct dma_device {
2.
3.     unsigned int chancnt;
4.     unsigned int privatecnt;
5.     struct list_head channels;
6.     struct list_head global_node;
```

```
    struct dma_filter filter;
    dma_cap_mask_t  cap_mask;
    unsigned short max_xor;
    unsigned short max_pq;
    enum dmaengine_alignment copy_align;
    enum dmaengine_alignment xor_align;
    enum dmaengine_alignment pq_align;
    enum dmaengine_alignment fill_align;
    #define DMA_HAS_PQ_CONTINUE (1 << 15)

    int dev_id;
    struct device *dev;

    u32 src_addr_widths;
    u32 dst_addr_widths;
    u32 directions;
    u32 max_burst;
    bool descriptor_reuse;
    enum dma_residue_granularity residue_granularity;

    int (*device_alloc_chan_resources)(struct dma_chan *chan);
    void (*device_free_chan_resources)(struct dma_chan *chan);

    struct dma_async_tx_descriptor *(*device_prep_dma_memcpy)(
        struct dma_chan *chan, dma_addr_t dst, dma_addr_t src,
        size_t len, unsigned long flags);
    struct dma_async_tx_descriptor *(*device_prep_dma_xor)(
        struct dma_chan *chan, dma_addr_t dst, dma_addr_t *src,
        unsigned int src_cnt, size_t len, unsigned long flags);
    struct dma_async_tx_descriptor *(*device_prep_dma_xor_val)(
        struct dma_chan *chan, dma_addr_t *src, unsigned int src_cnt,
        size_t len, enum sum_check_flags *result, unsigned long flags);

    struct dma_async_tx_descriptor *(*device_prep_dma_pq)(
        struct dma_chan *chan, dma_addr_t *dst, dma_addr_t *src,
        unsigned int src_cnt, const unsigned char *scf,
        size_t len, unsigned long flags);
    struct dma_async_tx_descriptor *(*device_prep_dma_pq_val)(
        struct dma_chan *chan, dma_addr_t *pq, dma_addr_t *src,
        unsigned int src_cnt, const unsigned char *scf, size_t len,
        enum sum_check_flags *pqres, unsigned long flags);
    struct dma_async_tx_descriptor *(*device_prep_dma_memset)(
        struct dma_chan *chan, dma_addr_t dest, int value, size_t len,
        unsigned long flags);
    struct dma_async_tx_descriptor *(*device_prep_dma_memset_sg)(
        struct dma_chan *chan, struct scatterlist *sg,
        unsigned int nents, int value, unsigned long flags);
    struct dma_async_tx_descriptor *(*device_prep_dma_interrupt)(
        struct dma_chan *chan, unsigned long flags);
    struct dma_async_tx_descriptor *(*device_prep_dma_sg)(
        struct dma_chan *chan,
        struct scatterlist *dst_sg, unsigned int dst_nents,
        struct scatterlist *src_sg, unsigned int src_nents,
        unsigned long flags);

    struct dma_async_tx_descriptor *(*device_prep_slave_sg)(
        struct dma_chan *chan, struct scatterlist *sgl,
        unsigned int sg_len, enum dma_transfer_direction direction,
        unsigned long flags, void *context);
```

```
65.     struct dma_async_tx_descriptor *(*device_prep_dma_cyclic)(
66.         struct dma_chan *chan, dma_addr_t buf_addr, size_t buf_len,
67.         size_t period_len, enum dma_transfer_direction direction,
68.         unsigned long flags);
69.     struct dma_async_tx_descriptor *(*device_prep_interleaved_dma)(
70.         struct dma_chan *chan, struct dma_interleaved_template *xt,
71.         unsigned long flags);
72.     struct dma_async_tx_descriptor *(*device_prep_dma_imm_data)(
73.         struct dma_chan *chan, dma_addr_t dst, u64 data,
74.         unsigned long flags);
75.
76.     int (*device_config)(struct dma_chan *chan,
77.                  struct dma_slave_config *config);
78.     int (*device_pause)(struct dma_chan *chan);
79.     int (*device_resume)(struct dma_chan *chan);
80.     int (*device_terminate_all)(struct dma_chan *chan);
81.     void (*device_synchronize)(struct dma_chan *chan);
82.
83.     enum dma_status (*device_tx_status)(struct dma_chan *chan,
84.                          dma_cookie_t cookie,
85.                          struct dma_tx_state *txstate);
86.     void (*device_issue_pending)(struct dma_chan *chan);
87. };
```

channels:链表头。

cap_mask:表示 controller 的传输能力，需要和后面 device_prep_dma_xxx 形式的回调函数对应。常见取值如下:

DMA_MEMCPY:可进行 memory copy。

DMA_MEMSET:可进行 memory set。

DMA_SG:可进行 scatter list 传输。

DMA_CYCLIC:可进行 cyclic 类的传输。

DMA_INTERLEAVE:可进行交叉传输。

src_addr_widths:表示 controller 支持哪些宽度的 src 类型。

dst_addr_widths:表示 controller 支持哪些宽度的 dst 类型。

directions:表示 controller 支持的传输方向取值参考枚举 dma_transfer_direction。

max_burst:最大的 burst 传输的 size。

descriptor_reuse:表示 controller 的传输描述能不能复用。

device_alloc_chan_resources:client 申请 channel 时会调用。

device_free_chan_resources:client 释放 channel 时会调用。

device_prep_dma_xxx:client 通过 dmaengine_prep_xxx 获取传输描述符时会调用。

device_config:client 调用 dmaengine_slave_configchannel 时会调用。

device_pause:client 调用 dmaengine_pause 时会调用。

device_resume:client 调用 dmaengine_resume 时会调用。

device_terminate_all:client 调用 dmaengine_terminate_xxx 时会调用。

device_issue_pending:client 调用 dma_async_issue_pending 启动传输时会调用。

DMA controller 驱动需要实现这些函数的具体处理内容，相当于字符设备框架中的 ops 操作函数。

2) struct dma_chan

定义如下：

```
struct dma_chan {
        struct dma_device *device;
        dma_cookie_t cookie;
        dma_cookie_t completed_cookie;

        /* sysfs */
        int chan_id;
        struct dma_chan_dev *dev;

        struct list_head device_node;
        struct dma_chan_percpu __percpu *local;
        int client_count;
        int table_count;

        /* DMA router */
        struct dma_router *router;
        void *route_data;

        void *private;
};
```

device:指向该 channel 所在的 dma controller。

cookie:client 以该 channel 为操作对象获取传输描述符时，dma controller 返回给 client 的最后一个 cookie。

completed_cookie:在当前 channel 上最后一次完成的传输的 cookie。

device_node:于将该 channel 添加到 dma_device 的 channel 列表中。

3) struct virt_dma_cha

定义在文件 drivers/dma/virt-dma.h 中如下：

```
struct virt_dma_desc {
        struct dma_async_tx_descriptor tx;
        /* protected by vc.lock */
        struct list_head node;
};

struct virt_dma_chan {
        struct dma_chan chan;
        struct tasklet_struct task;
        void (*desc_free)(struct virt_dma_desc *);

        spinlock_t lock;

        /* protected by vc.lock */
        struct list_head desc_allocated;
        struct list_head desc_submitted;
        struct list_head desc_issued;
        struct list_head desc_completed;

        struct virt_dma_desc *cyclic;

};
```

chan:一个 struct dma_chan 类型的变量，用于和 client 交互。

task:一个 tasklet，等待该虚拟 channel 的传输完成。

desc_allocated、desc_submitted、desc_issued、desc_completed:四个链表头，用于保存不同状态的虚拟 channel 描述符。

DMA controller 框架相关 API 如下：

1) struct dma_device 注册和注销

```
int dma_async_device_register(struct dma_device *device);
void dma_async_device_unregister(struct dma_device *device);
```

struct dma_device 变量初始化完成后，调用 dma_async_device_register 向内核注册。注册成功后 dma_device 会放在一个名称为 dma_device_list 的全局链表上，以便后面使用。

dma_async_device_unregister 是相对的注销函数。

2) cookie 相关接口

DMA cookie 表示 DMA engine 在数据传送中使用的一段连续内存。

```
static inline void dma_cookie_init(struct dma_chan *chan)
static inline dma_cookie_t dma_cookie_assign(struct dma_async_tx_descriptor *tx)
static inline void dma_cookie_complete(struct dma_async_tx_descriptor *tx)
static inline enum dma_status dma_cookie_status(struct dma_chan *chan, dma_cookie_t
cookie, struct dma_tx_state *state)
```

dma_cookie_init:初始化 channel 中的 cookie、completed_cookie。

dma_cookie_assign:为指针的传输描述分配一个 cookie。

dma_cookie_complete:当一个传输完成时，可调用该接口更新该传输所对应 channel 的 completed_cookie 字段。

dma_cookie_status:获取指定 channel 指定 cookie 的传输状态。

DMA controller 驱动实例可以参考 pl330.c(zynqmp 参考\drivers\dma\xilinx\zynqmp_dma.c)。大致流程如下：

1) 定义 struct dma_device 变量并初始化、根据硬件实现必要的回调函数。
2) 根据 controller 支持的 channel 数，为每个 channel 定义一个 struct dma_chan 变量并初始化后，将每个 channel 都添加到 dma_device 的 channels 链表中。
3) 调用 dma_async_device_register 向内核注册 dma_device。

16.4.2 DMA client 驱动框架

从源和目标的不同可以把 DMA 划分为四类：内存到内存、内存到外设、外设到内存、外设到外设。因为内存可以使用 memcpy、memset 等操作，linux engine 中把内存到内存这一部分分离出来单独提供了一套 API-Async TX API。而剩余的三类就共用一个结构 Slave-DMA API。这里的 Slave 指代的 client 视角，也就是 DMA 的使用者。

DMA client 框架中主要涉及到的数据结构如下：

1) struct dma_slave_config

定义在 include/linux/dmaengine.h 中如下：

```
1. struct dma_slave_config {
2.         enum dma_transfer_direction direction;
3.         phys_addr_t src_addr;
4.         phys_addr_t dst_addr;
```

```
5.          enum dma_slave_buswidth src_addr_width;
6.          enum dma_slave_buswidth dst_addr_width;
7.          u32 src_maxburst;
8.          u32 dst_maxburst;
9.          bool device_fc;
10.         unsigned int slave_id;
11. };
```

direction:传输方向，取值参考 enum dma_transfer_direction。

src_addr:传输方向是外设到内存或者外设到外设时，读取数据的位置。内存到设备类型的 channel 不需配置该参数。

dst_addr:传输方向是内存到外设或者外设到外设时，写入数据的位置。外设到内存类型的 channel 不需配置该参数。

src_addr_width:src 地址的宽度。

dst_addr_width:dst 地址的宽度。

src_maxburst:src 最大可传输的 burst size，单位是 src_addr_width。

dst_maxburst:dst 最大可传输的 burst size，单位是 dst_addr_width。

slave_id:外设对于 controller 的标识。

2) struct dma_async_tx_descriptor

定义如下：

```
1. struct dma_async_tx_descriptor {
2.          dma_cookie_t cookie;
3.          enum dma_ctrl_flags flags; /* not a 'long' to pack with cookie
    */
4.          dma_addr_t phys;
5.          struct dma_chan *chan;
6.          dma_cookie_t (*tx_submit)(struct dma_async_tx_descriptor *tx);

7.          int (*desc_free)(struct dma_async_tx_descriptor *tx);
8.          dma_async_tx_callback callback;
9.          void *callback_param;
10.         struct dmaengine_unmap_data *unmap;
11.#ifdef CONFIG_ASYNC_TX_ENABLE_CHANNEL_SWITCH
12.         struct dma_async_tx_descriptor *next;
13.         struct dma_async_tx_descriptor *parent;
14.         spinlock_t lock;
15.#endif
16. };
```

cookie:一个整型数，用于追踪本次传输。一般情况下，controller 会在内部维护一个递增的 number，当 client 获取传输描述时都会将该 number 赋给 cookie，然后递增。

flags:enum dma_ctrl_flags 枚举。

chan:对应的 channel。

tx_submit:controller 提供的回调函数，用于把改描述符提交到待传输列表。

desc_free:controller 提供的回调函数，用于释放描述符。

callback:传输完成的回调函数

callback_param:callback 的参数。

使用 Slave-DMA API 的 DMA client 实现步骤大致如下：

1) 申请 DMA channel

```
struct dma_chan *dma_request_chan(struct device *dev, const char *name);
void dma_release_channel(struct dma_chan *chan);
```

dma_request_chan 向内核申请 dma_chan。

dma_release_channel 则是相对的释放 dma_chan。

2) 设置 DMA channel 参数

设置参数使用下面的函数

```
int dmaengine_slave_config(struct dma_chan *chan, struct dma_slave_config *config)
```

首先把配置内容用 struct dma_slave_config 表示，再使用该函数提交配置信息。

3) 获取描述符

在 DMA 启动传输之前，client 需要把传输的信息(src、dst 等)告知 controller，controller 确认后就会返回一个描述符给 client，client 使用这个描述符控制跟踪这次传输。

根据不同的传输类型，使用下面三个接口来获取描述符

```
struct dma_async_tx_descriptor *dmaengine_prep_slave_sg(
        struct dma_chan *chan, struct scatterlist *sgl,
        unsigned int sg_len, enum dma_data_direction direction,
        unsigned long flags);
struct dma_async_tx_descriptor *dmaengine_prep_dma_cyclic(
        struct dma_chan *chan, dma_addr_t buf_addr, size_t buf_len,
        size_t period_len, enum dma_data_direction direction);
struct dma_async_tx_descriptor *dmaengine_prep_interleaved_dma(
        struct dma_chan *chan, struct dma_interleaved_template *xt,
        unsigned long flags);
```

4) 提交启动传输

使用下面的函数提交描述符

```
dma_cookie_t dmaengine_submit(struct dma_async_tx_descriptor *desc)
```

使用下面的函数启动传输

```
void dma_async_issue_pending(struct dma_chan *chan);
```

5) 等待传输结束

等待传输可以通过回调函数，也可以通过 dma_async_is_tx_complete 等函数查询传输是否完成。另外还可以使用 maengine_pause、dmaengine_resume 函数，暂停、终止传输。

client 简单的 demo 如下：

```
1.  #include <linux/dmaengine.h>
2.  #include <linux/dma-mapping.h>
3.  #include <linux/types.h>
4.  #include <linux/slab.h>
5.  #include <linux/module.h>
6.  #include <linux/init.h>
7.  #include <linux/fs.h>
8.  #include <linux/sched.h>
9.  #include <linux/miscdevice.h>
10. #include <linux/device.h>
11. #include <linux/string.h>
12. #include <linux/errno.h>
13. #include <linux/types.h>
14. #include <linux/slab.h>
```

```
15. #include <linux/of_device.h>
16. #include <linux/async_tx.h>
17. #include <asm/uaccess.h>
18. #include <asm/delay.h>
19.
20. #define DEVICE_NAME "ax_dma"
21.
22. #define MAX_SIZE (512*64)
23.
24. static char *src;
25. static char *dst;
26. dma_addr_t dma_src;
27. dma_addr_t dma_dst;
28.
29. struct ax_dma_drv {
30.     struct dma_chan *chan;
31.     struct dma_device *dev;
32.     struct dma_async_tx_descriptor *tx;
33.     enum dma_ctrl_flags flags;
34.     dma_cookie_t cookie;
35. };
36. struct ax_dma_drv ax_dma;
37.
38. void dma_cb(void *dma_async_param)
39. {
40.     if(!memcmp(src, dst, MAX_SIZE))
41.     {
42.         printk("dma irq test ok\r\n");
43.     }
44. }
45.
46. static int dma_open(struct inode *inode, struct file *file)
47. {
48.     printk("dma_open\r\n");
49.     return 0;
50. }
51.
52. static int dma_release(struct inode *indoe, struct file *file)
53. {
54.     printk("dma_release\r\n");
55.     return 0;
56. }
57.
58. static ssize_t dma_read(struct file *filp, char __user *buf, size_t size, loff_t
     *ppos)
59. {
60.     int ret = 0;
61.     printk("dma_read\r\n");
62.
63.     ax_dma.tx = ax_dma.dev->device_prep_dma_memcpy(ax_dma.chan, dma_dst, dma_src,
     MAX_SIZE, ax_dma.flags);
64.     if (!ax_dma.tx){
65.         printk(KERN_INFO "Failed to prepare DMA memcpy");
66.     }
67.
68.     ax_dma.tx->callback = dma_cb;
69.     ax_dma.tx->callback_param = NULL;
70.     ax_dma.cookie = ax_dma.tx->tx_submit(ax_dma.tx);
71.     if (dma_submit_error(ax_dma.cookie)){
72.         printk("DMA tx submit failed");
73.     }
74.     dma_async_issue_pending(ax_dma.chan);
75.
76.     return ret;
77. }
78.
```

```
79. static struct file_operations ax_fops =
80. {
81.     .owner    = THIS_MODULE,
82.     .open     = dma_open,
83.     .read     = dma_read,
84.     .release = dma_release,
85. };
86.
87. static struct miscdevice dma_misc =
88. {
89.     .minor = MISC_DYNAMIC_MINOR,
90.     .name  = DEVICE_NAME,
91.     .fops  = &ax_fops,
92. };
93.
94. static int __init dma_init(void)
95. {
96.     int ret=0;
97.     dma_cap_mask_t mask;
98.
99.     ret = misc_register(&dma_misc);
100.     if(ret)
101.     {
102.         printk("misc_register failed!\n");
103.         return 0;
104.     }
105.     printk("drv register ok\n");
106.     of_dma_configure(dma_misc.this_device, dma_misc.this_device->of_node, true)
    ;
107.     dma_misc.this_device->coherent_dma_mask = 0xffffffff;
108.
109.     //源
110.     src = dma_alloc_coherent(dma_misc.this_device, MAX_SIZE, &dma_src, GFP_KER
    NEL);
111.     if (NULL == src)
112.     {
113.         printk("can't alloc buffer for src\n");
114.         return -ENOMEM;
115.     }
116.     //目标
117.     dst = dma_alloc_coherent(dma_misc.this_device, MAX_SIZE, &dma_dst, GFP_KER
    NEL);
118.     if (NULL == dst)
119.     {
120.         dma_free_coherent(NULL, MAX_SIZE, src, dma_src);
121.         printk("can't alloc buffer for dst\n");
122.         return -ENOMEM;
123.     }
124.     printk("buffer alloc ok\n");
125.
126.     //初始化 mask
127.     dma_cap_zero(mask);
128.     dma_cap_set(DMA_MEMCPY, mask);
129.     ax_dma.chan = dma_request_channel(mask, NULL, NULL);
130.     ax_dma.flags = DMA_CTRL_ACK | DMA_PREP_INTERRUPT;
131.     ax_dma.dev = ax_dma.chan->device;
132.     printk("chan request ok\n");
133.
134.     //给源地址一个初值
135.     memset(src, 0x5A, MAX_SIZE);
136.     //给目标地址一个不一样的初值
137.     memset(dst, 0xA5, MAX_SIZE);
138.
139.     return 0;
140. }
```

```
141.
142. static void __exit dma_exit( void )
143. {
144.     dma_release_channel(ax_dma.chan);
145.     dma_free_coherent(dma_misc.this_device, MAX_SIZE, src, dma_src);
146.     dma_free_coherent(dma_misc.this_device, MAX_SIZE, dst, dma_dst);
147.     misc_deregister(&dma_misc);
148. }
149.
150. //驱动入口函数标记
151. module_init(dma_init);
152. //驱动出口函数标记
153. module_exit(dma_exit);
154.
155. /* 驱动描述信息 */
156. MODULE_AUTHOR("Alinx");
157. MODULE_ALIAS("dma");
158. MODULE_DESCRIPTION("DMA driver");
159. MODULE_VERSION("v1.0");
160. MODULE_LICENSE("GPL");
```

110 行中的 dma_alloc_coherent()函数在新版的内核中不可以再传 NULL 给第一个参数 dev 了。并且需要 106~107 行的代码来初始化传入的 dev 的相关参数。

这个代码可以编译成模块来测试，测试方法就是使用 dma 来拷贝 src 上的数据到 dst 中。应用层只要调用 open 和 read 函数即可。

16.4 实验

用 course2“vitis 应用教程”中的“19_dma_loopback”vivado 工程来制作 petalinux 工程，并且实现一个 dma 回环测试的驱动。

16.4.1 设备树

```
/include/ "system-conf.dtsi"
/ {
     dmatest_0: dmatest@0 {
          compatible ="xlnx,axi-dma-test-1.00.a";
          dmas = <&axi_dma_0 0
               &axi_dma_0 1>;
          dma-names = "axidma0", "axidma1";
     };
};

/* SD */
&sdhci1 {
     disable-wp;
     no-1-8-v;
};

/* USB  */
&dwc3_0 {
     status = "okay";
     dr_mode = "host";
};
```

修改了设备树和硬件描述文件，编译完 petalinux 工程后，打包命令如下：

```
petalinux-package --boot --u-boot --fsbl --force --fpga
```

16.4.2 驱动程序

驱动程序是通过 xilinx 的例程修改的，设备树直接用 petalinux 生成的即可。代码比较长，就不贴出来了，可在配套的源码包中查看，编译好驱动后直接加载就可以看到测试结果。可以

对照下面的 dma 使用流程来理解代码。

dma 使用流程大致如下：

1---> dma_request_chan(TX/RX)

申请 dma 通道。

2-—-> alloc src dst len

分配源地址长度，如果 cpu 支持可以用 dma_alloc_coherent()函数来分配非缓存的地址。

3-—-> flash cache

如果 cpu 不支持分配非缓存的地址，就下面这些函数来刷新缓存。

dma_map_single() / dma map_sg()

dma_sync_single_for_cpu() / dma_sync_single_for_deive()

dma_unmap_single() / dma_unmap_sg()

4-—-> device_prep_slave_sg()

用这个函数来获取一个 dma 操作的描述符

5-—-> dma async tx_descriptor init

初始化描述符

6-—-> tx_submit()

提交 dma 描述符

7-—-> dma_async_issue_pending()

执行 dma 操作

8-—-> wait complete or timewout

等待 dma 操作完成

9-—-> dma_async_is_tx_complete()

确认 dma 操作结果

16.4.3 测试结果

```
root@petlinux:/mnt# insmod ax-dma.ko
[   55.884753] dmatest: Started 1 threads using dma1chan0 dma1chan1
[   55.885222] dma1chan0-dma1c: #0: No errors with
[   55.885225] src_off=0x1168 dst_off=0x126c len=0x199c
[   55.900758] dma1chan0-dma1c: #1: No errors with
[   55.900761] src_off=0x2834 dst_off=0x134c len=0x744
[   55.910692] dma1chan0-dma1c: #2: No errors with
[   55.910694] src_off=0xcec dst_off=0x151c len=0x1b00
[   55.920611] dma1chan0-dma1c: #3: No errors with
[   55.920613] src_off=0x1894 dst_off=0x1a4c len=0x1334
[   55.930591] dma1chan0-dma1c: #4: No errors with
[   55.930593] src_off=0x37c4 dst_off=0x2aa0 len=0x300
[   55.940083] dma1chan0-dma1c: terminating after 5 tests, 0 failures 93 iops 374 KB/s (status 0)
root@petlinux:/mnt#
```